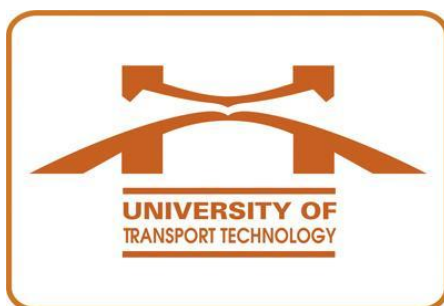


TRƯỜNG ĐẠI HỌC CÔNG NGHỆ GIAO THÔNG VẬN TẢI

KHOA CÔNG NGHỆ THÔNG TIN



ĐỀ TÀI : NGHIÊN CỨU KHOA HỌC SINH VIÊN

NĂM HỌC 2025-2026

Tên đề tài:

“NGHIÊN CỨU XÂY DỰNG HỆ THỐNG HỖ TRỢ, CẢNH BÁO, GIÁM SÁT GIAO THÔNG CÓ SỬ DỤNG TRÍ TUỆ NHÂN TẠO”

Giáo viên hướng dẫn: ĐOÀN THỊ THANH HẰNG

Sinh viên tham gia: ĐÀO VĂN VINH

ĐÀO PHÚC DÂN

NGUYỄN THANH XUÂN

ĐỖ QUANG ANH

HOÀNG VĂN THÀNH

HÀ NỘI 2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ GIAO THÔNG VẬN TẢI

KHOA CÔNG NGHỆ THÔNG TIN



ĐỀ TÀI : NGHIÊN CỨU KHOA HỌC SINH VIÊN

NĂM HỌC 2025-2026

**Tên đề tài: “NGHIÊN CỨU XÂY DỰNG HỆ THỐNG HỖ TRỢ, CẢNH
BÁO, GIÁM SÁT GIAO THÔNG CÓ SỬ DỤNG TRÍ TUỆ NHÂN TẠO”**

Giáo viên hướng dẫn: ĐOÀN THỊ THANH HẰNG

Sinh viên tham gia: ĐÀO VĂN VINH

ĐÀO PHÚC DÂN

NGUYỄN THANH XUÂN

ĐỖ QUANG ANH

HOÀNG VĂN THÀNH

HÀ NỘI 2026

Mục Lục

I. THÔNG TIN CHUNG ĐỀ TÀI	2
1. Tên đề tài	2
2. Giáo viên hướng dẫn	2
3. Sinh viên tham gia	2
II. NỘI DUNG KHOA HỌC VÀ CÔNG NGHỆ CỦA ĐỀ TÀI	3
1. Lý do chọn đề tài	3
2. Tổng quan đề tài.....	4
3.1 Mục tiêu tổng quát	5
3.2 Mục tiêu cụ thể	5
4. Đối tượng và phạm vi nghiên cứu	7
4.1 . Đối tượng nghiên cứu	7
4.1 . Phạm vi nghiên cứu.....	8
5. Phương pháp nghiên cứu.....	9
5.1. Phương pháp nghiên cứu lý thuyết.....	9
5.2. Phương pháp thu thập và xử lý dữ liệu	9
5.3. Phương pháp xây dựng và huấn luyện mô hình	10
5.4. Phương pháp thiết kế và xây dựng hệ thống	10
5.5. Phương pháp thực nghiệm và đánh giá	11
6. Nội dung chính	11
CHƯƠNG I: GIỚI THIỆU VỀ HỆ THỐNG HỆ THỐNG GIÁM SÁT GIAO THÔNG	
ỨNG DỤNG AI.....	13
1.1. Tổng quan về bài toán giám sát giao thông và nhận diện vi phạm	13
1.1.1. Thực trạng và thách thức của giao thông đô thị	13
1.1.2. Vai trò của yếu tố con người	13
1.1.3. Sự chuyển đổi từ giám sát truyền thống sang AI	14
1.2. Tính thiết thực đề tài	14
1.3. Tổng quan các nghiên cứu liên quan.....	15
1.3.1 Các nghiên cứu quốc tế.....	15
1.3.2 Các nghiên cứu trong nước	16
1.4. Cơ sở lý thuyết về trí tuệ nhân tạo:.....	17
1.4.1. Deep Learning & Computer Vision	17
1.4.2. Mạng nơ-ron tích chập (CNN) cho phân loại ảnh.....	18
1.4.3. Mô hình phát hiện đối tượng YOLO (YOLOv8n)	18
1.4.4. Nhận diện hành vi và cử chỉ với Landmark Detection & MediaPipe	19
1.5. Tổng quan công nghệ sử dụng	19
1.5.1 Mô hình nhận diện đối tượng CNN	19
1.5.3 Nhận diện hành vi và cử chỉ với Landmark Detection & MediaPipe	21
1.5.4 Sự kết hợp giữa YOLO – CNN trong hệ thống.....	22
1.6. Khoảng trống nghiên cứu	23

CHƯƠNG II: PHÂN TÍCH PHÁT TRIỂN VÀ THIẾT KẾ HỆ THỐNG	24
2.1. Mục tiêu và yêu cầu hệ thống.....	24
2.1.1. Mục tiêu hệ thống	24
2.1.2. Yêu cầu chức năng.....	25
2.2. Kiến trúc tổng thể hệ thống	26
2.2.1 Phân tích các lớp kiến trúc.....	26
2.2.2 Luồng dữ liệu (Workflow).....	32
2.3. Quy trình xử lý dữ liệu:	33
2.3.1. Thu thập dữ liệu thực tế	33
2.3.2. Tiền xử lý dữ liệu (Preprocessing).....	34
2.3.3. Gán nhãn dữ liệu (Labeling)	34
2.4 PHÁT TRIỂN MÔ HÌNH NHẬN DIỆN	35
2.4.1. Phát triển mô hình phân loại CNN.....	35
2.4.2. Phát triển mô hình YOLO (phát hiện đối tượng)	39
2.5. Pipeline hệ thống nhận diện.....	43
2.6. Thiết kế các mô-đun chức năng:	45
2.6.1. Phân tích điều kiện xác định hành vi (Tài xế, biển báo, vi phạm)	45
2.6.2. Thiết kế cơ sở dữ liệu (MySQL)	56
2.6.3. Thiết Web cảnh báo (giao diện tài xế) và giám sát Dashboard (giao diện quản lý)	56
2.6.4. Thiết kế gọi API từ Grod và dataset xây dựng chatbot	59
2.7. Đánh giá hệ thống và phân tích lỗi.....	61
2.7.1. Ưu điểm	61
2.7.2. Nhược điểm.....	62
CHƯƠNG III: TRIỂN KHAI, THỬ NGHIỆM VÀ ĐÁNH GIÁ HỆ THỐNG	63
3.1 Mục tiêu chương.....	63
3.2. Thiết lập môi trường triển khai, công cụ và thư viện (Python, OpenCV, Flask, MySQL)63	
3.2.1. Khởi tạo mô hình và cấu hình thư viện AI.....	65
3.2.2 Quá trình nhận diện hành vi, vi phạm từ webcam.....	65
3.2.3 Nhận diện giọng nói và phát ra âm thanh cảnh báo	67
3.2.4 Hiện thị kết quả và điều khiển	68
3.3 Xây dựng ứng dụng Web giám sát và quản lý kết quả thời gian thực	68
1. Backend (Flask API).....	69
2. Kết nối Database	69
3. Giao diện realtime.....	70
3.4 Kiểm thử hệ thống	71
3.4.1 Kiểm thử phát hiện vi phạm.....	71
3.4.2 Kiểm thử cảnh báo theo thời gian thực	71
3.4.3 Kiểm thử tính năng cảnh cáo tài xế.....	72
3.4.4 Kiểm thử truy xuất vi phạm	72
3.4.5 Kiểm thử chức năng chatbot	72
CHƯƠNG IV: QUY TRÌNH HOÀN THIỆN VÀ VẬN HÀNH SẢN PHẨM	73

4.1. Lập kế hoạch phát triển sản phẩm.....	73
4.2. Quy trình hoàn thiện hệ thống.....	74
4.2.1. Chuẩn hóa mã nguồn và cấu trúc dự án	74
4.2.2. Tối ưu hiệu năng mô hình và xử lý video	75
4.2.3. Tăng cường độ ổn định và khả năng mở rộng.....	76
4.3. Triển khai thực tế và vận hành thử nghiệm	76
CHƯƠNG V: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN MỞ RỘNG ĐỀ TÀI	78
5.1. Kết luận quả đạt được của đề tài về mức độ hoàn thành.....	78
5.2. Những đóng góp chính ứng dụng trong thực tế.....	79
5.2.1. Xây dựng hệ thống giám sát giao thông ứng dụng AI	79
5.2.2. Tích hợp các mô hình AI: YOLOv8, CNN, MediaPipe.....	79
5.3. Hướng phát triển và mở rộng	80
5.3.1 Cải thiện hiệu năng và tối ưu hệ thống.....	80
5.3.2. Tích hợp vào các hệ thống	82
TÀI LIỆU THAM KHẢO	84

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ GIAO THÔNG VẬN TẢI

KHOA CÔNG NGHỆ THÔNG TIN

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC SINH VIÊN

NĂM HỌC 2025-2026

I. THÔNG TIN CHUNG ĐỀ TÀI

1. Tên đề tài

“ NGHIÊN CỨU XÂY DỰNG HỆ THỐNG HỖ TRỢ, CẢNH BÁO, GIÁM SÁT GIAO THÔNG CÓ SỬ DỤNG TRÍ TUỆ NHÂN TẠO”

2. Giáo viên hướng dẫn

Họ và tên	Đơn vị	Điện thoại	Email
Đoàn Thị Thanh Hằng			

3. Sinh viên tham gia

Họ và tên	Lớp	Điện thoại	Email
Đào Văn Vinh	74DCTT27	0389784619	daovinhgm2005@gmail.com
Nguyễn Thanh Xuân	74DCTT27	0795376983	xuanng205@gmail.com
Đào Phúc Dân	74DCTT27	0862757951	dunghiep6b@gmail.com
Hoàng Văn Thành	74DCTT27	0961191146	thanhkuka72005@gmail.com
Đỗ Quang Anh	74DCTT27	0559217842	doquangah@gmail.com

II. NỘI DUNG KHOA HỌC VÀ CÔNG NGHỆ CỦA ĐỀ TÀI

1. Lý do chọn đề tài

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của nền kinh tế và quá trình đô thị hóa, nhu cầu tham gia giao thông tại Việt Nam ngày càng gia tăng nhanh chóng. Sự gia tăng đột biến về số lượng phương tiện, đặc biệt là ô tô cá nhân và xe vận tải, trong khi hạ tầng giao thông chưa được cải thiện tương xứng, đã dẫn đến nhiều vấn đề nghiêm trọng như ùn tắc giao thông kéo dài, ô nhiễm môi trường và đặc biệt là tai nạn giao thông ngày càng gia tăng cả về số lượng lẫn mức độ nghiêm trọng[1].

Theo nhiều nghiên cứu và thống kê thực tế, nguyên nhân chủ yếu dẫn đến tai nạn giao thông không chỉ xuất phát từ yếu tố hạ tầng mà còn đến từ hành vi của người điều khiển phương tiện[2]. Các hành vi nguy hiểm như mất tập trung khi lái xe, sử dụng điện thoại, không tuân thủ quy định an toàn, buồn ngủ hoặc thiếu ý thức chấp hành luật giao thông đang diễn ra phổ biến và tiềm ẩn nhiều rủi ro. Điều này đặt ra yêu cầu cấp thiết về việc xây dựng các giải pháp có khả năng giám sát và hỗ trợ người lái xe một cách hiệu quả và chủ động hơn.

Đặc biệt, đối với các doanh nghiệp vận tải, đội ngũ lái xe đường dài, các công ty logistics hay các đơn vị quản lý phương tiện dịch vụ, bài toán đảm bảo an toàn giao thông và quản lý đội xe trở nên phức tạp và mang tính chiến lược. Trong môi trường vận hành với quy mô lớn, mỗi doanh nghiệp có thể phải quản lý đồng thời hàng chục đến hàng trăm phương tiện hoạt động liên tục trên nhiều tuyến đường khác nhau. Mỗi một sự cố xảy ra không chỉ gây nguy hiểm đến tính mạng con người mà còn kéo theo những thiệt hại nặng nề về tài sản, chi phí bồi thường, gián đoạn hoạt động kinh doanh và ảnh hưởng tiêu cực đến uy tín thương hiệu.

Tuy nhiên, các phương pháp quản lý truyền thống hiện nay như giám sát thủ công, báo cáo định kỳ hoặc các thiết bị theo dõi đơn lẻ vẫn còn nhiều hạn chế. Những phương pháp này thường mang tính bị động, không cung cấp thông tin theo thời gian thực và không có khả năng phát hiện sớm các hành vi nguy hiểm. Điều này khiến doanh

ngành chủ yếu chỉ có thể xử lý hậu quả sau khi sự cố xảy ra, thay vì chủ động phòng ngừa rủi ro.

2. Tổng quan đề tài

Trong bối cảnh đó, sự phát triển vượt bậc của trí tuệ nhân tạo (Artificial Intelligence – AI), đặc biệt trong các lĩnh vực như thị giác máy tính (Computer Vision), học sâu (Deep Learning) và xử lý ngôn ngữ tự nhiên (Natural Language Processing), đã mở ra những hướng tiếp cận mới trong việc xây dựng các hệ thống giao thông thông minh[3]. Việc ứng dụng AI cho phép hệ thống có khả năng phân tích dữ liệu hình ảnh và hành vi theo thời gian thực, nhận diện các nguy cơ tiềm ẩn và đưa ra cảnh báo kịp thời, từ đó góp phần giảm thiểu tai nạn và nâng cao hiệu quả quản lý.

Xuất phát từ những vấn đề thực tiễn nêu trên, đề tài “Xây dựng hệ thống giám sát, cảnh báo và hỗ trợ lái xe an toàn, quản lý giao thông ứng dụng trí tuệ nhân tạo” được đề xuất nhằm phát triển một giải pháp tích hợp, có khả năng kết hợp nhiều công nghệ AI tiên tiến như mạng nơ-ron tích chập (CNN), mô hình phát hiện đối tượng YOLOv8n, nhận diện khuôn mặt và nhận diện giọng nói [4]. Hệ thống không chỉ thực hiện chức năng giám sát hành vi lái xe và môi trường giao thông mà còn đóng vai trò như một trợ lý lái xe thông minh, hỗ trợ người dùng trong quá trình điều khiển phương tiện.

Bên cạnh đó, hệ thống còn hướng tới việc xây dựng một nền tảng quản lý giao thông thông minh, cho phép các doanh nghiệp và cơ quan quản lý theo dõi, phân tích và đánh giá dữ liệu một cách toàn diện. Qua đó, đề tài không chỉ mang ý nghĩa nghiên cứu trong lĩnh vực công nghệ thông tin và trí tuệ nhân tạo mà còn có giá trị ứng dụng thực tiễn cao, góp phần nâng cao an toàn giao thông và hướng tới phát triển hệ thống giao thông thông minh trong tương lai.

3. Mục tiêu đề tài

3.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống ứng dụng trí tuệ nhân tạo có khả năng giám sát hành vi người lái xe và phân tích môi trường giao thông theo thời gian thực, từ đó đưa ra các cảnh báo kịp thời nhằm nâng cao an toàn giao thông, giảm thiểu nguy cơ tai nạn và hỗ trợ hiệu quả công tác quản lý phương tiện. Để đạt được mục tiêu này, hệ thống hướng tới việc tích hợp các công nghệ nhận diện hiện đại như nhận diện khuôn mặt, ánh mắt, cử chỉ và xử lý lệnh thoại tiếng Việt nhằm phát hiện sớm các hành vi nguy hiểm và hỗ trợ tương tác thông minh với người lái. Đồng thời, đề tài tập trung tối ưu hóa trải nghiệm lái xe thông qua việc cung cấp các cảnh báo trực quan, chính xác và kịp thời, góp phần nâng cao ý thức tham gia giao thông. Bên cạnh đó, hệ thống còn được phát triển theo định hướng xây dựng một nền tảng giám sát và phân tích giao thông thông minh, cho phép theo dõi vị trí, lưu lượng phương tiện, phát hiện các bất thường và tích hợp chatbot trí tuệ nhân tạo hỗ trợ tư vấn luật giao thông, từ đó nâng cao hiệu quả quản lý và hướng tới phát triển hệ sinh thái giao thông hiện đại, an toàn và bền vững.

3.2 Mục tiêu cụ thể

3.2.1. Phát hiện và cảnh báo hành vi nguy hiểm của tài xế

Một trong những mục tiêu trọng tâm của đề tài là phát triển hệ thống có khả năng nhận diện các hành vi nguy hiểm trong quá trình lái xe thông qua việc phân tích hình ảnh từ camera. Các hành vi cần được phát hiện bao gồm buồn ngủ, mất tập trung, sử dụng điện thoại, không thắt dây an toàn hoặc điều khiển phương tiện không đúng tư thế.

Hệ thống sẽ sử dụng các mô hình trí tuệ nhân tạo để xử lý dữ liệu hình ảnh theo thời gian thực và đưa ra cảnh báo kịp thời thông qua âm thanh hoặc giao diện hiển thị. Điều này giúp tài xế nhanh chóng nhận biết và điều chỉnh hành vi, từ đó giảm thiểu nguy cơ xảy ra tai nạn giao thông.

3.2.2. Nhận diện và phân tích môi trường giao thông

Đề tài hướng tới việc xây dựng khả năng nhận diện các yếu tố trong môi trường giao thông như phương tiện, biển báo, làn đường và chướng ngại vật. Việc phân tích chính xác các yếu tố này giúp hệ thống hỗ trợ tài xế đưa ra quyết định phù hợp trong các tình huống phức tạp.

Thông qua việc ứng dụng các mô hình phát hiện đối tượng tiên tiến, hệ thống có thể đánh giá tình trạng giao thông xung quanh và cung cấp các cảnh báo sớm về nguy cơ va chạm hoặc vi phạm luật giao thông.

3.2.3. Xây dựng hệ thống giám sát và quản lý giao thông thông minh

Một mục tiêu quan trọng khác là phát triển hệ thống có khả năng giám sát nhiều phương tiện cùng lúc và cung cấp công cụ quản lý hiệu quả cho doanh nghiệp. Hệ thống cho phép theo dõi vị trí phương tiện theo thời gian thực thông qua GPS, lưu trữ dữ liệu hành vi lái xe và phân tích các vi phạm giao thông.

Nhờ đó, các doanh nghiệp vận tải có thể đánh giá hiệu suất hoạt động của tài xế, phát hiện các rủi ro tiềm ẩn và đưa ra các biện pháp quản lý phù hợp nhằm nâng cao hiệu quả vận hành.

3.2.4. Nâng cao trải nghiệm và khả năng tương tác của người dùng

Bên cạnh các chức năng giám sát, hệ thống còn hướng tới việc cải thiện trải nghiệm người dùng thông qua việc tích hợp các công nghệ hiện đại như điều khiển bằng giọng nói và chatbot hỗ trợ. Người dùng có thể tương tác với hệ thống một cách thuận tiện mà không làm ảnh hưởng đến quá trình lái xe.

Giao diện hệ thống được thiết kế trực quan, dễ sử dụng, giúp người dùng dễ dàng tiếp cận và khai thác các chức năng của hệ thống một cách hiệu quả.

3.2.5. Hướng tới phát triển hệ sinh thái giao thông thông minh

Trong dài hạn, đề tài đặt mục tiêu phát triển hệ thống thành một phần của hệ sinh thái giao thông thông minh. Dữ liệu thu thập từ hệ thống có thể được sử dụng để phân tích lưu lượng giao thông, dự đoán tình trạng ùn tắc và hỗ trợ các cơ quan chức năng trong việc điều phối giao thông[5]. Việc tích hợp với các hệ thống dữ liệu lớn và nền tảng đô thị thông minh sẽ góp phần xây dựng một môi trường giao thông hiện đại, an toàn và bền vững.

4. Đối tượng và phạm vi nghiên cứu

4.1 . Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài tập trung vào việc ứng dụng các công nghệ trí tuệ nhân tạo trong lĩnh vực giao thông nhằm nâng cao an toàn khi tham gia giao thông và tối ưu hóa công tác quản lý phương tiện.

Cụ thể, đề tài hướng đến các đối tượng chính sau:

- Theo dõi vào các hành vi của người lái xe trong quá trình điều khiển phương tiện. Đây là yếu tố then chốt ảnh hưởng trực tiếp đến an toàn giao thông. Các hành vi như buồn ngủ, mất tập trung, sử dụng điện thoại, không tuân thủ quy định an toàn sẽ được hệ thống thu thập, phân tích và nhận diện thông qua dữ liệu hình ảnh từ camera.

- Quan sát môi trường giao thông xung quanh phương tiện, bao gồm các yếu tố như phương tiện khác, biển báo giao thông, làn đường và chướng ngại vật, ổ voi, ổ gà, vũng nước.... Việc nghiên cứu và nhận diện chính xác các yếu tố này giúp hệ thống có khả năng đánh giá tình huống giao thông và hỗ trợ người lái đưa ra quyết định phù hợp.

- Sử dụng các mô hình và kỹ thuật trí tuệ nhân tạo được áp dụng trong hệ thống, bao gồm các phương pháp xử lý ảnh, nhận diện đối tượng và phân tích hành vi. Đây là nền tảng cốt lõi giúp hệ thống hoạt động hiệu quả trong việc giám sát và cảnh báo theo thời gian thực.

- Xây dựng một hệ thống giám sát và quản lý phương tiện, bao gồm việc theo dõi vị trí, trạng thái hoạt động của xe và hành vi của tài xế. Đối tượng này hướng đến các doanh nghiệp vận tải, logistics và các đơn vị quản lý đội xe.

Như vậy, đối tượng nghiên cứu của đề tài không chỉ dừng lại ở công nghệ mà còn bao gồm cả con người, môi trường và hệ thống quản lý, tạo nên một cách tiếp cận toàn diện trong bài toán giao thông thông minh.

4.1 . Phạm vi nghiên cứu

Thời gian nghiên cứu: Từ tháng 01 đến tháng 04 năm 2026.

Do giới hạn về thời gian, nguồn lực và điều kiện triển khai, đề tài được xác định trong một phạm vi nghiên cứu cụ thể nhằm đảm bảo tính khả thi và hiệu quả.

Về phạm vi nội dung, đề tài tập trung vào việc xây dựng hệ thống giám sát và cảnh báo hỗ trợ lái xe an toàn dựa trên công nghệ trí tuệ nhân tạo. Các chức năng chính bao gồm nhận diện hành vi tài xế, phát hiện các yếu tố trong môi trường giao thông và đưa ra cảnh báo theo thời gian thực. Đề tài không đi sâu vào việc phát triển phần cứng chuyên dụng mà chủ yếu sử dụng các thiết bị phổ biến như camera và máy tính để triển khai hệ thống.

Về phạm vi không gian, hệ thống được thiết kế để hoạt động trong môi trường giao thông đường bộ tại Việt Nam, bao gồm khu vực đô thị, cao tốc, bến xe và đặc biệt là trung tâm điều hành . Dữ liệu sử dụng trong quá trình huấn luyện và mô hình cũng được thu thập hoặc xây dựng dựa trên điều kiện giao thông thực tế tại Việt Nam nhằm đảm bảo tính phù hợp và độ chính xác của hệ thống.

Về phạm vi đối tượng sử dụng, đề tài hướng đến hai nhóm chính là người lái xe cá nhân và các doanh nghiệp vận tải xe khách, xe tải, xe bus, logistics, cơ quan quản lý giao thông. Hệ thống có thể áp dụng cho các loại phương tiện như ô tô cá nhân, xe tải, xe khách hoặc xe dịch vụ.

Về phạm vi công nghệ, đề tài tập trung vào việc ứng dụng các mô hình trí tuệ nhân tạo như học sâu (Deep Learning), thị giác máy tính (Computer Vision) và nhận

diện giọng nói. Các mô hình được sử dụng ở mức độ phù hợp với yêu cầu bài toán và khả năng triển khai thực tế, chưa mở rộng sang các công nghệ nâng cao như hệ thống tự lái hoàn toàn (Autonomous Driving).

Về phạm vi thời gian và triển khai, hệ thống được xây dựng và thử nghiệm ở mức mô hình (prototype), tập trung vào việc chứng minh tính khả thi của giải pháp. Các chức năng nâng cao như triển khai quy mô lớn, tích hợp hệ thống thành phố thông minh hoặc xử lý dữ liệu lớn trong thời gian tới sẽ định hướng phát triển phạm vi nghiên cứu của đề tài.

5. Phương pháp nghiên cứu

Để thực hiện đề tài “Xây dựng hệ thống giám sát, cảnh báo và hỗ trợ lái xe an toàn, quản lý giao thông ứng dụng trí tuệ nhân tạo”, nhóm nghiên cứu đã áp dụng kết hợp nhiều phương pháp nghiên cứu khác nhau, bao gồm cả phương pháp lý thuyết và thực nghiệm, nhằm đảm bảo tính khoa học, tính khả thi và hiệu quả của hệ thống.

5.1. Phương pháp nghiên cứu lý thuyết

Trước hết, nhóm tiến hành thu thập, tổng hợp và phân tích các tài liệu liên quan đến lĩnh vực trí tuệ nhân tạo, thị giác máy tính và các hệ thống giao thông thông minh. Các tài liệu này bao gồm sách chuyên ngành, bài báo khoa học, tài liệu kỹ thuật và các nguồn học liệu trực tuyến.

Thông qua quá trình nghiên cứu lý thuyết, nhóm đã nắm được các nguyên lý hoạt động của các mô hình như mạng nơ-ron tích chập (CNN), các thuật toán phát hiện đối tượng (YOLO), cũng như các kỹ thuật xử lý ảnh và nhận diện hành vi với các thư viện Mediapipe, Landmark Detection[6]. Đây là cơ sở quan trọng để lựa chọn giải pháp công nghệ phù hợp và xây dựng hệ thống một cách hiệu quả.

5.2. Phương pháp thu thập và xử lý dữ liệu

Dữ liệu đóng vai trò quan trọng trong việc huấn luyện và đánh giá các mô hình trí tuệ nhân tạo. Trong đề tài này, dữ liệu được thu thập từ nhiều nguồn khác nhau, bao gồm các bộ dữ liệu công khai và dữ liệu thực tế do nhóm tự xây dựng.

Sau khi thu thập, dữ liệu được tiến hành gán nhãn thủ công nhằm xác định các đối tượng và hành vi cần nhận diện. Bên cạnh đó, nhóm cũng áp dụng các kỹ thuật tăng cường dữ liệu (data augmentation) nhằm cải thiện độ đa dạng của dữ liệu bằng cách

5.3. Phương pháp xây dựng và huấn luyện mô hình

Nhóm sử dụng các mô hình học sâu để giải quyết bài toán nhận diện và phân tích dữ liệu hình ảnh. Cụ thể, các mô hình như CNN và YOLO được áp dụng để phát hiện đối tượng và nhận diện hành vi trong thời gian thực.

Quá trình huấn luyện mô hình được thực hiện theo các bước:

- Chia dữ liệu thành tập huấn luyện bao gồm: (70% cho bộ train), (20% cho bộ validation), (10% cho bộ test)
- Tiến hành huấn luyện mô hình với các tham số phù hợp và thay đổi các tham số làm sao để có được kết quả và độ chính xác tốt nhất
- Đánh giá hiệu quả mô hình thông qua các chỉ số như Precision, Recall, F1-score[7].

Phương pháp này giúp đảm bảo mô hình đạt độ chính xác cao và có khả năng hoạt động ổn định khi triển khai trong các điều kiện thực tế.

5.4. Phương pháp thiết kế và xây dựng hệ thống

Đề tài áp dụng phương pháp thiết kế hệ thống theo mô hình phân lớp (Client–Server), trong đó hệ thống được chia thành các thành phần như thu thập dữ liệu, xử lý AI và hiển thị kết quả.

Nhóm sử dụng các công cụ và framework như Flask và các thư viện xử lý ảnh để xây dựng hệ thống. Các mô hình AI sau khi huấn luyện được tích hợp vào hệ thống nhằm xử lý dữ liệu theo thời gian thực và đưa ra cảnh báo kịp thời.

Phương pháp này giúp hệ thống có cấu trúc rõ ràng, dễ mở rộng và thuận tiện trong quá trình phát triển.

5.5. Phương pháp thực nghiệm và đánh giá

Sau khi xây dựng hệ thống, nhóm tiến hành thử nghiệm trong các điều kiện mô phỏng và thực tế nhằm đánh giá hiệu quả hoạt động.

Các tiêu chí đánh giá bao gồm:

- Độ chính xác trong việc nhận diện hành vi và đối tượng
- Tốc độ xử lý và phản hồi của hệ thống
- Khả năng hoạt động ổn định trong thời gian thực

Kết quả thực nghiệm được sử dụng để điều chỉnh và tối ưu hệ thống, đảm bảo đáp ứng yêu cầu đề ra.

6. Nội dung chính

Chương 1: Giới thiệu về Hệ thống giám sát Giao thông ứng dụng AI

- Tổng quan bài toán: Phân tích thực trạng tai nạn giao thông, áp lực hạ tầng đô thị và sự cần thiết của việc giám sát tự động.
- Tính cấp thiết của đề tài: Giải quyết vấn đề an toàn cho tài xế đường dài, quản lý đội xe doanh nghiệp và nâng cao ý thức tham gia giao thông.
- Tổng quan công nghệ sử dụng: Giới thiệu sơ lược về các công nghệ cốt lõi như CNN (phân loại), YOLOv8 (phát hiện đối tượng), MediaPipe & Dlib (nhận diện hành vi), EasyOCR & gTTS (xử lý văn bản và âm thanh).

Chương 2: Phân tích, Phát triển và Thiết kế Hệ thống

- Mục tiêu và yêu cầu hệ thống: Xác định các mục tiêu tổng quát, mục tiêu cụ thể và các yêu cầu chức năng (phát hiện hành vi, nhận diện môi trường, cảnh báo thông minh).
- Kiến trúc tổng thể: Mô tả các lớp kiến trúc bao gồm lớp thu nhận dữ liệu, lớp xử lý AI cốt lõi và lớp hiển thị/tương tác.
- Quy trình xử lý dữ liệu: Thu thập dữ liệu thực tế tại Việt Nam, tiền xử lý (chuẩn hóa, làm sạch) và gán nhãn dữ liệu theo chuẩn YOLO/CNN.
- Phát triển các mô hình nhận diện:
 - Xây dựng mạng nơ-ron tích chập (CNN) để phân loại chi tiết đối tượng.

- Huấn luyện mô hình YOLOv8n để phát hiện phương tiện, biển báo, vật cản.
 - Ứng dụng MediaPipe và Landmark Detection để phân tích cử chỉ, tư thế và trạng thái tài xế (buồn ngủ, mất tập trung).
- Thiết kế cơ sở dữ liệu và API: Xây dựng cấu trúc MySQL và tích hợp Groq API cho Chatbot tư vấn luật.

Chương 3: Triển khai, Thử nghiệm và Đánh giá Hệ thống

- Thiết lập môi trường triển khai: Cài đặt ngôn ngữ Python và các thư viện hỗ trợ (OpenCV, Flask, PyTorch, MySQL, MediaPipe...).
- Triển khai chạy thực tế từ webcam: Quy trình nhận diện hành vi, vi phạm và phát cảnh báo âm thanh thời gian thực từ luồng video trực tiếp.
- Xây dựng ứng dụng Web (Dashboard): Triển khai giao diện realtime cho tài xế và bảng điều khiển quản lý cho doanh nghiệp.
- Kết quả thử nghiệm và đo lường: Đánh giá độ chính xác (mAP, Precision, Recall), tốc độ xử lý (FPS) và độ trễ cảnh báo trong các điều kiện môi trường khác nhau.

Chương 4: Quy trình Hoàn thiện và Vận hành Sản phẩm

- Quy trình phát triển sản phẩm (SDLC): Từ lập kế hoạch, thu thập dữ liệu đến huấn luyện và đóng gói sản phẩm (mô hình Prototype).
- Tối ưu hóa hiệu năng: Áp dụng kỹ thuật Transfer Learning (Fine-tuning, Freezing layers) cho mô hình và Frame Skipping trong xử lý video để tăng tốc độ thực thi.
- Tăng cường độ ổn định: Bổ sung cơ chế xử lý lỗi, tối ưu pipeline để tránh tắc nghẽn dữ liệu và đảm bảo hệ thống vận hành liên tục.
- Hướng dẫn vận hành: Quy trình cài đặt, cấu hình phần cứng và khởi chạy hệ thống.

Chương 5: Kết luận và Hướng phát triển mở rộng

- Kết luận kết quả đạt được: Đánh giá mức độ hoàn thành so với mục tiêu ban đầu (đạt khoảng 85-90%) và những đóng góp chính của đề tài.
- Hạn chế của hệ thống: Phân tích sự phụ thuộc vào phần cứng (GPU) và độ ổn định trong điều kiện ánh sáng yếu hoặc che khuất.
- Hướng phát triển mở rộng:
 - Nâng cấp mô hình lên YOLOv10/v11, tối ưu bằng TensorRT.
 - Tích hợp Edge AI (NVIDIA Jetson) và hệ sinh thái Đô thị thông minh (iHanoi).
 - Mở rộng đối tượng nhận diện (biển số xe, vạch kẻ đường phức tạp) và kết nối với các thiết bị ADAS chuyên dụng.

CHƯƠNG I: GIỚI THIỆU VỀ HỆ THỐNG GIÁM SÁT GIAO THÔNG ỨNG DỤNG AI

1.1. Tổng quan về bài toán giám sát giao thông và nhận diện vi phạm

1.1.1. Thực trạng và thách thức của giao thông đô thị

Quá trình đô thị hóa nhanh tại các thành phố lớn như Hà Nội và TP. Hồ Chí Minh làm gia tăng mạnh nhu cầu di chuyển, trong khi năng lực hạ tầng giao thông chưa mở rộng tương ứng[8].

Sự mất cân đối giữa tốc độ tăng phương tiện và khả năng đáp ứng của hệ thống đường bộ dẫn đến hai hệ quả chính.

Thứ nhất, tình trạng ùn tắc diễn ra thường xuyên, đặc biệt trong khung giờ cao điểm, làm gia tăng thời gian di chuyển, chi phí nhiên liệu và phát thải môi trường.

Thứ hai, áp lực giao thông cao làm tăng nguy cơ tai nạn và mức độ nghiêm trọng của sự cố, gây tổn thất đáng kể về con người và kinh tế - xã hội. Bối cảnh này đặt ra yêu cầu cấp thiết đối với các giải pháp giám sát và cảnh báo có khả năng hoạt động liên tục, độ phủ lớn và phản hồi theo thời gian thực.

1.1.2. Vai trò của yếu tố con người

Nhiều nghiên cứu thực nghiệm cho thấy yếu tố con người là một trong các nguyên nhân chi phối rủi ro tai nạn giao thông. Bên cạnh các yếu tố khách quan như hạ tầng hoặc mật độ phương tiện, hành vi điều khiển phương tiện thiếu an toàn có ảnh hưởng trực tiếp đến xác suất phát sinh sự cố.

Các hành vi rủi ro thường gặp gồm: mất tập trung khi lái xe, sử dụng điện thoại, không thắt dây an toàn, buồn ngủ hoặc mệt mỏi, điều khiển phương tiện sai làn, vượt quá tốc độ và vi phạm tín hiệu giao thông. Đặc điểm chung của các hành vi này là xuất hiện nhanh, khó giám sát thủ công liên tục và dễ bị bỏ sót nếu chỉ dựa vào quan sát

bằng con người. Vì vậy, giám sát hành vi người lái theo thời gian thực trở thành một thành phần trọng tâm trong các hệ thống giao thông thông minh.

1.1.3. Sự chuyển đổi từ giám sát truyền thống sang AI

Các mô hình giám sát truyền thống, dựa chủ yếu vào lực lượng điều tiết trực tiếp hoặc tổng hợp báo cáo định kỳ, bộc lộ hạn chế về tính liên tục, độ bao phủ và tốc độ phản ứng. Trong bối cảnh lưu lượng phương tiện lớn, cách tiếp cận này khó đáp ứng yêu cầu phát hiện sớm và xử lý kịp thời các hành vi nguy hiểm.

Sự phát triển của trí tuệ nhân tạo, đặc biệt là thị giác máy tính và học sâu, cho phép chuyển dịch sang mô hình giám sát chủ động và tự động hóa cao. Trên hạ tầng đô thị, camera AI hỗ trợ nhận diện vi phạm như vượt đèn đỏ, sai làn, dừng đỗ không đúng quy định và hỗ trợ xử lý theo cơ chế “phạt nguội”. Trong cabin, các hệ thống giám sát người lái (DMS) ứng dụng nhận diện điểm mốc khuôn mặt và các chỉ số hành vi để phát hiện sớm trạng thái buồn ngủ hoặc mất tập trung. Ở quy mô điều hành, dữ liệu từ camera và cảm biến có thể được khai thác để tối ưu chu kỳ đèn tín hiệu, giảm thời gian chờ và hỗ trợ dự báo điểm ùn tắc.

Từ các phân tích trên, có thể thấy việc ứng dụng AI trong giám sát giao thông không chỉ là xu hướng công nghệ, mà là yêu cầu thực tiễn nhằm nâng cao an toàn, hiệu quả vận hành và năng lực quản lý giao thông đô thị.

1.2. Tính thiết thực đề tài

Trong bối cảnh đô thị hóa nhanh và số lượng phương tiện giao thông ngày càng gia tăng, bài toán giám sát giao thông và nhận diện vi phạm trở nên cấp thiết. Các hành vi vi phạm như sử dụng điện thoại khi lái xe, không thắt dây an toàn, mất tập trung hoặc vi phạm biển báo là những nguyên nhân chính dẫn đến tai nạn giao thông.

Các phương pháp giám sát truyền thống chủ yếu dựa vào con người hoặc hệ thống camera đơn giản, còn hạn chế về độ chính xác và khả năng xử lý theo thời gian thực. Do đó, việc ứng dụng trí tuệ nhân tạo, đặc biệt là các kỹ thuật học sâu và thị giác

máy tính, cho phép tự động phát hiện và phân tích hành vi vi phạm từ dữ liệu hình ảnh/video một cách hiệu quả hơn.

Bên cạnh đó, để khai thác hiệu quả dữ liệu từ hệ thống AI, việc xây dựng một nền tảng quản lý tập trung là cần thiết. Hệ thống quản lý giúp tổng hợp, hiển thị và theo dõi các thông tin quan trọng như phương tiện, vi phạm và trạng thái cảnh báo theo thời gian thực. Nhờ đó, nhà quản lý có thể quan sát tổng thể hệ thống, phân tích xu hướng và đưa ra quyết định kịp thời.

Sự kết hợp giữa trí tuệ nhân tạo và nền tảng hệ thống không chỉ nâng cao khả năng giám sát mà còn góp phần xây dựng hệ thống giao thông thông minh, hiện đại và có khả năng ứng dụng thực tế cao.

1.3. Tổng quan các nghiên cứu liên quan

1.3.1 Các nghiên cứu quốc tế

Trí tuệ nhân tạo (AI) đã được ứng dụng rộng rãi trên thế giới để giải quyết các bài toán giao thông phức tạp, tập trung vào hai mảng chính là quản lý lưu lượng và giám sát an toàn[9]:

Dự báo và quản lý lưu lượng: Nhóm nhà nghiên cứu của UNIST (Hàn Quốc) phối hợp với Đại học Purdue và Arizona (Hoa Kỳ) đã phát triển hệ thống sử dụng mô hình LSTM (Bộ nhớ ngắn hạn dài) có khả năng dự báo tình trạng tắc giao thông trong 5-15 phút với tỷ lệ lỗi cực thấp ($<4\text{km/h}$). Các thành phố lớn như London (Anh) sử dụng hệ thống UTMIC để tối ưu hóa đèn tín hiệu, trong khi Dubai (UAE) tích hợp AI để giám sát thời gian thực hiện và phát hiện sự cố tự động,. Singapore cũng nổi tiếng với hệ thống Giao thông Thông minh (ITS) tiên tiến nhờ sự đầu tư mạnh mẽ vào hạ tầng cảm biến và mạng lưới camera,.

Nhận diện và giám sát an toàn: Các nghiên cứu về nhận diện nước đường đã đạt được bước tiến lớn với mô hình LD-CAM (Phát hiện làn đường với Cơ chế chú ý chuyển đổi), đạt độ chính xác 97,9% ngay cả trên những giai đoạn hư hỏng hoặc điều kiện thời tiết đường khắc sâu,.

Về giám sát tài xế, các bài đánh giá từ năm 2019-2025 cho thấy công việc kết hợp camera với học sâu (Deep Learning) như Vision Transformers (ViT) và YOLOv8 có thể đưa ra độ chính xác nhận diện giấc ngủ và mất tập trung lên tới 99,15% - 99,94%,.

1.3.2 Các nghiên cứu trong nước

Tại Việt Nam, việc ứng dụng AI trong giao thông đang chuyển dịch mạnh mẽ từ nghiên cứu sang triển khai thực tế tại các đô thị lớn:

Triển khai camera AI đô thị: Thành phố Hà Nội đã chính thức vận hành hơn 1.800 camera AI để tự động phát hiện vi phạm (vượt đèn đỏ, đi sai làn) và phân tích lưu lượng để điều khiển đèn tín hiệu từ xa,. Tại TP. Hồ Chí Minh, công nghệ AI được thí điểm để điều chỉnh pha đèn linh hoạt tại các nút giao trọng điểm như Hàng Xanh và Ngã năm Đài Liệt Sỹ dựa trên dữ liệu mật độ thực tế,[10]....

Các giải pháp từ doanh nghiệp và viện nghiên cứu:

- BA GPS đã phát triển thành công giải pháp BA-AI ứng dụng hệ thống DMS, sử dụng AI để phát hiện và cảnh báo tài xế buồn ngủ, sử dụng điện thoại hoặc hút thuốc khi lái xe,.
- Hệ thống TriS Data Analytics ứng dụng Big Data và học sâu để dự báo xu hướng biến đổi giao thông tại TP.HCM, Tây Ninh và Đồng Tháp,.
- Các nhà nghiên cứu tại Đại học Cần Thơ đã xây dựng hệ thống giám sát trạng thái đèn giao thông sử dụng YOLOv8, đạt độ chính xác 83% và tích hợp ứng dụng Android để truyền dữ liệu từ xa,.
- Nhóm tác giả từ Đại học Hàng hải Việt Nam đã đề xuất mô hình kết hợp YOLO và DeepSORT để theo dõi quỹ đạo phương tiện với độ chính xác nhận diện đạt 94%, hỗ trợ phân tích luồng di chuyển tại các nút giao thông,,.
- Đại học Bách khoa cũng triển khai hệ thống AI giám sát hành trình và điểm danh tự động cho hơn 4.000 học sinh mỗi ngày thông qua camera nhận dạng và GPS.

1.4. Cơ sở lý thuyết về trí tuệ nhân tạo:

1.4.1. Deep Learning & Computer Vision

a. Deep Learning

Học sâu (Deep Learning) là một nhánh quan trọng của học máy (Machine Learning), cho phép máy tính tự động học và trích xuất các đặc trưng từ dữ liệu thông qua các mô hình mạng nơ-ron nhiều lớp[11].

Khác với các phương pháp truyền thống yêu cầu thiết kế đặc trưng thủ công, các mô hình Deep Learning có khả năng học các biểu diễn dữ liệu ở nhiều mức độ trừu tượng khác nhau. Điều này đặc biệt hiệu quả trong các bài toán xử lý ảnh và video, nơi dữ liệu có tính phức tạp và biến động cao.

Trong lĩnh vực giao thông thông minh, Deep Learning được ứng dụng rộng rãi để:

- Nhận diện đối tượng (xe, người, biển báo)
- Phân tích hành vi (buồn ngủ, mất tập trung)
- Dự đoán tình huống giao thông

Nhờ khả năng học từ dữ liệu lớn, các mô hình Deep Learning giúp hệ thống đạt độ chính xác cao và thích nghi tốt với môi trường thực tế.

b. Computer Vision

Computer Vision (Thị giác máy tính) là lĩnh vực trong trí tuệ nhân tạo cho phép máy tính có khả năng “nhìn” và hiểu nội dung của hình ảnh hoặc video. Mục tiêu của Computer Vision là mô phỏng khả năng quan sát và nhận thức của con người thông qua việc phân tích dữ liệu hình ảnh.

Khác với việc chỉ xử lý ảnh ở mức cơ bản, Computer Vision kết hợp các thuật toán học máy và học sâu để trích xuất thông tin có ý nghĩa từ dữ liệu thị giác. Điều này

giúp hệ thống không chỉ nhận diện đối tượng mà còn hiểu được ngữ cảnh và hành vi trong môi trường thực tế.

Các bài toán phổ biến trong Computer Vision bao gồm:

- Phân loại hình ảnh (Image Classification)
- Phát hiện đối tượng (Object Detection)
- Nhận diện và theo dõi hành vi (Behavior Recognition)

Trong lĩnh vực giao thông thông minh, Computer Vision được ứng dụng rộng rãi để:

- Nhận diện phương tiện, người và biển báo giao thông
- Phát hiện các hành vi vi phạm hoặc nguy hiểm
- Phân tích tình huống giao thông theo thời gian thực

Nhờ khả năng xử lý dữ liệu hình ảnh liên tục từ camera, Computer Vision đóng vai trò quan trọng trong việc xây dựng các hệ thống giám sát giao thông thông minh, giúp nâng cao an toàn và hiệu quả quản lý giao thông.

1.4.2. Mạng nơ-ron tích chập (CNN) cho phân loại ảnh

CNN (Convolutional Neural Network) là một loại mạng nơ-ron sâu chuyên xử lý ảnh đầu vào [12]. Các lớp tích chập trong CNN có khả năng tự động học các đặc trưng không gian của ảnh như cạnh, hình dạng và cấu trúc đối tượng mà không cần trích xuất đặc trưng thủ công. CNN đã được ứng dụng rộng rãi trong nhiều bài toán thị giác máy tính, đặc biệt là phân loại hình ảnh.

1.4.3. Mô hình phát hiện đối tượng YOLO (YOLOv8n)

YOLO (You Only Look Once) là một mô hình học sâu nổi tiếng cho bài toán phát hiện đối tượng (object detection)[13]. Khác với các phương pháp cũ xử lý từng vùng ảnh, YOLO xử lý toàn bộ ảnh đầu vào chỉ trong một lần dự đoán, cho kết quả nhanh và chính xác. YOLOv8n là phiên bản cải tiến, nhẹ, dễ huấn luyện và triển khai, hỗ trợ tốt trên thiết bị cá nhân.

1.4.4. Nhận diện hành vi và cử chỉ với Landmark Detection & MediaPipe

Để giải quyết bài toán trên, đề tài kết hợp nhiều công nghệ hiện đại trong lĩnh vực trí tuệ nhân tạo và thị giác máy tính:

a. Landmark Detection

Landmark Detection là kỹ thuật trong thị giác máy tính dùng để xác định các điểm đặc trưng (keypoints) trên khuôn mặt hoặc cơ thể người như mắt, mũi, miệng, vai và tay. Thông qua các điểm mốc này, hệ thống có thể phân tích hình dạng, chuyển động và trạng thái của đối tượng. Phương pháp này giúp hiểu rõ hơn hành vi con người thay vì chỉ xác định vị trí như các mô hình phát hiện đối tượng thông thường.

b. MediaPipe

MediaPipe là một framework mã nguồn mở do Google phát triển, hỗ trợ xây dựng các hệ thống xử lý hình ảnh và video theo thời gian thực. MediaPipe cung cấp các mô hình sẵn có như Face Mesh, Pose và Hands để phát hiện và theo dõi các điểm mốc trên khuôn mặt và cơ thể. Nhờ khả năng xử lý nhanh, độ chính xác cao và dễ tích hợp, MediaPipe được sử dụng rộng rãi trong các bài toán nhận diện hành vi và cử chỉ.

1.5. Tổng quan công nghệ sử dụng

1.5.1 Mô hình nhận diện đối tượng CNN

Sau khi vùng đối tượng (ROI) được tách ra từ ảnh đầu vào, hệ thống sử dụng mô hình mạng nơ-ron tích chập (CNN) để thực hiện phân loại chi tiết các đối tượng như: biển báo giao thông, phương tiện (ô tô, xe máy, xe khách), người, dây đai an toàn, điện thoại và các vật thể khác.

CNN có khả năng tự động trích xuất đặc trưng từ dữ liệu ảnh và học các đặc điểm phân biệt giữa các lớp đối tượng với độ chính xác cao, đặc biệt hiệu quả đối với các đối tượng có hình dạng và màu sắc tương đồng.

Lý do lựa chọn:

Trong hệ thống, CNN được sử dụng như một bước phân loại chuyên sâu sau khi các vùng đối tượng đã được phát hiện bởi YOLO. Mô hình CNN được huấn luyện riêng với tập dữ liệu ảnh đã được chuẩn hóa về kích thước (32×32), giúp tăng tốc độ xử lý và đảm bảo tính đồng nhất của dữ liệu đầu vào[14].

Việc sử dụng CNN giúp cải thiện độ chính xác trong việc phân biệt các đối tượng có đặc điểm tương tự nhau như:

- Biển báo: “cấm rẽ trái”, “giới hạn tốc độ”, “đường một chiều”
- Phương tiện: ô tô, xe máy, xe khách
- Hành vi: sử dụng điện thoại, không thắt dây đai an toàn

Bên cạnh đó, CNN có khả năng học các đặc trưng phức tạp, giúp hệ thống hoạt động ổn định trong nhiều điều kiện khác nhau như thay đổi ánh sáng, góc nhìn hoặc nhiễu ảnh. Ngoài ra, mô hình CNN có thể được thiết kế với kiến trúc gọn nhẹ, phù hợp cho các hệ thống xử lý thời gian thực và triển khai trên thiết bị có cấu hình hạn chế.

1.5.2 Mô hình phát hiện đối tượng YOLOv8n

YOLOv8n (You Only Look Once version 8 nano) là một trong những mô hình nhận diện đối tượng thời gian thực mạnh mẽ và tối ưu nhất hiện nay. Trong hệ thống, YOLOv8n chịu trách nhiệm:

- Phát hiện chính xác vị trí nhận diện vùng chứa phương tiện, biển báo, người, vật cản... trong giao thông trong khung hình từ camera.
- Cắt vùng chứa đối tượng (ROI) để đưa vào các bước xử lý tiếp theo.
- Xử lý hiệu quả trong điều kiện thời tiết, ánh sáng và môi trường thực tế.

Lý do lựa chọn:

YOLOv8n được lựa chọn nhờ khả năng phát hiện đối tượng nhanh và chính xác trong điều kiện thực tế. Mô hình hoạt động tốt trong nhiều môi trường khác nhau như thay đổi ánh sáng, thời tiết hoặc góc quay camera.

Sau khi phát hiện, các vùng chứa đối tượng sẽ được cắt ra và chuyển sang mô hình CNN để phân loại chi tiết hơn. Cách tiếp cận này giúp:

- Giảm nhiễu từ nền ảnh
- Tăng độ chính xác phân loại
- Tối ưu hiệu năng hệ thống

Ngoài ra, YOLOv8n dễ dàng tích hợp với các thư viện như OpenCV và PyTorch, hỗ trợ triển khai thuận tiện trong các hệ thống xử lý thời gian thực sử dụng webcam.

1.5.3 Nhận diện hành vi và cử chỉ với Landmark Detection & MediaPipe

1.5.3.1. Landmark Detection – Nhận diện điểm mốc khuôn mặt và trạng thái tài xế

Landmark Detection là kỹ thuật xác định các điểm đặc trưng (keypoints) trên khuôn mặt hoặc cơ thể người như mắt, mũi, miệng,... Trong hệ thống, Landmark Detection được sử dụng để:

- Phát hiện trạng thái mắt (mở/nhắm)
- Xác định hướng nhìn của tài xế
- Phân tích mức độ tập trung khi lái xe

Nhờ việc xác định chính xác các điểm mốc, hệ thống có thể suy ra hành vi như buồn ngủ, mất tập trung hoặc không chú ý khi điều khiển phương tiện.

Lý do lựa chọn:

Landmark Detection cho phép phân tích chi tiết trạng thái khuôn mặt thay vì chỉ xác định vị trí đối tượng. Phương pháp này có độ chính xác cao, hoạt động tốt trong thời gian thực và phù hợp để phát hiện các hành vi nguy hiểm của tài xế như nhắm mắt quá lâu hoặc nhìn lệch hướng.

1.5.3.2 MediaPipe (Pose) – Nhận diện cử chỉ tay và hình dáng cơ thể người

MediaPipe là framework hỗ trợ nhận diện và theo dõi các điểm mốc trên cơ thể người theo thời gian thực. Trong hệ thống, MediaPipe (Pose) được sử dụng để:

- Nhận diện cử chỉ tay (ví dụ: sử dụng điện thoại)
- Phân tích tư thế cơ thể của tài xế
- Hỗ trợ phát hiện các hành vi không an toàn khi lái xe

MediaPipe cung cấp dữ liệu vị trí các khớp trên cơ thể, giúp hệ thống hiểu rõ hơn hành vi và chuyển động của người điều khiển phương tiện.

Lý do lựa chọn:

MediaPipe có tốc độ xử lý nhanh, độ chính xác cao và dễ tích hợp với Python và OpenCV. Ngoài ra, thư viện này được tối ưu cho các ứng dụng thời gian thực, phù hợp với hệ thống giám sát giao thông sử dụng webcam, giúp phát hiện hành vi tài xế một cách hiệu quả và ổn định.

1.5.4 Sự kết hợp giữa YOLO – CNN trong hệ thống

Hệ thống đề xuất sử dụng mô hình YOLO để xác định nhanh vị trí các đối tượng trong ảnh đầu vào từ webcam, bao gồm người, phương tiện và các vật thể liên quan trong giao thông [15]. Sau khi phát hiện, các vùng quan tâm (ROI) sẽ được trích xuất và đưa vào mô hình CNN để thực hiện phân loại đối tượng. Đồng thời, MediaPipe và Landmark Detection được sử dụng để phân tích hành vi và trạng thái của tài xế thông qua các điểm mốc trên khuôn mặt và cơ thể.

Kết quả từ các mô hình được kết hợp để đưa ra quyết định và phát cảnh báo bằng âm thanh, tạo thành một hệ thống giám sát giao thông thông minh và hỗ trợ người lái theo thời gian thực.

Việc kết hợp các công nghệ này mang lại các lợi thế:

- Nhận diện nhanh và chính xác trong thời gian thực
- Phân tích được cả đối tượng và hành vi của tài xế
- Giảm nhiễu và tăng độ tin cậy nhờ kết hợp nhiều mô hình
- Dễ mở rộng và tích hợp với các hệ thống giám sát giao thông thông minh

1.6. Khoảng trống nghiên cứu

Mặc dù đã có nhiều nghiên cứu về giám sát giao thông ứng dụng AI, phần lớn vẫn tập trung vào từng bài toán riêng lẻ như phát hiện đối tượng, nhận diện hành vi tài xế hoặc đọc nội dung biển báo, thay vì tối ưu một hệ thống tích hợp hoạt động đồng thời trong thời gian thực[16]. Đây là khoảng trống quan trọng, vì hiệu quả của từng mô hình độc lập chưa phản ánh đầy đủ hiệu quả của toàn bộ pipeline khi triển khai thực tế, nơi dữ liệu luôn biến động và yêu cầu phản hồi rất nhanh.

Bên cạnh đó, nhiều kết quả nghiên cứu hiện nay chủ yếu được đánh giá trên dữ liệu tương đối chuẩn hóa, trong khi bối cảnh giao thông thực tế có nhiều yếu tố gây suy giảm chất lượng nhận diện như ánh sáng thay đổi, rung camera, che khuất, mật độ phương tiện cao và hành vi tài xế diễn ra ngắn, khó bắt. Ngoài khoảng trống về độ bền mô hình, còn có khoảng trống ở lớp ứng dụng: ít nghiên cứu đi đến mức liên thông từ phát hiện vi phạm, cảnh báo tức thời cho tài xế, lưu trữ lịch sử vi phạm, đến giám sát tập trung cho quản trị viên trên cùng một nền tảng.

Đặc biệt, một khoảng trống thực tiễn đáng chú ý là thiếu mô-đun quản trị đội xe ở cấp hệ thống. Nhiều nghiên cứu chỉ dừng ở mức “mô hình nhận diện hoạt động tốt”, nhưng chưa xây dựng được dashboard quản lý giám sát riêng cho đội xe để theo dõi trạng thái phương tiện, tổng hợp vi phạm theo từng tài xế/ chuyến xe, và hỗ trợ ra quyết định điều phối hoặc nhắc nhở kịp thời. Trong khi đó, với bài toán doanh nghiệp vận tải, chính lớp dashboard này mới là thành phần chuyển kết quả AI thành giá trị vận hành thực tế.

Vì vậy, khoảng trống nghiên cứu của đề tài được xác định là: thiếu một hướng tiếp cận vừa đảm bảo độ chính xác, vừa giữ được tốc độ thời gian thực, đồng thời tích hợp đa mô-đun AI với lớp quản trị vận hành trong điều kiện thực tế. Từ đó, đề tài định hướng xây dựng hệ thống kết hợp YOLOv8, CNN, MediaPipe/Landmark, OCR và cảnh báo âm thanh; đồng thời triển khai dashboard giám sát chuyên biệt cho đội xe (theo dõi tập trung, lưu lịch sử vi phạm, hỗ trợ cảnh báo và quản lý theo vai trò). Phần mở rộng này đã góp phần lấp đầy khoảng trống mà nhiều nghiên cứu trước chưa giải quyết trọn vẹn: kết nối liền mạch giữa nhận diện thông minh và quản trị vận hành thực tế trong một nền tảng thống nhất.

CHƯƠNG II: PHÂN TÍCH PHÁT TRIỂN VÀ THIẾT KẾ HỆ THỐNG

2.1. Mục tiêu và yêu cầu hệ thống

2.1.1. Mục tiêu hệ thống

Hệ thống **DVX Traffic System** được xây dựng nhằm ứng dụng trí tuệ nhân tạo (AI) vào giám sát hành vi tài xế và môi trường giao thông theo thời gian thực, góp phần nâng cao an toàn giao thông và hỗ trợ quản lý hiệu quả.

Các mục tiêu chính của hệ thống bao gồm:

- **Ứng dụng AI để nhận diện hành vi và điều khiển bằng giọng nói**

Hệ thống tích hợp các công nghệ như CNN, YOLOv8n, MediaPipe và nhận diện giọng nói tiếng Việt để phát hiện các hành vi của tài xế như mất tập trung, buồn ngủ, sử dụng điện thoại,... đồng thời cho phép tương tác thông qua lệnh thoại.

- **Nâng cao an toàn giao thông và giảm thiểu tai nạn**

Hệ thống phát hiện sớm các hành vi nguy hiểm và đưa ra cảnh báo kịp thời bằng âm thanh hoặc thông báo trực quan giúp tài xế điều chỉnh hành vi khi đang lái xe

- **Phân tích môi trường giao thông theo thời gian thực**

Thông qua các mô hình AI, hệ thống nhận diện biển báo, phương tiện, chướng ngại vật và làn đường, từ đó hỗ trợ tài xế đưa ra quyết định an toàn hơn.

- **Xây dựng hệ sinh thái giám sát giao thông thông minh**

Hệ thống cho phép theo dõi nhiều phương tiện, lưu trữ dữ liệu vi phạm, phân tích hành vi và tích hợp chatbot AI hỗ trợ tra cứu luật giao thông.

- **Nâng cao trải nghiệm người dùng**

Tích hợp điều khiển bằng giọng nói, cảnh báo tự động và giao diện trực quan giúp người dùng tương tác thuận tiện mà không gây mất tập trung khi lái xe.

2.1.2. Yêu cầu chức năng

Hệ thống cần đáp ứng các chức năng chính sau:

- **Phát hiện hành vi tài xế theo thời gian thực**

Nhận diện các hành vi nguy hiểm như: nhắm mắt lâu, ngáp ngủ, mất tập trung, sử dụng điện thoại, không thắt dây an toàn, sai tư thế lái xe.

- **Nhận diện môi trường giao thông**

Phát hiện và phân loại:

- Biển báo giao thông
- Phương tiện xung quanh
- Chướng ngại vật
- Làn đường

- **Cảnh báo thông minh**

- Phát cảnh báo bằng âm thanh (gTTS)
- Hiện thị cảnh báo trên giao diện
- Gửi thông báo khi phát hiện vi phạm

- **Theo dõi và quản lý phương tiện**

- Theo dõi vị trí phương tiện (GPS)
- Gán ID và theo dõi nhiều đối tượng
- Lưu lịch sử hành vi và vi phạm

- **Lưu trữ và truy xuất dữ liệu**

- Lưu video/ảnh vi phạm
- Lưu log hành vi tài xế
- Truy xuất dữ liệu phục vụ phân tích

- **Tích hợp chatbot AI**

- Hỗ trợ tra cứu luật giao thông, tư vấn mức phạt
- Hỗ trợ người dùng bằng ngôn ngữ tự nhiên

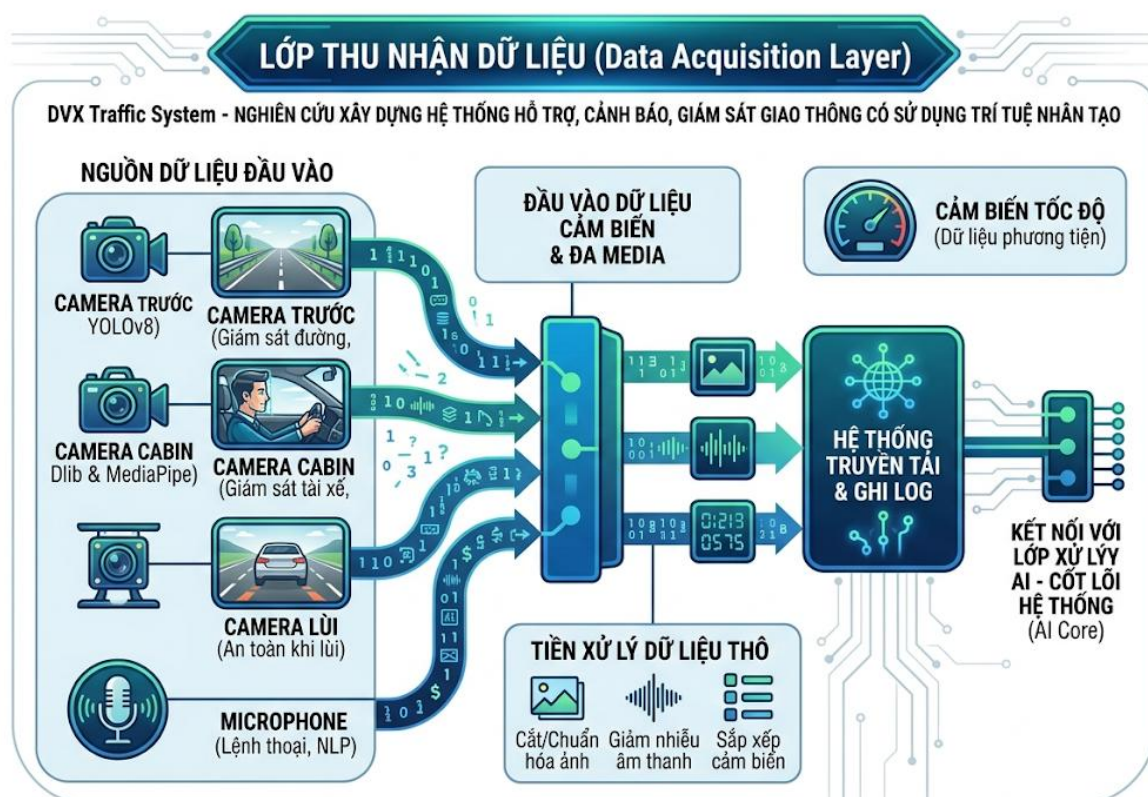
- **Giao diện người dùng (UI/UX)**

- Dashboard quản lý cho admin
- Giao diện realtime cho tài xế
- Hiện thị bản đồ và trạng thái phương tiện

2.2. Kiến trúc tổng thể hệ thống

2.2.1 Phân tích các lớp kiến trúc

a. Lớp Thu nhận Dữ liệu (Data Acquisition Layer)

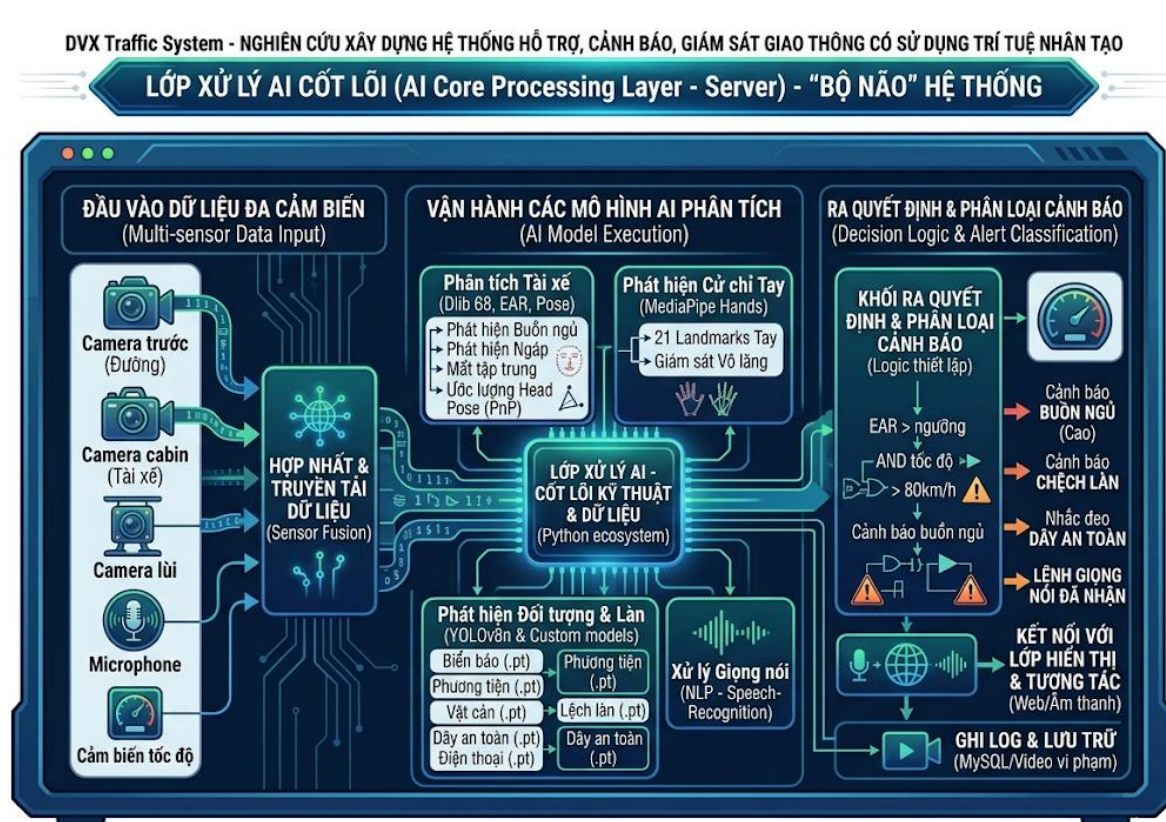


Hình 2. Sơ đồ khối Lớp Thu nhận Dữ liệu Đa cảm biến thời gian thực

Đây là đầu vào của hệ thống, chịu trách nhiệm tiếp nhận thông tin đa phương tiện từ môi trường:

- **Video & Audio:** Thu thập từ hệ thống camera đa hướng gồm:
 - *Camera trước:* Giám sát lộ trình đường đi.
 - *Camera cabin:* Giám sát hành vi tài xế.
 - *Camera trong xe:* Theo dõi hành khách.
 - *Camera lùi:* Hỗ trợ quan sát phía sau.
- **Âm thanh:** Thu âm từ Microphone để tiếp nhận các lệnh thoại.
- **Cảm biến:** Tiếp nhận dữ liệu về tốc độ và các thông số vận hành khác của xe.

b. Lớp Xử lý AI - Cốt lõi hệ thống (AI Core Layer)



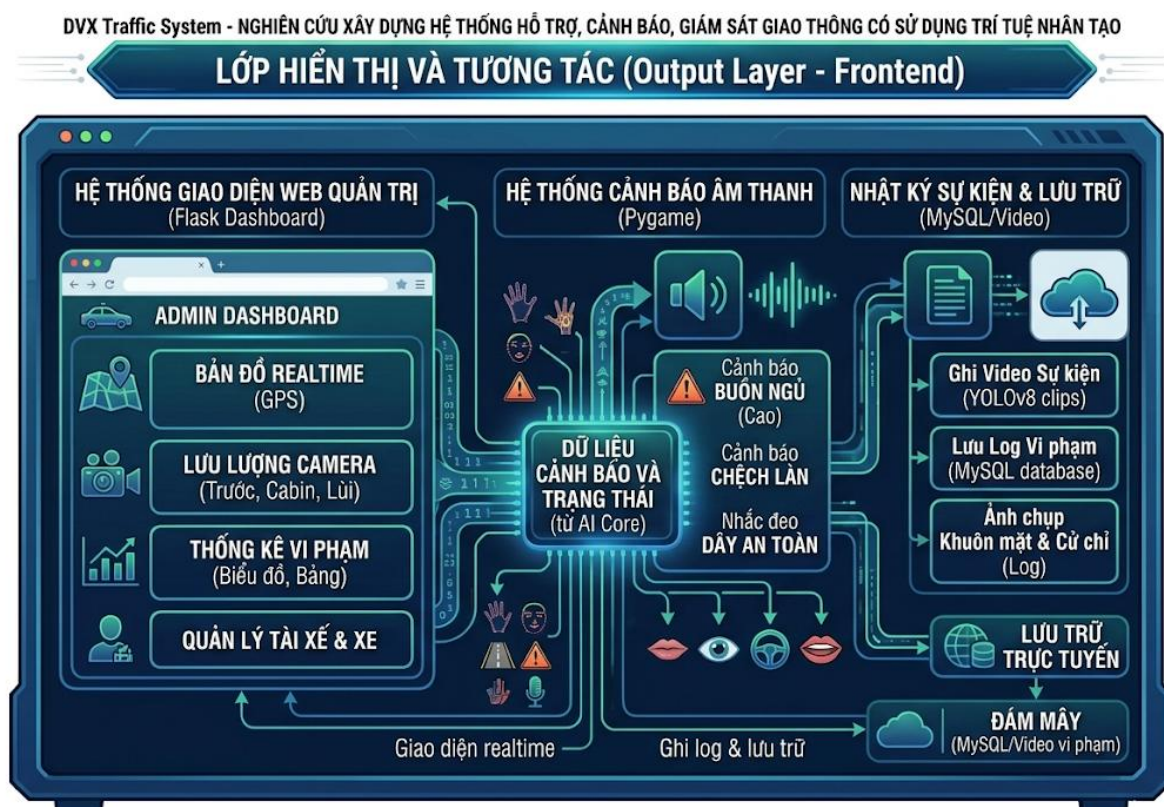
Hình 3. Sơ đồ khối Lớp Xử lý AI Cốt lõi (Báo cáo NCKH)

Đây là tầng quan trọng nhất, nơi các thuật toán Deep Learning và Computer Vision hoạt động để phân tích dữ liệu:

- **Phân tích Tài xế (Dlib 68 points):** Sử dụng chỉ số EAR (Eye Aspect Ratio) để phát hiện nhắm mắt, ngáp ngủ và đưa ra cảnh báo buồn ngủ[17].
- **Ước lượng Tư thế (Head Pose - PnP):** Trích xuất các góc Euler (Yaw, Pitch, Roll) để xác định xem tài xế có đang mất tập trung hay không[18].
- **Phát hiện Đối tượng & Làn đường (YOLOv8n & Custom models):**
 - Nhận diện phương tiện, vật cản, biển báo giao thông.
 - Phát hiện vạch kẻ đường (Hough Line Transform)[19].
 - Giám sát hành vi vi phạm như sử dụng điện thoại hoặc không thắt dây an toàn.
- **Phát hiện Cử chỉ (MediaPipe):** Theo dõi 21 điểm chạm trên bàn tay để giám sát việc cầm vô lăng.

- **Xử lý Giọng nói (NLP):** Nhận dạng tiếng Việt để thực hiện các lệnh điều khiển hoặc hỏi đáp.
- **Khởi Quyết định (Decision Logic):** Tổng hợp dữ liệu từ tất cả các module trên kết hợp với dữ liệu cảm biến tốc độ để phân loại mức độ cảnh báo.

c. Lớp Hiển thị, Tương tác & Quản lý (Application Layer)



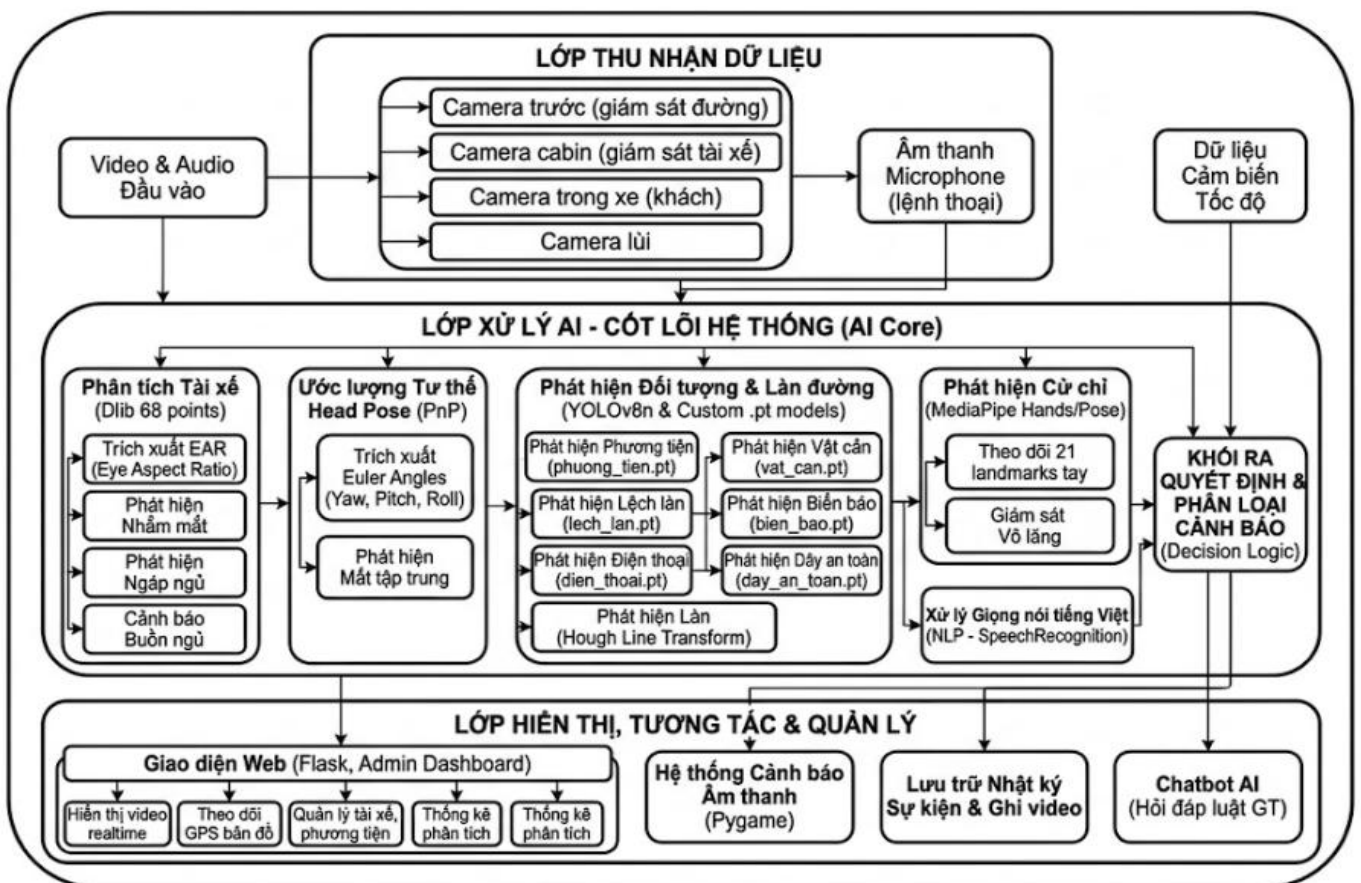
Hình 4. Sơ đồ khối Lớp Hiển thị & Tương tác (Dashboard & Frontend)

Kết quả sau khi xử lý được chuyển xuống tầng ứng dụng để tương tác với người dùng:

- **Giao diện Web (Flask):** Cung cấp Dashboard cho quản trị viên xem video thời gian thực, bản đồ GPS, quản lý tài xế/phương tiện và xem báo cáo thống kê.
- **Hệ thống Cảnh báo (Pygame):** Phát âm thanh cảnh báo trực tiếp trong cabin.
- **Lưu trữ:** Ghi lại nhật ký sự kiện và video để làm bằng chứng khi cần thiết.
- **Chatbot AI:** Hỗ trợ tài xế tra cứu luật giao thông bằng giọng nói hoặc văn bản.

2.2.2 Luồng dữ liệu (Workflow)

1. Dữ liệu thô từ Camera và Cảm biến được đẩy vào AI Core.
2. Các mô hình YOLO, Dlib, MediaPipe chạy song song để trích xuất đặc trưng (hành vi tài xế, vật cản, làn đường).
3. Decision Logic tiếp nhận các đặc trưng này, nếu phát hiện nguy hiểm sẽ kích hoạt Hệ thống cảnh báo âm thanh.
4. Toàn bộ trạng thái được đồng bộ hóa lên Giao diện Web và Cơ sở dữ liệu để quản lý từ xa.



Hình 1. Sơ đồ khối hệ thống DVX Traffic System (Kiến trúc Tổng thể & Xử lý AI Chi tiết)

2.3. Quy trình xử lý dữ liệu:

Để hệ thống giám sát giao thông hoạt động ổn định trong điều kiện thực tế, dữ liệu không chỉ cần “nhiều” mà phải “đúng ngữ cảnh vận hành”. Vì vậy, quy trình xử lý dữ liệu của đề tài được thiết kế theo hướng kỹ thuật hóa đầy đủ từ thu thập, chuẩn hóa, tăng cường dữ liệu đến gán nhãn và kiểm định chất lượng. Mục tiêu là tạo bộ dữ liệu có tính đại diện cao cho các tình huống giao thông Việt Nam, đồng thời giảm rủi ro overfitting và giảm cảnh báo sai khi chạy thời gian thực.

2.3.1. Thu thập dữ liệu thực tế

Dữ liệu huấn luyện được thu thập từ hai nguồn chính:

- Dữ liệu tự xây dựng: ảnh/frame trích xuất từ video camera thực tế (camera trước xe, camera giao thông, video hành trình).
- Dữ liệu công khai: các bộ dataset trên mạng có cùng miền bài toán (phát hiện phương tiện, biển báo, hành vi tài xế, điều kiện đường)

Cách kết hợp này giúp tăng độ đa dạng mẫu và giảm rủi ro thiếu dữ liệu ở các tình huống hiếm. Dữ liệu sau khi thu thập được chuẩn hóa về cùng định dạng trước khi đưa vào pipeline xử lý.

Để đảm bảo dữ liệu có giá trị huấn luyện, quá trình thu thập đặt ra các điều kiện bắt buộc:

- Ánh sáng: ban ngày, chiều tối, ban đêm, ngược sáng.
- Thời tiết: nắng, mưa nhẹ, mặt đường phản quang, sương mù nhẹ.
- Góc nhìn và khoảng cách: gần, trung bình, xa; nhiều góc camera khác nhau.
- Mật độ giao thông: đường thoáng và đường đông.
- Trạng thái vi phạm/không vi phạm: cần có đủ mẫu âm tính để giảm cảnh báo giả.

Tiêu chí tối thiểu của tập dữ liệu:

- Mỗi lớp mục tiêu có số lượng mẫu đủ lớn để huấn luyện ổn định.
- Phân bố lớp không chênh lệch quá mạnh (tránh bias).
- Mỗi kịch bản chính xuất hiện trong nhiều điều kiện môi trường khác nhau.
- Đầu ra của bước này là dữ liệu thô (video, frame, ảnh tĩnh) và metadata cơ bản (nguồn dữ liệu, điều kiện thu thập, lớp dự kiến)

2.3.2. Tiền xử lý dữ liệu (Preprocessing)

Sau khi thu thập, dữ liệu được tiền xử lý ở mức cần thiết để bảo đảm chất lượng trước khi huấn luyện mô hình. Mục tiêu của bước này là làm sạch dữ liệu, chuẩn hóa định dạng và giảm sai lệch giữa các mẫu.

Các bước chính gồm:

- *Chuẩn hóa dữ liệu đầu vào*

- + Đồng bộ định dạng ảnh/video.
- + Chuẩn hóa kích thước ảnh theo đầu vào của mô hình.
- + Loại bỏ các frame trùng lặp quá nhiều.

- *Làm sạch dữ liệu*

- + Loại bỏ ảnh quá mờ, cháy sáng hoặc thiếu đối tượng chính.
- + Loại các mẫu sai ngữ cảnh hoặc không phù hợp với bài toán.

- *Tăng cường dữ liệu cơ bản*

- + Áp dụng một số biến đổi đơn giản như thay đổi sáng/tối nhẹ, xoay nhẹ, phóng to/thu nhỏ nhẹ để tăng độ đa dạng dữ liệu.
- + Chỉ áp dụng augmentation trên tập train, không áp dụng cho validation/test.

- *Chia tập dữ liệu*

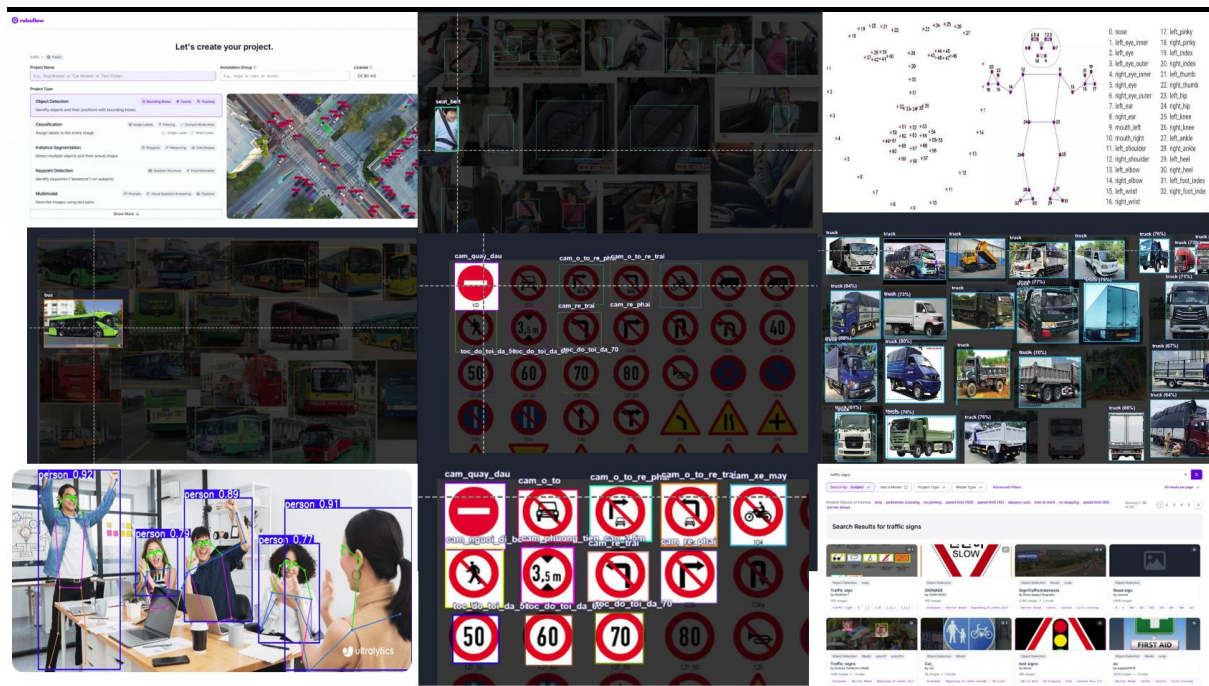
- + Chia dữ liệu thành train/validation/test theo tỷ lệ phù hợp (ví dụ 70/20/10 hoặc 80/10/10).
- + Bảo đảm các mẫu gần giống nhau không đồng thời xuất hiện ở cả train và test để tránh rò rỉ dữ liệu.

Kết quả của bước tiền xử lý là bộ dữ liệu gọn, sạch, đồng nhất và đủ đa dạng để phục vụ huấn luyện, đánh giá mô hình một cách khách quan hơn.

2.3.3. Gán nhãn dữ liệu (Labeling)

Gán nhãn được thực hiện theo từng tác vụ AI trong hệ thống để đảm bảo liên thông khi tích hợp:

- **Detection (YOLO):** nhãn bounding box cho phương tiện, người, biển báo, đối tượng nguy cơ.
- **Classification (CNN):** nhãn trạng thái/hành vi tài xế theo từng lớp xác định trước.



Hình 4. Gán nhãn các đối tượng

- Để nâng chất lượng nhãn, quy trình nên áp dụng 3 lớp kiểm soát:
 - + Kiểm tra kỹ thuật: lỗi thiếu nhãn, box sai vị trí, sai định dạng.
 - + Kiểm tra ngữ nghĩa: nhãn có đúng ngữ cảnh không, có mâu thuẫn giữa các frame liên tiếp không.
 - + Kiểm tra chéo: một phần dữ liệu được gán nhãn lại bởi người thứ hai để đo độ nhất quán.
- Một bộ tiêu chí “nhãn đạt chuẩn” nên bao gồm:
 - + Tỷ lệ nhãn lỗi dưới ngưỡng cho phép.
 - + Tỷ lệ nhãn thiếu được khắc phục trước huấn luyện.
- Mỗi lớp đều có mẫu ở cả điều kiện dễ và điều kiện khó.

2.4 PHÁT TRIỂN MÔ HÌNH NHẬN DIỆN

2.4.1. Phát triển mô hình phân loại CNN

a. Giới thiệu về mô hình CNN

Mạng nơ-ron tích chập (Convolutional Neural Network – CNN) là một kiến trúc mạng học sâu được thiết kế chuyên biệt cho việc xử lý dữ liệu hình ảnh. CNN sử

dùng các lớp tích chập để trích xuất các đặc trưng không gian như cạnh, hình dạng và cấu trúc của đối tượng.

b. Kiến trúc mô hình

- Ba tầng Convolutional + MaxPooling

Layer	Input	Output	Activation	Dropout
fc1	6272	512	LeakyReLU	0.5
fc2	512	1024	LeakyReLU	0.5
fc3	1024	num_classes		0.5

- Cấu trúc chi tiết mỗi Conv Block

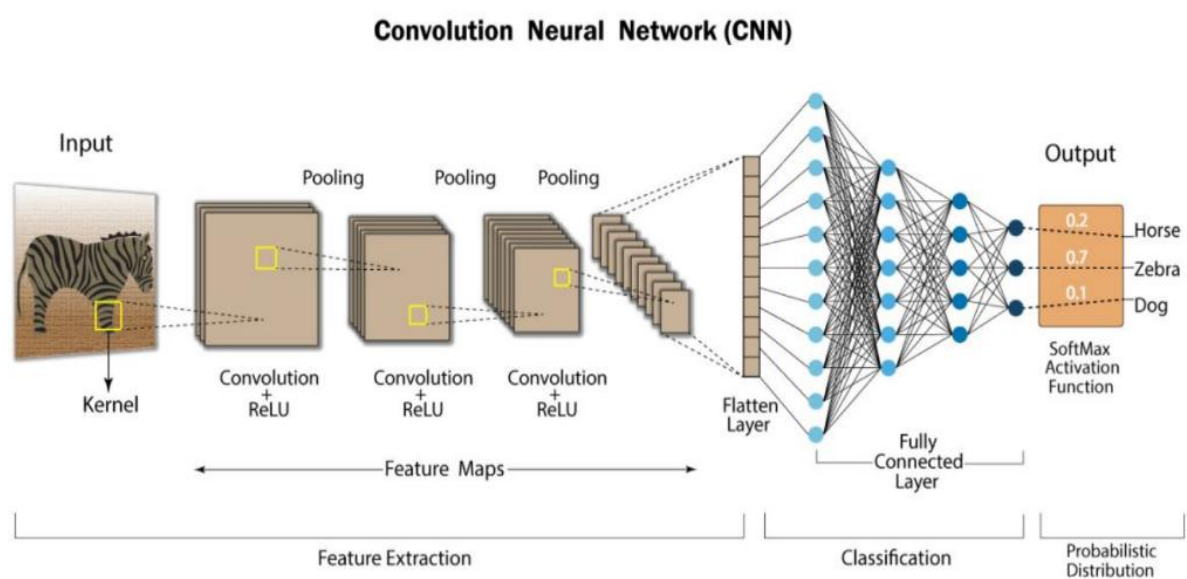
Mỗi block có 7 lớp con:

1	Conv Block N:
2	└── Conv2D(in_channels → out_channels, kernel=3×3, padding=same)
3	└── BatchNorm2D(out_channels)
4	└── LeakyReLU()
5	└── Conv2D(out_channels → out_channels, kernel=3×3, padding=same)
6	└── BatchNorm2D(out_channels)
7	└── LeakyReLU()
8	└── MaxPool2D(kernel=2×2) ← Giảm kích thước 1/2

- Lớp Flatten → Dense → Output

Tầng	Operation
Flatten	Reshape 3D → 1D
Dense 1	Dropout(0.5) → Linear → LeakyReLU
Dense 2	Dropout(0.5) → Linear → LeakyReLU
Output	Dropout(0.5) → Linear

- Hàm kích hoạt (Activation Function)
 - + LeakyReLU sau mỗi Conv2D và Linear [20].
- Dropout chống Overfitting



Hình.. Mô hình CNN (Convolution Neural Network)

b. Thông số huấn luyện

Thông số	Giá trị	Mô tả
<i>Epochs</i>	100	Số lần duyệt toàn bộ dataset đi qua mô hình
<i>Batch Size</i>	8	Số mẫu mỗi lần huấn luyện
<i>Image Size</i>	224x224	Kích thước đầu vào
<i>Learning Rate</i>	1e-3 (0.001)	Tốc độ học
<i>Optimizer</i>	SGD	Stochastic Gradient Descent
<i>Loss Function</i>	CrossEntropyLoss	Hàm mất mát phân loại

c. Lưu mô hình

Trong quá trình huấn luyện mô hình CNN, hệ thống sử dụng hai file checkpoint để lưu trữ weights của mô hình theo hai mục đích khác nhau: last.pt để lưu mô hình ở epoch cuối cùng nhằm mục đích tiếp tục huấn luyện nếu bị gián đoạn, và best.pt để lưu mô hình có độ chính xác cao nhất nhằm phục vụ cho việc dự đoán và triển khai thực tế.

2.4.2. Phát triển mô hình YOLO (phát hiện đối tượng)

a. Giới thiệu YOLOv8n

YOLO (You Only Look Once) là một mô hình phát hiện đối tượng theo phương pháp One-Stage, cho phép xử lý toàn bộ hình ảnh trong một lần dự đoán duy nhất. Điều này giúp YOLO đạt tốc độ xử lý rất cao, phù hợp với các ứng dụng thời gian thực.

Phiên bản YOLOv8n (nano) được lựa chọn trong đề tài nhờ khả năng cân bằng giữa tốc độ và độ chính xác. Mô hình này có thể đạt tốc độ xử lý khoảng 30 khung hình/giây (FPS), đáp ứng yêu cầu của hệ thống giám sát giao thông thời gian thực [21].

b. Huấn luyện mô hình YOLOv8n phát hiện tài xế, biển báo, vật thể & phương tiện.

```
from ultralytics import YOLO
model = YOLO("yolov8n.pt")
model.train(
    data="C:/Users/phucd/Downloads/DMS-20250511T143406Z-1-001/DMS/seat_belt_detection.yolov8/data.yaml",
    epochs=50,
    imgsz=320,
    batch=8,
    save=True,
    project="runs/train",
    name="seatbelt_model"
)model.export(format="torch", name="yolov8n_seatbelt.pt")
```

- Sử dụng tập dữ liệu hình ảnh biển báo giao thông đã được gán nhãn bằng công cụ LabelStudio, Roboflow theo chuẩn định dạng YOLO.

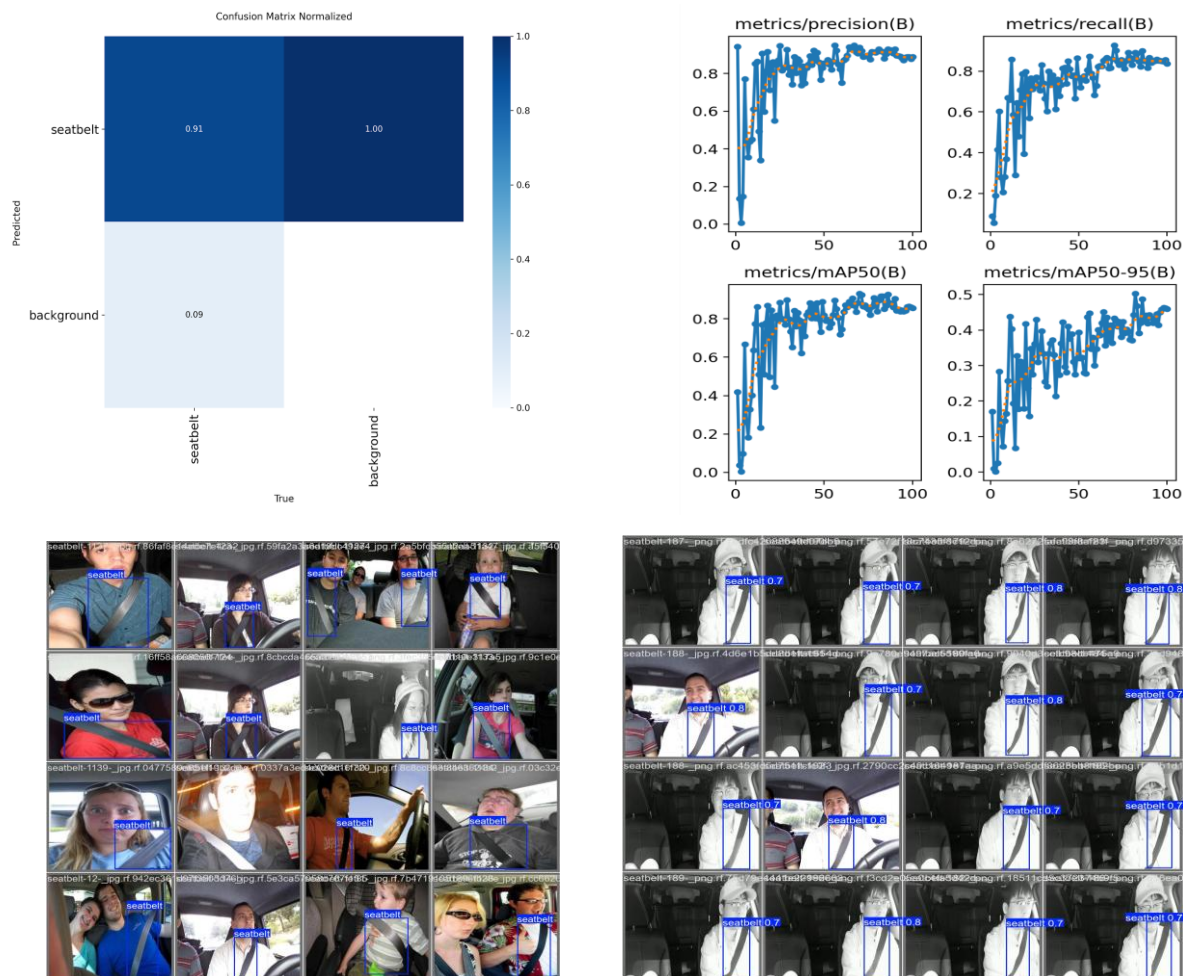
- Tiến hành huấn luyện mô hình dựa trên kiến trúc YOLOv8n (trọng số khởi tạo *yolov8n.pt*) hoặc thực hiện fine-tuning từ các mô hình đã được huấn luyện trước nhằm cải thiện độ chính xác và rút ngắn thời gian huấn luyện.
- Áp dụng thư viện Ultralytics YOLOv8 (dựa trên nền tảng PyTorch) để thực hiện quá trình huấn luyện mô hình cũng như triển khai suy luận (inference) trên dữ liệu thực tế.
- Bộ dữ liệu được chia thành 3 tập con, mỗi tập phục vụ một mục đích riêng trong quá trình huấn luyện và đánh giá mô hình phát hiện phương tiện. Tập **Train** bao gồm 2100 hình ảnh (tỉ lệ 70%), được sử dụng để huấn luyện mô hình nhận diện các đối tượng phương tiện. Tập **Validation (Val)** gồm 600 hình ảnh (tỉ lệ 20%), dùng để đánh giá hiệu suất mô hình trong quá trình huấn luyện và hỗ trợ điều chỉnh siêu tham số. Tập **Test** gồm 300 hình ảnh (tỉ lệ 10%), đóng vai trò là tập dữ liệu độc lập để kiểm tra khả năng tổng quát hóa của mô hình sau khi huấn luyện. Với các tệp trọng số :
 - Phát hiện diện thoại: *py/weights/yolov8n.pt*
 - Phát hiện day an toan: *py/weights/day_an_toan.pt*
 - Bien bao: *py/weights/bien_bao.pt*
 - Phương tiện: *py/weights/yolov8n.pt*
 - Lech lan: *py/weights/lech_lan.pt*
 - Vat can/ho ga: *py/weights/vat_can.pt*

Mô hình **YOLOv8n** được sử dụng để huấn luyện trên bộ dữ liệu này trong môi trường Kaggle. Sau quá trình huấn luyện, mô hình đạt được các chỉ số đánh giá gồm **precision**, **recall**, **mAP@50** và **mAP@50-95** lần lượt là 0.84; 0.84; 0.87 và 0.53. Các kết quả này cho thấy mô hình có khả năng phát hiện đối tượng phương tiện với độ chính xác cao và hiệu suất ổn định.

c. Nhận diện và cắt phương tiện, người, vũng nước, ổ gà,...

Sau khi mô hình YOLOv8 phát hiện các đối tượng trong ảnh thông qua bounding box:

- Ảnh đầu vào được đưa qua mạng CNN để trích xuất đặc trưng, sau đó mô hình dự đoán đồng thời vị trí (bounding box) và loại đối tượng (class) như phương tiện, người, vũng nước, ổ gà.
- Các bounding box có độ tin cậy cao được giữ lại thông qua thuật toán lọc (Non-Max Suppression).
- Cắt (crop) các vùng ảnh tương ứng với từng đối tượng đã phát hiện.



Hình 5 . Đánh giá mô hình huấn luyện

2.4.3 Mô hình MediaPipe (phát hiện bàn tay và cử chỉ cơ thể người)

a.Giới thiệu MediaPipe

MediaPipe là một (framework) mã nguồn mở cung cấp các giải pháp Học máy (ML) thời gian thực nổi bật với khả năng nhận diện khuôn mặt, bàn tay, dáng người (pose) và vật thể hiệu suất cao, tối ưu hóa trên CPU/GPU/TPU mà không cần phần cứng chuyên dụng [22].

b.Xử lý MediaPipe

- Sử dụng MediaPipe để trích xuất tọa độ các điểm đặc trưng (landmarks) của bàn tay và cơ thể người trong quá trình ghi hình.
- Các dữ liệu này được thu thập trong nhiều trạng thái khác nhau như buồn ngủ, mất tập trung, cầm vô lăng,... và được gán nhãn tương ứng để tạo thành tập dữ liệu huấn luyện.
- Tập dữ liệu landmarks sau đó được sử dụng để huấn luyện mô hình phân loại, giúp hệ thống nhận diện chính xác các cử chỉ và hành vi của người dùng.

c.Nhận diện và cắt bàn tay và cử chỉ cơ thể người

Trong đề tài, MediaPipe được áp dụng với dữ liệu landmarks được trích xuất để phân loại cử chỉ nhằm phù hợp với bài toán phát hiện cử chỉ của tài xế trong quá trình lái xe.

2.4.4. Mô hình Landmark Detection (phát hiện điểm mốc trên khuôn mặt)

a.Giới thiệu Landmark Detection

- Landmark Detection là kỹ thuật xác định các điểm đặc trưng trên khuôn mặt như mắt, mũi, miệng,... nhằm mô tả cấu trúc và trạng thái khuôn mặt.
- Trong đề tài, sử dụng Dlib – thư viện mạnh trong xử lý ảnh, cung cấp mô hình phát hiện 68 điểm landmark trên khuôn mặt với độ chính xác cao [23].

b. Xử lý Landmark Detection

- Dlib đã cung cấp sẵn mô hình phát hiện landmark khuôn mặt (pretrained), do đó không cần huấn luyện lại từ đầu.
- Trong hệ thống, các điểm landmark được trích xuất và sử dụng để xây dựng tập dữ liệu, sau đó huấn luyện thêm mô hình phân loại nhằm nhận diện các trạng thái như buồn ngủ, mất tập trung,...

c. Nhận diện và cắt điểm mốc trên khuôn mặt

- Sau khi đưa ảnh đầu vào vào Dlib:
 - Phát hiện khuôn mặt trong ảnh bằng bộ phát hiện face detector.
 - Xác định 68 điểm landmark trên khuôn mặt (mắt, mũi, miệng, viền mặt).
 - Trích xuất tọa độ các điểm để biểu diễn trạng thái khuôn mặt.
 - Xác định vùng khuôn mặt (ROI) và thực hiện cắt (crop) vùng ảnh tương ứng.
 - Chuẩn hóa dữ liệu và đưa vào mô hình phân tích để nhận diện trạng thái như nhắm mắt, quay đầu, buồn ngủ,...

2.5. Pipeline hệ thống nhận diện

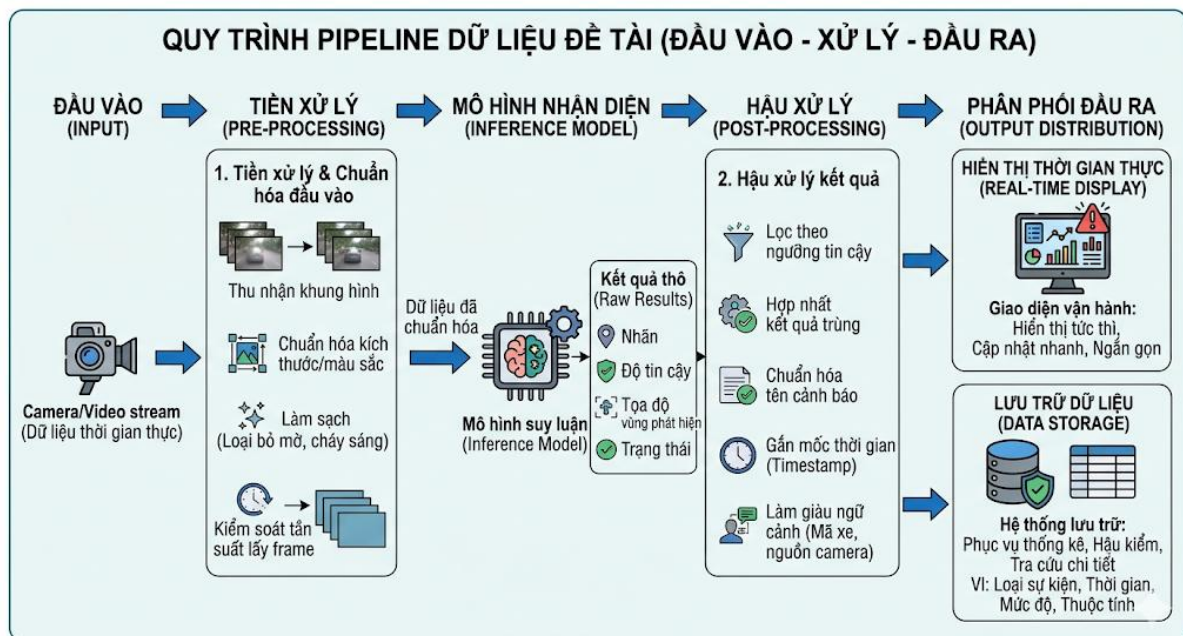
Camera → Preprocessing → YOLO Detection → ROI Extraction → (CNN+MediaPipe) → Decision

Hình 6. Pipeline hệ thống nhận diện

Trong phạm vi đề tài, pipeline dữ liệu được thiết kế tập trung vào hai phần chính: xử lý dữ liệu đầu vào trước suy luận và chuẩn hóa dữ liệu đầu ra sau suy luận. Ở đầu vào, hệ thống tiếp nhận dữ liệu từ các luồng ảnh/video theo thời gian thực, sau đó thực hiện tiền xử lý ở mức cần thiết để bảo đảm tính ổn định cho mô hình. Các khung hình được chuẩn hóa kích thước, đồng bộ định dạng màu, loại bỏ những frame lỗi nghiêm trọng như mờ nặng hoặc chói sáng, đồng thời kiểm soát tần suất lấy frame để tránh trùng lặp dữ liệu liên tiếp. Việc chuẩn hóa này giúp giảm nhiễu, giảm dao động khi suy luận và giữ tốc độ xử lý ổn định.

Sau bước chuẩn hóa đầu vào, dữ liệu được đưa vào các mô-đun nhận diện tương ứng và tạo ra kết quả thô dưới dạng nhãn, độ tin cậy, tọa độ vùng phát hiện hoặc trạng thái hành vi. Từ kết quả thô này, pipeline đầu ra tiếp tục thực hiện bước hậu xử lý nhằm chuyển kết quả mô hình thành dữ liệu nghiệp vụ có thể sử dụng được. Cụ thể, hệ thống áp dụng ngưỡng tin cậy để lọc dự đoán yếu, hợp nhất các kết quả trùng lặp trong cùng thời điểm, chuẩn hóa tên loại cảnh báo và gắn mốc thời gian cho từng sự kiện. Đồng thời, dữ liệu đầu ra được làm giàu bằng thông tin ngữ cảnh như mã xe hoặc nguồn camera để phục vụ truy vết.

Ở giai đoạn cuối, dữ liệu đầu ra được đóng gói thành hai lớp. Lớp thứ nhất là dữ liệu thời gian thực phục vụ hiển thị tức thì trên giao diện, ưu tiên ngắn gọn và cập nhật nhanh. Lớp thứ hai là dữ liệu lưu trữ phục vụ thống kê và hậu kiểm, bao gồm loại sự kiện, thời gian phát sinh, mức độ cảnh báo và các thuộc tính liên quan. Nhờ tách rõ hai lớp này, hệ thống vừa đáp ứng yêu cầu phản hồi nhanh khi vận hành, vừa bảo đảm khả năng truy xuất và phân tích dữ liệu về sau.



Hình 7. Pipeline dữ liệu hệ thống trực quan

Tóm lại, pipeline dữ liệu của đề tài có thể diễn giải ngắn gọn theo chu trình: thu nhận dữ liệu đầu vào, chuẩn hóa và làm sạch, suy luận, hậu xử lý kết quả, rồi phân phối đầu ra theo hai mục tiêu là hiển thị thời gian thực và lưu trữ có cấu trúc. Cách tiếp cận này giúp dữ liệu đi qua hệ thống theo một luồng nhất quán, hạn chế nhiễu ở đầu vào và tăng giá trị sử dụng của dữ liệu ở đầu ra.

2.6. Thiết kế các mô-đun chức năng:

2.6.1. Phân tích điều kiện xác định hành vi (Tài xế, biển báo, vi phạm)

2.6.1.1 Phát hiện buồn ngủ (Drowsiness Detection)

a) Nhắm mắt – EAR (Eye Aspect Ratio)

Để xác định trạng thái đóng/mở của mắt, ta sử dụng 6 điểm đặc trưng (landmarks) trên khuôn mặt cho mỗi mắt (thường được đánh số từ p1 đến p6 theo chuẩn Dlib hoặc MediaPipe).

Công thức tính EAR: Chỉ số EAR được tính bằng tỉ lệ giữa chiều cao và chiều rộng của mắt:

$$EAR = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2\|p_1 - p_4\|}$$

- Trong đó:

p1, p4: Các điểm ở góc mắt trái và góc mắt phải (chiều ngang).

p2, p3, p5, p6: Các điểm ở mí mắt trên và mí mắt dưới (chiều dọc).

$\|p_i - p_j\|$: Khoảng cách Euclidean giữa hai điểm i và j.

- Logic xử lý và ngưỡng cảnh báo

Hệ thống sẽ theo dõi giá trị EAR liên tục theo thời gian thực:

- Trạng thái mở mắt: EAR thường duy trì ở mức cao và ổn định (thường > 0.25).
- Trạng thái nhắm mắt: EAR giảm đột ngột về gần bằng 0.
- Ngưỡng cảnh báo (Threshold):
 - Điều kiện: $EAR < 0.25$ (hoặc 0.30 tùy môi trường sáng).
 - Thời gian duy trì: > 2 giây (để phân biệt giữa nhắm mắt buồn ngủ và phản xạ chớp mắt tự nhiên vốn chỉ diễn ra trong khoảng 0.1s - 0.3s).

- Phản hồi của hệ thống:

Khi điều kiện nhắm mắt vượt ngưỡng thời gian quy định:

1. **Kích hoạt âm thanh:** Phát tín hiệu cảnh báo cường độ cao để đánh thức người dùng.
2. **Ghi log dữ liệu:** Lưu lại thời điểm và hình ảnh sự kiện để phục vụ giám sát hoặc phân tích hành vi sau này.

b) Ngáp ngủ

Chỉ số tỉ lệ mở miệng – MAR (Mouth Aspect Ratio)

Thay vì sử dụng khoảng cách pixel cố định (vốn sẽ thay đổi tùy thuộc vào khoảng cách từ mặt đến camera), ta sử dụng chỉ số MAR để chuẩn hóa dữ liệu. Ta sử dụng các điểm Landmark vùng môi (thường từ điểm 48 đến 67 theo chuẩn 68 points).

Công thức tính MAR: Tương tự như EAR, MAR tính toán dựa trên tỉ lệ giữa chiều cao và chiều ngang của miệng:

$$MAR = \frac{\|p_{51} - p_{59}\| + \|p_{53} - p_{57}\|}{2\|p_{49} - p_{55}\|}$$

Trong đó:

- p51, p53, p57, p59: Các điểm đặc trưng ở môi trên và môi dưới (đo độ mở dọc).
- p49, p55: Hai điểm ở khóe miệng (đo độ rộng ngang).

b) Logic xử lý và ngưỡng cảnh báo

Hệ thống theo dõi sự biến thiên của chỉ số MAR để nhận diện hành vi ngáp:

- **Ngưỡng giá trị (Threshold):** $MAR > 0.5$ (Giá trị này thể hiện miệng đang mở rộng vượt mức nói chuyện thông thường).
- **Thời gian duy trì:** Liên tiếp trong ít nhất **15 - 20 frames** (tương đương khoảng 1-2 giây tùy vào tốc độ khung hình của camera). Việc kiểm tra liên tiếp giúp loại trừ các hành động như nói chuyện, cười hoặc há miệng nhanh.

c) Phản hồi của hệ thống:

Nếu các điều kiện trên được thỏa mãn:

1. **Cảnh báo:** Hiện thị thông báo hoặc phát âm thanh nhắc nhở: "Bạn đang có dấu hiệu mệt mỏi, hãy nghỉ ngơi".
2. **Ghi log:** Lưu lại thời điểm phát hiện ngáp để đánh giá mức độ tỉnh táo tổng thể của người dùng trong suốt hành trình.

2.6.1.2 Phát hiện mất tập trung – Head Pose Estimation

a) Cơ chế hoạt động

Hệ thống sử dụng thuật toán PnP (Perspective-n-Point) để ước lượng tư thế đầu trong không gian 3D. Quá trình này bao gồm:

- Dữ liệu đầu vào: Sử dụng 6 điểm chuẩn landmarks trên khuôn mặt: đỉnh mũi, cằm, góc mắt trái, góc mắt phải, mép miệng trái và mép miệng phải.
- Xử lý: Tính toán ma trận xoay (Rotation Matrix) và trích xuất các góc Euler (Euler Angles) bao gồm: Yaw (quay trái/phải), Pitch (cúi/ngẩng), và Roll (ngiên đầu).

b) Ngưỡng cảnh báo (Thresholds)

Hệ thống sẽ kích hoạt cảnh báo "Mất tập trung" nếu các chỉ số vượt quá các ngưỡng sau:

- **Quay trái/phải:** $|\text{Yaw}| > 40^\circ$
- **Cúi đầu:** $|\text{Pitch}| > 35^\circ$
- **Ngiên đầu:** $|\text{Roll}| > 45^\circ$

c) Logic xử lý

- **Điều kiện:** Nếu bất kỳ góc nào vượt ngưỡng trên trong một khoảng thời gian liên tục (thường tính bằng frame hoặc giây).
- **Hành động:** * Phát âm thanh/hình ảnh cảnh báo mất tập trung.
 - Ghi log sự kiện vào hệ thống để theo dõi lịch sử hành vi.

Tính năng	Chỉ số (Ratio/Angle)	Ngưỡng (Threshold)	Thời gian duy trì
Buồn ngủ (Mắt)	EAR	< 0.30	> 2 giây
Ngáp ngủ (Miệng)	Khoảng cách môi	> 25 pixel	15 frames liên tiếp
Mất tập trung	Yaw/Pitch/Roll	> 40°/35°/45°	Tức thời/Liên tục

2.6.1.3 Phát hiện điện thoại & dây an toàn (YOLOv8)

a) Mô hình kiến trúc

Hệ thống sử dụng mô hình **YOLOv8n (You Only Look Once version 8 Nano)**. Đây là dòng mô hình *One-Stage Detector* tiên tiến nhất, giúp đạt được sự cân bằng giữa độ chính xác và tốc độ xử lý thời gian thực.

b) Thông số huấn luyện và cấu hình

Mô hình được huấn luyện trên bộ dữ liệu tùy chỉnh (Custom Dataset) với các thông số kỹ thuật sau:

- **Tham số huấn luyện:**
 - **Epochs:** 50 (Số lượt huấn luyện toàn bộ tập dữ liệu).
 - **Image size:** 640 x 640 (Kích thước ảnh đầu vào chuẩn hóa).
 - **Batch size:** 8 (Số lượng mẫu dữ liệu trong mỗi lần cập nhật trọng số).
- **Tham số suy luận (Inference):**
 - **Confidence Threshold:** 0.5 (Chỉ hiển thị các kết quả có độ tin cậy trên 50% để loại bỏ các nhận diện sai).

c) Dữ liệu huấn luyện (Dataset)

Hệ thống sử dụng tập dữ liệu được gán nhãn tùy chỉnh bao gồm:

- **Phone detection dataset:** Hình ảnh người lái xe sử dụng điện thoại ở các tư thế khác nhau.
- **Seatbelt detection dataset:** Hình ảnh người lái xe có thắt dây và không thắt dây an toàn trong các điều kiện ánh sáng khác nhau.

d) Đầu ra và Phản hồi hệ thống

Với mỗi khung hình, mô hình sẽ trả về:

- **Bounding box:** Tọa độ khung bao quanh đối tượng (điện thoại hoặc vị trí dây an toàn).
- **Class label:** Nhãn đối tượng ("Cell phone", "No seatbelt").
- **Confidence score:** Độ tin cậy của dự đoán.

Quy trình xử lý vi phạm:

1. **Cảnh báo trực quan:** Hiển thị **khung đỏ (Red Bounding Box)** bao quanh đối tượng vi phạm trên màn hình giám sát để người lái dễ dàng nhận biết.
2. **Cảnh báo âm thanh:** Phát tín hiệu âm thanh nhắc nhở tương ứng (ví dụ: "Vui lòng không sử dụng điện thoại" hoặc "Vui lòng thắt dây an toàn").
3. **Lưu trữ:** Ghi lại ảnh chụp sự kiện vi phạm vào nhật ký hệ thống.

2.6.1.4 Giám sát tay lái

a) Công nghệ sử dụng

- **MediaPipe Pose:** Xác định khung xương (vai, khuỷu tay, cổ tay) và vùng thân người.
- **MediaPipe Hands:** Theo dõi chi tiết 21 điểm landmarks trên bàn tay.

b) Nguyên lý nhận diện và Ngưỡng cảnh báo

Thành phần	Đặc điểm nhận diện	Ngưỡng (Threshold)
Bàn tay (Hands)	Phân tích 21 landmarks; kiểm tra trạng thái nắm (≥ 3 ngón tay cong).	Không nắm vô lăng > 3 giây.
Cánh tay (Arms)	Theo dõi chuỗi điểm: Vai (11, 12) → Khuỷu tay (13, 14) → Cổ tay (15, 16).	Cổ tay rời xa vùng vô lăng (ROI) > 100 pixel.
Khuôn ngực (Chest)	Tạo vùng ROI dựa trên 4 điểm: 2 vai và 2 hông.	Tọa độ vùng ngực không ổn định (nghiêng/ngả quá mức).

c) Logic xử lý kết hợp

- Xác định vùng làm việc (ROI):** Sử dụng điểm vai và hông từ Pose để khoanh vùng Khuôn ngực (phục vụ kiểm tra dây an toàn).
 - Xác định vùng không gian của vô lăng dựa trên vị trí ngồi của người lái.
- Kiểm tra đa tầng:**
 - Tầng 1 (Cánh tay): Nếu cổ tay nằm ngoài vùng vô lăng -> Nghi ngờ rời tay lái.
 - Tầng 2 (Bàn tay): Nếu cổ tay ở gần vô lăng nhưng các ngón tay không ở trạng thái nắm -> Cảnh báo hờ tay (không làm chủ tay lái).
- Hành động:**
 - Cảnh báo: Phát âm thanh "Vui lòng đặt tay lên vô lăng" hoặc "Giữ đúng tư thế lái xe".
 - Hiển thị:** Vẽ khung xương (Skeleton) màu xanh (bình thường) hoặc đỏ (vi phạm) đè lên hình ảnh thực tế.
 - Ghi log:** Lưu lại loại hành vi (ví dụ: "Rời tay lái", "Thả tay khi xe đang chạy").

2.6.1.5 Phát hiện biến báo giao thông

a) Mô hình và Dữ liệu

- **Mô hình sử dụng: YOLOv8 (phiên bản Nano hoặc Small)** tùy chọn theo cấu hình phần cứng để đảm bảo tốc độ nhận diện biển báo khi xe đang di chuyển ở tốc độ cao.
- **Dataset:** Sử dụng bộ dữ liệu tùy chỉnh (Custom Dataset) tập trung vào các loại biển báo giao thông đường bộ Việt Nam (theo Quy chuẩn QCVN 41 : 2019/BGTVT).
- **Cấu hình huấn luyện:**
 - Image size: 640 x 640.
 - Confidence threshold: 0.6 (đặt mức cao hơn một chút để tránh nhận diện nhầm các vật thể có màu sắc tương tự bên đường).

b) Phân loại đối tượng nhận diện

Mô hình tập trung nhận diện 3 nhóm biển báo chính:

- **Biển báo tốc độ:** Các biển báo giới hạn tốc độ tối đa cho phép (ví dụ: 50km/h, 60km/h, 80km/h...).
- **Biển cảnh báo nguy hiểm:** Các biển hình tam giác vàng, viền đỏ (ví dụ: đường giao nhau, công trường, đường trơn trượt...).
- **Biển báo cấm:** Các biển hình tròn viền đỏ (ví dụ: cấm đi ngược chiều, cấm rẽ trái/phải, cấm dừng đỗ...).

c) Logic xử lý và Phản hồi hệ thống

Hệ thống thực hiện quy trình sau ngay khi có kết quả từ mô hình:

1. **Hiển thị trực quan:** Vẽ khung bao (Bounding box) quanh biển báo và **hiển thị tên biển báo** (Class name) bằng tiếng Việt trên màn hình giám sát.
2. **Cảnh báo âm thanh:** Tự động phát âm thanh nhắc nhở tương ứng với loại biển báo (Ví dụ: "Phát hiện biển báo giới hạn tốc độ 50 km/h").
3. **Thống kê dữ liệu:**
 - **Bộ đếm (Counter):** Tự động đếm và cập nhật số lượng biển báo xuất hiện trong suốt hành trình.
 - **Logic lọc trùng:** Sử dụng thuật toán theo dõi đối tượng (Object Tracking) để đảm bảo một biển báo chỉ được đếm một lần khi xe đi ngang qua, tránh việc đếm lặp lại trên nhiều khung hình liên tiếp.

2.6.1.6 Phát hiện lệch làn

a) Cơ chế nhận diện đa tầng

Hệ thống kết hợp sức mạnh của học sâu và toán học hình học để tối ưu độ chính xác:

- **Tầng 1 (YOLOv8):** Nhận diện và phân loại các loại vạch kẻ đường (vạch đứt, vạch liền, vạch vàng/trắng). Bước này đóng vai trò bộ lọc để xác định vùng quan tâm (ROI) chứa làn đường tiềm năng.
- **Tầng 2 (Hough Line Transform):** Trích xuất các đường thẳng toán học từ vùng vạch kẻ đã phát hiện để tính toán góc và vị trí chính xác của làn đường.

b) Quy trình xử lý ảnh chi tiết

Để thuật toán Hough Line hoạt động hiệu quả, hình ảnh được xử lý qua chuỗi pipeline sau:

1. **Grayscale & Gaussian Blur:** Chuyển ảnh về thang độ xám và làm mờ để giảm nhiễu hạt từ mặt đường.
2. **Canny Edge Detection:** Trích xuất các biên (edges) dựa trên sự thay đổi cường độ sáng đột ngột.
3. **ROI Masking:** Chỉ giữ lại vùng hình thang phía trước mũi xe, loại bỏ bầu trời và cảnh quan hai bên để tăng tốc độ xử lý.
4. **Hough Line Transform:** Chuyển đổi các điểm ảnh biên thành các phương trình đường thẳng.
5. **Phân loại làn (Slope Classification):** Tính toán độ dốc (m):
 - $m < 0$: Làn đường bên trái.
 - $m > 0$: Làn đường bên phải.

c) Logic phát hiện lệch làn (Departure Logic)

Hệ thống xác định trạng thái an toàn dựa trên tọa độ hình học:

- **Tâm làn đường (C_{lane}):** Trung điểm giữa đường biên trái và đường biên phải tại vị trí đáy khung hình.
- **Tâm xe ($C_{vehicle}$):** Thường được cố định tại tọa độ giữa của chiều rộng ảnh ($Image_Width / 2$).
- **Sai số lệch (Offset):** $Offset = |C_{lane} - C_{vehicle}|$.

d) Ngưỡng cảnh báo và Phản hồi

- **Điều kiện cảnh báo:** Nếu $Offset > \text{Ngưỡng quy định}$ (ví dụ: 50 pixels hoặc 20% chiều rộng làn đường) liên tục trong khoảng 0.5 - 1 giây.

- **Phản hồi:** Hiển thị thông báo "**Cảnh báo lệch làn!**" trên màn hình.
 - Phát âm thanh bíp hoặc rung (nếu có thiết bị hỗ trợ).
 - Vẽ hai đường làn màu đỏ đè lên video để người lái nhận biết.

2.6.1.7 Cảnh báo va chạm

a) Cơ chế phát hiện và Ước lượng khoảng cách

- **Phát hiện đối tượng:** Sử dụng mô hình **YOLOv8** để nhận diện và theo dõi các phương tiện phía trước (ô tô, xe máy, xe tải).
- **Thuật toán ước lượng khoảng cách (Monocular Distance Estimation):**
Sử dụng mối quan hệ tỉ lệ nghịch giữa khoảng cách thực tế và kích thước đối tượng trên khung hình (đơn vị: pixel).

Công thức:

$$d = \frac{f \times H_{real}}{h_{bbox}}$$

(Trong dự án của bạn, công thức rút gọn là: $d = 2500 / h_{bbox}$)

Trong đó:

- d (distance): Khoảng cách ước lượng từ xe đến phương tiện phía trước (mét).
- h_{bbox} : Chiều cao của khung bao (bounding box) tính bằng pixel.
- 2500: Hệ số thực nghiệm (đã bao gồm tiêu cự camera f và chiều cao trung bình của xe H_{real}).

b) Logic cảnh báo thông minh theo tốc độ

Hệ thống không chỉ dựa vào khoảng cách tĩnh mà kết hợp với tốc độ thực tế của xe (lấy từ dữ liệu GPS hoặc OBD-II) để đưa ra các mức cảnh báo an toàn phù hợp với các dải tốc độ (60 – 80 – 100 km/h).

Bảng phân cấp cảnh báo (Ví dụ cho dải tốc độ 60-80 km/h):

Khoảng cách (d)	Trạng thái	Hành động của hệ thống
$d > 55 \text{ m}$	An toàn	Hiển thị khoảng cách màu xanh, trạng thái bình thường.
$35 \text{ m} \leq d \leq 55 \text{ m}$	Cảnh báo vàng	Hiển thị khung vàng, phát âm thanh nhắc: "Giữ khoảng cách an toàn".
$d < 35 \text{ m}$	Cảnh báo đỏ	Khung bao nhấp nháy đỏ, phát âm thanh cường độ cao: "Nguy hiểm! Phanh gấp".

c) Phản hồi hệ thống

- Giao diện (UI):** Vẽ khung bao quanh xe phía trước kèm theo con số khoảng cách thời gian thực. Màu sắc khung thay đổi (Xanh -> Vàng -> Đỏ) theo mức độ nguy hiểm.
- Âm thanh:** Tần số âm thanh cảnh báo tăng dần khi khoảng cách rút ngắn đột ngột.
- Ghi log:** Lưu lại các tình huống "Near-miss" (suýt va chạm) để phân tích hành vi lái xe.

2.6.1.8 Điều khiển giọng nói

a) Công nghệ và Cấu hình Hệ thống tích hợp module nhận dạng giọng nói để tối ưu hóa trải nghiệm người dùng:

- **Thư viện:** SpeechRecognition kết hợp với **Google Speech API** để đạt độ chính xác cao trong việc chuyển đổi âm thanh thành văn bản (Speech-to-Text).
- **Ngôn ngữ hỗ trợ:** Tiếng Việt (vi-VN).
- **Cấu hình phần cứng:** Sử dụng Microphone tích hợp của thiết bị (ví dụ: mic trên Dell G16) với bộ lọc nhiễu chủ động để hoạt động tốt trong môi trường có tiếng ồn động cơ.

Lệnh giọng nói	Hành động tương ứng
"Mở hỗ trợ lái xe"	Kích hoạt đồng thời các module: Phát hiện buồn ngủ, Lềch làn và Va chạm.
"Mở tư vấn"	Khởi chạy Chatbot hỏi đáp luật giao thông và hỗ trợ hành trình.
"Dừng cảnh báo"	Tạm thời tắt âm thanh cảnh báo khi người lái đã nhận biết tình huống.
"Kiểm tra hệ thống"	Yêu cầu báo cáo trạng thái các cảm biến và camera.

c) **Cơ chế xử lý luồng (Multi-threading)** Để đảm bảo tính ổn định của hệ thống, module giọng nói được triển khai theo cấu trúc sau:

- **Tách biệt tiến trình:** Module Speech Recognition được chạy trên một **Thread (luồng) riêng biệt**.
- **Mục đích:** Việc nhận dạng giọng nói từ Google API yêu cầu thời gian phản hồi từ server (độ trễ network). Việc tách luồng giúp luồng xử lý video (Video Stream) và các mô hình AI (YOLO, MediaPipe) vẫn hoạt động liên tục với FPS cao, không bị khựng (lag) khi đang chờ xử lý lệnh thoại.

d) **Phản hồi người dùng**

- **Xác nhận:** Sau khi nhận lệnh thành công, hệ thống phát âm thanh phản hồi (Ví dụ: "Đã mở hỗ trợ lái xe") để người dùng biết lệnh đã được thực thi.
- **Thông báo lỗi:** Nếu không có kết nối internet hoặc không rõ lệnh, hệ thống sẽ báo nhẹ bằng tín hiệu âm thanh hoặc văn bản trên màn hình.

2.6.3. Thiết Web cảnh báo (giao diện tài xế) và giám sát Dashboard (giao diện quản lý)

a. Backend (Xử lý phía server)

1. Công nghệ sử dụng

- Python 3.9. trở lên
- Flask làm web framework chính (render template, API, routing).
- Flask-CORS cho chia sẻ tài nguyên khác nguồn.
- Flask-Session để quản lý session đăng nhập.
- Flask-Bcrypt và bcrypt để hash/kiểm tra mật khẩu.
- PyMySQL kết nối cơ sở dữ liệu MySQL.
- OpenCV xử lý ảnh/video thời gian thực.
- Ultralytics YOLO phục vụ nhận diện vi phạm.
- dlib + MediaPipe cho theo dõi khuôn mặt/tư thế/tài xế.
- NumPy cho tính toán ma trận và xử lý dữ liệu ảnh.
- Pygame phát âm thanh cảnh báo.
- Flask-Limiter (đã khai báo dependency) cho kiểm soát tần suất request.
- SpeechRecognition (Python package)

2. Triển khai

Backend được xây dựng bằng Python sử dụng framework Flask, đóng vai trò trung gian xử lý dữ liệu giữa hệ thống AI và giao diện người dùng.

Chức năng chính:

- Nhận dữ liệu từ hệ thống nhận diện (YOLO, CNN, MediaPipe)
- Xử lý và chuẩn hóa dữ liệu (JSON)

- Cung cấp API cho frontend
- Gửi dữ liệu realtime lên giao diện
- Quản lý logic hệ thống (cảnh báo, lưu trữ, truy vấn)

b.Thiết kế cơ sở dữ liệu (MySQL)

Hệ thống sử dụng MySQL để lưu trữ và quản lý dữ liệu, lưu trữ chạy local với phpAdmin của Xampp, lưu trên server hosting của Aiven, cấu hình với MySQL WorkBen

Dữ liệu lưu trữ bao gồm:

- Thông tin phương tiện
- Thông tin tài xế
- Lịch sử vi phạm
- Hình ảnh/video vi phạm
- Trạng thái cảnh báo

Chức năng:

- Lưu trữ dữ liệu lâu dài
- Truy vấn và thống kê
- Hỗ trợ quản lý và phân tích

c. Fontend (Giao diện)

1.Công nghệ sử dụng

- HTML5, CSS3, JavaScript thuần
- Leaflet.js để hiển thị bản đồ và vị trí xe theo thời gian thực.
- Chart.js để vẽ biểu đồ thống kê trên Dashboard.
- Font Awesome, Themify Icons để hiển thị icon giao diện.
- Google Fonts (Roboto) và Normalize.css để chuẩn hóa hiển thị.
- Web Speech API (SpeechRecognition/webkitSpeechRecognition)

2. Triển khai

Giao diện Web được xây dựng nhằm hiển thị trực quan các thông tin từ hệ thống.

Chức năng chính:

- Hiển thị video từ camera theo thời gian thực
- Hiển thị kết quả nhận diện (bounding box, cảnh báo)
- Hiển thị thông báo vi phạm ngay lập tức
- Hiển thị danh sách và lịch sử vi phạm
- Dashboard thống kê (số vi phạm, loại vi phạm, thời gian)

Đặc điểm:

- Cập nhật dữ liệu liên tục (real-time)
- Giao diện trực quan, dễ sử dụng
- Hỗ trợ quản lý và giám sát hiệu quả

2.6.4. Thiết kế gọi API từ Grod và dataset xây dựng chatbot

2.6.4.1. Công nghệ sử dụng

Tích hợp mô hình ngôn ngữ lớn (LLM)

Sử dụng dịch vụ API: Groq API

Mô hình: llama-3.3-70b-versatile

Dataset:

****NGUỒN LUẬT**:**

- Luật Giao thông đường bộ 2008
- Nghị định 100/2019/NĐ-CP (xử phạt vi phạm giao thông)
- Nghị định 123/2021/NĐ-CP (sửa đổi, bổ sung)

- Các văn bản hướng dẫn thi hành

- ****MỨC PHẠT CHÍNH XÁC 2026:****

+ PHẠT NỒNG ĐỘ CỒN (Điều 5, Nghị định 100)

+ PHẠT KHÔNG ĐỘI MŨ BẢO HIỂM (Điều 6, Nghị định 100)

+ PHẠT KHÔNG CÓ GIẤY PHÉP LÁI (Điều 21, Nghị định 100)

****Hồ sơ gồm:****

1. CCCD/CMND + photo

2. Hóa đơn mua xe (bản gốc)

3. Giấy khai đăng ký xe...

Vai trò:

- Hiểu ngôn ngữ tự nhiên, đặc biệt là dữ liệu luật giao thông Việt Nam
- Sinh câu trả lời thông minh, dựa vào model và dataset

Triển khai :

```
def call_groq Law_advisor(question):
    """Gọi Groq API để tư vấn luật giao thông"""
    try:
        from groq import Groq
        import os
        from dotenv import load_dotenv

        load_dotenv()
        GROQ_API_KEY = os.getenv('GROQ_API_KEY')

        if not GROQ_API_KEY:
            print("⚠ Không tìm thấy GROQ_API_KEY")
            return generate_law_response_fallback(question)

        client = Groq(api_key=GROQ_API_KEY)

        law_database = """
        **NGUỒN LUẬT:**
        - Luật Giao thông đường bộ 2008
        - Nghị định 100/2019/NĐ-CP (xử phạt vi phạm giao thông)
        - Nghị định 123/2021/NĐ-CP (sửa đổi, bổ sung)
        - Các văn bản hướng dẫn thi hành
        """
```

Hình 8. Gọi api Groq tích hợp nguồn luật giao thông

2.6.4.2. Xử lý triển khai

Quy trình xử lý được thiết kế theo ba lớp. Lớp thứ nhất là lớp tiếp nhận yêu cầu: frontend gửi nội dung hội thoại đến API chat; backend chuẩn hóa dữ liệu đầu vào và gắn thông tin ngữ cảnh người dùng. Lớp thứ hai là lớp tăng cường ngữ cảnh: hệ thống truy xuất dữ liệu thực tế từ database, đồng thời nạp law dataset vào prompt hệ thống để định hướng mô hình trả lời theo miền giao thông. Lớp thứ ba là lớp suy luận và hậu xử lý: backend gọi Groq API để sinh phản hồi, kiểm tra tính hợp lệ cơ bản, sau đó trả về giao diện. Trong trường hợp Groq không khả dụng, luồng xử lý tự động chuyển sang rule-based fallback để trả lời các tình huống phổ biến như quá tốc độ, nồng độ cồn, vượt đèn, giấy phép lái xe và thủ tục hành chính. Cách tổ chức này giúp chatbot vừa linh hoạt trong diễn đạt, vừa ổn định trong vận hành thực tế [22].

2.6.4.3. Kết quả

Kết quả triển khai cho thấy chatbot đạt khả năng phản hồi đa dạng theo ngữ cảnh nghiệp vụ, bao gồm cả hỏi đáp vận hành đội xe và tư vấn luật giao thông. So với cách trả lời tĩnh, phương án tích hợp Groq giúp chất lượng diễn đạt tự nhiên hơn, xử lý tốt các câu hỏi mở và đối thoại nhiều lượt. Việc bổ sung dataset nội bộ giúp tăng độ nhất quán trong các nội dung trọng điểm (mức phạt, nhóm hành vi vi phạm, hướng dẫn cơ bản), đồng thời fallback rule-based giúp hệ thống không gián đoạn khi phụ thuộc dịch vụ ngoài. Tổng thể, mô hình triển khai đã đáp ứng mục tiêu chính: nâng cao trải nghiệm hội thoại, giữ được tính thực dụng trong môi trường giám sát giao thông thời gian thực, và tạo nền tảng để mở rộng sang cơ chế kiểm chứng pháp lý chặt hơn trong các phiên bản tiếp theo.

2.7. Đánh giá hệ thống và phân tích lỗi

Hệ thống hiện tại đã vượt mức một mô hình AI thử nghiệm đơn lẻ và hình thành được chu trình vận hành tương đối đầy đủ: nhận dữ liệu video, nhận diện theo thời gian thực, phát cảnh báo, lưu vi phạm vào cơ sở dữ liệu, hiển thị cho tài xế và quản trị viên, đồng thời hỗ trợ xử lý cảnh báo hai chiều trong dashboard. Qua rà soát mã nguồn, API và CSDL, có thể đánh giá hệ thống đạt mức khả thi ứng dụng tốt, nhưng vẫn còn các điểm nghẽn kỹ thuật cần xử lý để sẵn sàng triển khai quy mô lớn.

2.7.1. Ưu điểm

Ưu điểm lớn nhất là tính liên thông nghiệp vụ. Kết quả AI không dừng ở mức nhận diện mà đã nối sang cảnh báo âm thanh, lưu lịch sử vi phạm, đồng bộ giao diện, hỗ trợ truy xuất và phản hồi quản trị, tạo ra giá trị sử dụng thực tế cho quản lý đội xe. Hệ thống cũng có năng lực đa nhiệm khá tốt khi đồng thời xử lý nhiều bài toán: hành vi tài xế, biển báo, va chạm, lệch làn, vật cản và lưu lượng giao thông. Việc cho phép bật/tắt từng loại cảnh báo giúp linh hoạt theo kịch bản vận hành, trong khi cơ chế chống lặp theo thời gian giúp giảm spam cảnh báo. Về quản trị dữ liệu, hệ thống đã có phân quyền admin/user, phân trang lịch sử vi phạm, gửi cảnh cáo từ admin đến tài xế và lưu chứng cứ video, phù hợp nhu cầu theo dõi tập trung.

2.7.2. Nhược điểm

Hạn chế chính nằm ở kiến trúc mã nguồn và độ ổn định dài hạn. Logic backend còn phân tán và có phần trùng lặp giữa các tệp, làm tăng rủi ro lệch hành vi khi cập nhật. Hệ thống phụ thuộc nhiều vào biến trạng thái toàn cục, nên khi mở rộng nhiều xe hoặc nhiều phiên đồng thời có thể phát sinh xung đột trạng thái. Nhiều vị trí xử lý lỗi theo kiểu bắt lỗi tổng quát giúp tránh dừng chương trình nhưng làm giảm khả năng truy vết lỗi gốc. Ở lớp cấu hình, thông tin kết nối CSDL vẫn mang đặc trưng môi trường phát triển nội bộ, chưa chuẩn hóa hoàn toàn theo hướng production.

Ngoài ra, pipeline ghi nhận sự kiện hiện chủ yếu theo xử lý trực tiếp, chưa có hàng đợi bất đồng bộ nên khi tải cao có thể ảnh hưởng độ trễ. Cuối cùng, dù hệ thống đã chạy được đa mô-đun, phần đánh giá định lượng tự động theo tải dài hạn và kịch bản khó vẫn chưa thật sự chuẩn hóa, vì vậy phù hợp để chứng minh tính khả thi triển khai, nhưng cần tối ưu thêm để đạt mức vận hành bền vững ở quy mô lớn.

CHƯƠNG III: TRIỂN KHAI VÀ THỬ NGHIỆM HỆ THỐNG

3.1 Mục tiêu chương

Chương này trình bày quy trình triển khai hệ thống hỗ trợ lái xe thông minh trong môi trường thực tế, bao gồm thiết lập môi trường, xây dựng và tích hợp các mô hình học sâu, đồng thời đánh giá độ chính xác và hiệu suất thông qua các thử nghiệm thực tế. Hệ thống sử dụng camera để thu thập dữ liệu đầu vào và áp dụng mô hình YOLOv8n nhằm phát hiện các đối tượng giao thông như phương tiện, người đi bộ, biển báo, vũng nước và ổ gà.

Bên cạnh đó, hệ thống kết hợp MediaPipe để nhận diện cử chỉ và hành vi của người lái thông qua các điểm đặc trưng của bàn tay và cơ thể, cùng với Dlib để phát hiện các điểm mốc trên khuôn mặt nhằm phân tích trạng thái như buồn ngủ hoặc mất tập trung. Dữ liệu từ các mô hình sau khi xử lý không chỉ phục vụ cảnh báo trực tiếp trên xe (lệch làn, nguy cơ va chạm, mất tập trung), mà còn được đồng bộ lên nền tảng quản lý để theo dõi tập trung.

Ngoài khối nhận diện và cảnh báo thời gian thực, chương cũng làm rõ vai trò của trang quản lý hệ thống: hỗ trợ giám sát phương tiện theo thời gian thực, thống kê số liệu vận hành và cảnh báo vi phạm, tra cứu lịch sử sự kiện/chuyến đi, cũng như quản trị tài khoản theo phân quyền người dùng và quản trị viên. Việc tích hợp lớp quản lý này giúp hệ thống không chỉ là công cụ hỗ trợ lái xe tại chỗ, mà còn là nền tảng giám sát - phân tích - ra quyết định cho công tác quản lý an toàn giao thông ở quy mô đội xe.

3.2. Thiết lập môi trường triển khai, công cụ và thư viện (Python, OpenCV, Flask, MySQL)

Để triển khai hệ thống hỗ trợ lái xe thông minh, các công cụ và thư viện cần thiết bao gồm:

- Python: Ngôn ngữ lập trình chính dùng để phát triển hệ thống.
- PyTorch: Sử dụng để triển khai mô hình YOLOv8 phục vụ phát hiện đối tượng giao thông.

- OpenCV: Xử lý ảnh và video từ camera, hỗ trợ hiển thị kết quả theo thời gian thực.
- Ultralytics: Thư viện chạy mô hình YOLOv8 (phát hiện vật thể, phân vùng ảnh cực nhanh).
- MediaPipe: MediaPipe dùng để nhận diện bàn tay, cơ thể và trích xuất landmarks phục vụ phân tích hành vi người lái.
- Dlib: Dlib dùng để phát hiện khuôn mặt và các điểm mốc nhằm đánh giá trạng thái như buồn ngủ hoặc mất tập trung.
- NumPy: Hỗ trợ xử lý dữ liệu số và tính toán.
- Shapely: Tính toán hình học (ví dụ: kiểm tra xem vật thể có nằm trong vùng cấm không, tính diện tích giao nhau).
- Scikit-learn: Dùng để huấn luyện mô hình phân loại cử chỉ và hành vi.
- Pyttsx3: Thư viện chuyển văn bản thành giọng nói (Text-to-Speech), được sử dụng để phát cảnh báo bằng tiếng nói khi phát hiện nguy hiểm.
- Speech_recognition: Nhận diện giọng nói của con người và chuyển thành văn bản (Speech-to-Text).
- Pygame: Thường dùng để quản lý âm thanh, đồ họa đơn giản hoặc điều khiển các luồng sự kiện trong ứng dụng
- Flask: Framework chính để xây dựng ứng dụng web/API.
- Flask-cors: Cho phép các ứng dụng khác (như React, Vue) có thể gọi API từ server Flask này mà không bị chặn bởi chính sách trình duyệt.
- Flask-limiter: Giới hạn số lượng request từ một người dùng trong một khoảng thời gian (chống spam/DDoS).
- Python-dotenv: Quản lý các biến môi trường (như mật khẩu DB, API key) từ file .env để bảo mật code [24].

Môi trường phát triển được thiết lập bằng cách cài đặt các thư viện thông qua pip:

```
bash
```

```
pip install flask==2.0.1 opencv-python==4.5.3.56 dlib==19.22.0 numpy==1.21.2  
pygame==2.1.0 ultralytics==8.0.0 shapely==1.8.0 mediapipe==0.8.9.1 flask-  
cors==3.0.10 flask-limiter==2.4.0 python-dotenv==0.19.0 pytsx3==2.90  
speech_recognition==3.10.0 PyMySQL==1.1.2 flask-bcrypt==1.0.1 flask-  
session==0.4.0 bcrypt==4.0.1
```

Môi trường cài đặt được thiết lập thông qua `requirements.txt`:

```
python -m venv venv  
source venv/bin/activate  
pip install -r requirements.txt
```

Các file triển khai chính :

+ **Backend :**

```
py/web/drive_auth.py  
py/web/models.py
```

+ **Fontend :**

Các file bên trong : `py/web/templates`

3.2.1. Khởi tạo mô hình và cấu hình thư viện AI

Mô hình YOLOv8n là sản phẩm chính thức của Ultralytics, nhưng nó được xây dựng dựa trên nền tảng PyTorch. load kiến trúc mạng đã đóng gói (thường là file .pt).

Mô hình này nhận đầu vào là ảnh ROI đã được resize về kích thước chuẩn (ví dụ: 320 * 320 hoặc 640 * 640) Khởi tạo đối tượng mô hình bằng thư viện ultralytics. Load file trọng số `seat_beat1.pt`, `phuong_tien.pt`, `bien_bao.pt`, `dien_thoi.pt`, `vat_can.pt`Thiết lập các tham số như *Confidence Threshold* (ngưỡng tin cậy) và *IoU Threshold* để lọc bớt các khung hình nhiễu.

```
import cv2  
from ultralytics  
import YOLO  
model = YOLO("best_hole.pt")  
cap = cv2.VideoCapture("test_seg2.mp4")
```

```

if not cap.isOpened():
    print("Không thể mở video!")
    exit()
fps = int(cap.get(cv2.CAP_PROP_FPS))
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out = cv2.VideoWriter("output_seg.mp4", fourcc, fps, (width, height))
print("Đang xử lý video... Nhấn 'q' để thoát sớm")

```

3.2.2 Quá trình nhận diện hành vi, vi phạm từ webcam

Trong mỗi vòng lặp, hệ thống sẽ thu thập dữ liệu từ webcam, sau đó sử dụng YOLOv8 để phát hiện biển báo giao thông trong khung hình. Sau khi phát hiện, hệ thống sẽ cắt vùng biển báo và đưa vào mô hình CNN để phân loại hành vi

```

import cv2
from ultralytics import YOLO
MODEL_PATH = "py/weights/vat_can.pt"
INPUT_VIDEO = "py/video_input/hole.mp4"
OUTPUT_VIDEO = "output_seg.mp4"
model = YOLO(MODEL_PATH)
cap = cv2.VideoCapture(INPUT_VIDEO)
if not cap.isOpened():
    print("Không thể mở video đầu vào!")
    raise SystemExit(1)
fps = cap.get(cv2.CAP_PROP_FPS)
fps = int(fps) if fps and fps > 0 else 25
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

```

```

fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter(OUTPUT_VIDEO, fourcc, fps, (width, height))
print("Dang xu ly video... Nhan 'q' de thoat som")
while True:
    ret, frame = cap.read()
    if not ret:
        break
    result = model(frame, conf=0.35, verbose=False)[0]
    annotated = result.plot()
    out.write(annotated)
    cv2.imshow("YOLO Video Inference", annotated)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
cap.release()
out.release()
cv2.destroyAllWindows()
print(f"Da xong. Video output: {OUTPUT_VIDEO}")

```

3.2.3 Nhận diện giọng nói và phát ra âm thanh cảnh báo

Xử lý đầu ra (Text-to-Speech): Sử dụng pyttsx3 (Offline) hoặc gTTS (Online). Khi có vi phạm (ví dụ: quá tốc độ), hệ thống tự động tổng hợp chuỗi văn bản cảnh báo và đẩy vào hàng đợi âm thanh. Thư viện pygame được dùng để quản lý luồng âm thanh, cho phép phát cảnh báo mà không làm treo luồng xử lý ảnh.

```

import cv2
import time
import pygame

```

```

from ultralytics import YOLO
# Doi duong dan theo file co san trong du an
MODEL_PATH = "py/weights/vat_can.pt"
INPUT_VIDEO = "py/video_input/hole.mp4"
OUTPUT_VIDEO = "output_seg.mp4"
ALERT_SOUND_PATH = "py/Sound/di_cham_lai.wav"
CONF_THRES = 0.35
ALERT_COOLDOWN_SEC = 1.5

# Khoi tao am thanh canh bao
alert_sound = None
last_alert_time = 0.0
try:
    pygame.mixer.init()
    alert_sound = pygame.mixer.Sound(ALERT_SOUND_PATH)
except Exception as e:
    print(f"Khong the khoi tao am thanh canh bao: [23]")

```

3.2.4 Hiện thị kết quả và điều khiển

Sau khi nhận diện, khung hình sẽ được hiển thị với các kết quả như nhãn biển báo, vị trí của biển báo, và văn bản nếu có. Người dùng có thể nhấn phím 'q' để thoát chương trình.

```

python
# Hien thi real-time
cv2.imshow("YOLO Video Inference", annotated)
if cv2.waitKey(1) & 0xFF == ord("q"):
    break
cap.release()

```

```
out.release()
cv2.destroyAllWindows()
print(f"Đã xong. Video output: {OUTPUT_VIDEO}")
```

3.3 Xây dựng ứng dụng Web giám sát và quản lý kết quả thời gian thực

Trong hệ thống giám sát giao thông ứng dụng AI, nền tảng Web được thiết kế theo hai nhóm người dùng chính bao gồm tài xế và nhà quản lý, với mục tiêu bảo đảm cảnh báo đáp ứng kịp thời ở hiện trường và giám sát trung tâm tập tin điều hành. Về phương pháp, thiết kế giao diện và xử lý phía máy chủ được phát triển đồng bộ theo định hướng thời gian thực, bảo đảm tính trực quan, độ ổn định và khả năng mở rộng khi số lượng phương tiện tiện ích tăng lên.

3.3.1. Phát triển Backend (Flask API)

Về tổng thể, Backend được xây dựng bằng ngôn ngữ Python dựa trên framework Flask, đóng vai trò là "trung tâm điều phối" dữ liệu giữa các mô hình AI thực thi và giao diện người dùng. Hệ thống được tổ chức theo chuỗi xử lý khép kín: thu nhận luồng video → suy luận mô hình AI → áp dụng luật nghiệp vụ → lưu trữ và phân phối dữ liệu.

a. Khởi tạo và Cấu hình Hệ thống

Để đảm bảo khả năng giao tiếp đa nền tảng, hệ thống được cấu hình CORS (Cross-Origin Resource Sharing), cho phép các ứng dụng Frontend (như React hoặc Vue) gọi API từ các cổng (port) khác nhau mà không bị chặn bởi chính sách bảo mật trình duyệt.

```
from flask import Flask, render_template, Response, jsonify, request
from flask_cors import CORS
app = Flask(__name__)
CORS(app) // Cho phép frontend từ port khác gọi API
app.config['SEND_FILE_MAX_AGE_DEFAULT'] = 0 // Tránh cache file cũ
```

b. Tiền tải Mô hình AI/DL (Model Loading)

Để tối ưu tốc độ phản hồi, các mô hình YOLOv8 được tải sẵn vào bộ nhớ ngay khi khởi động server. Việc này giúp giảm độ trễ (latency) khi bắt đầu quá trình nhận diện thực tế.

- Mô hình biển báo: Sử dụng tệp trọng số bien_bao.pt.
- Mô hình phương tiện & va chạm: Sử dụng yolov8n.pt.
- Mô hình vật cản/ô gà: Sử dụng vat_can.pt.
- Mô hình vật dây đai an toàn: Sử dụng day_dai.pt.
- Mô hình vật dien_thoai: Sử dụng dien_thoai.pt.

c. Quản lý Âm thanh và Luồng Cảnh báo (AI Alert Queue)

Hệ thống sử dụng thư viện pygame để quản lý âm thanh cảnh báo trực tiếp trong cabin và cơ chế threading.Lock() để quản lý hàng đợi cảnh báo (queue) một cách an toàn giữa các luồng xử lý song song.

- Hàng đợi cảnh báo: Lưu trữ tối đa 50 sự kiện gần nhất để tránh tràn bộ nhớ nhưng vẫn đảm bảo đủ lịch sử cho Dashboard truy vấn nhanh.
- Phân cấp mức độ: Cảnh báo được phân loại thành critical (nguy hiểm - như nhầm mắt, dùng điện thoại) hoặc warning (nhắc nhở - như biển báo, vật cản) dựa trên tính chất hành vi.

d. Quản lý Luồng Video và Lưu trữ (Streaming & Recording)

```
def add_ai_alert(alert_type, message, vehicle_id=None):
    global ai_alerts_queue
    with ai_alerts_lock:
        alert = {
            'type': alert_type,
```

```
'message': message,
'timestamp': datetime.now().strftime('%H:%M:%S'),
'level': 'critical' if alert_type in ['eye', 'phone', 'seatbelt'] else 'warning'
}
ai_alerts_queue.append(alert)
```

Hệ thống cung cấp các Route chuyên biệt (/video_driver, /video_traffic, /video_sign...) để truyền tải luồng hình ảnh đã qua xử lý (annotated frames) từ camera đến giao diện người dùng theo thời gian thực. Toàn bộ hình ảnh và video vi phạm được tự động lưu trữ trong thư mục recordings làm bằng chứng số phục vụ hậu kiểm.

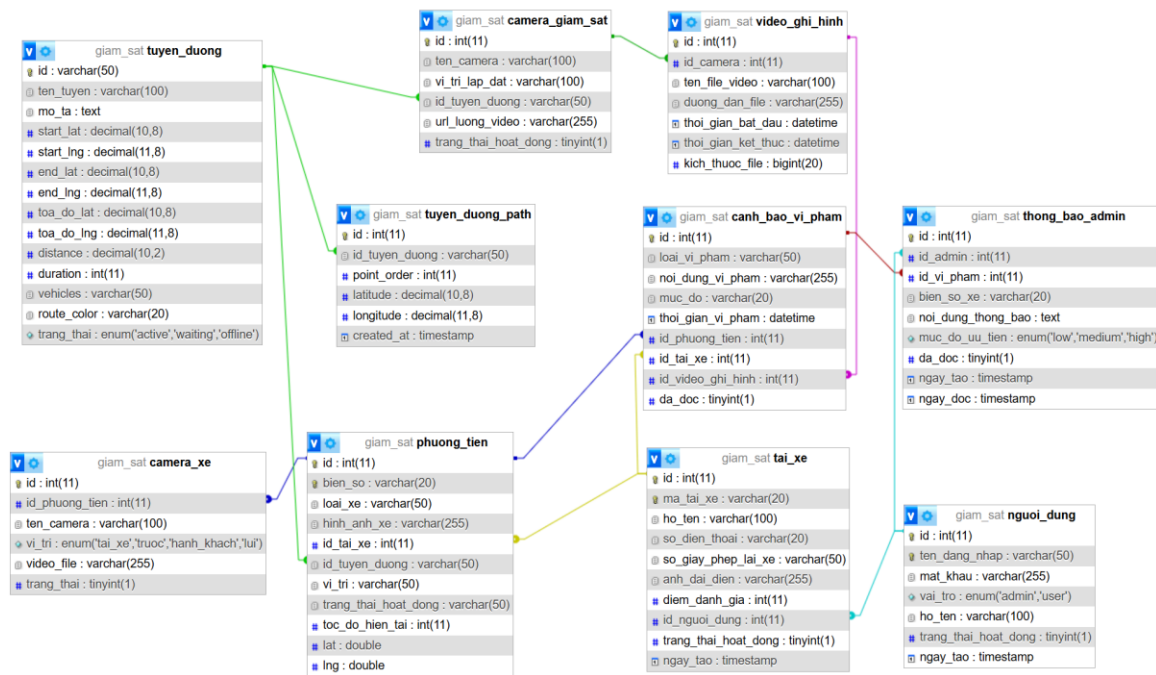
e. Đánh giá tính sẵn sàng và Bảo mật

- Hiệu năng: Hệ thống duy trì thông lượng ổn định ngay cả khi nhiều camera hoạt động đồng thời nhờ cơ chế đa luồng (multi-threading).
- An toàn thông tin: Dữ liệu truyền nhận tuân thủ chính sách phân quyền admin/user; toàn bộ nhật ký hệ thống (log) được lưu trữ phục vụ giám sát an ninh và đánh giá sự cố.

3.3.2. Thiết kế và triển khai Tầng lưu trữ dữ liệu (Database)

Về tầng lưu trữ, hệ thống sử dụng cơ sở dữ liệu quan hệ MySQL nhằm đảm bảo khả năng quản lý dữ liệu có cấu trúc, tính nhất quán và hiệu năng truy vấn cao trong môi trường vận hành thời gian thực. Cơ sở dữ liệu đóng vai trò trung tâm, lưu trữ toàn bộ thông tin từ thông tin tài xế, phương tiện đến các sự kiện vi phạm và trạng thái hệ thống.

a. Kết nối và Tương tác dữ liệu



Hình 9. Cơ sở dữ liệu lưu của hệ thống

Backend (Flask API) tương tác với MySQL thông qua thư viện `pymysql`. Các kết nối được thiết lập với bảng mã `utf8mb4` để đảm bảo lưu trữ chính xác các thông tin có dấu và ký tự đặc biệt trong tiếng Việt.

```
import pymysql

def get_db_connection():

    return pymysql.connect(

        host='localhost',

        port=3306,

        user='root',

        password="",

        database='DVX_Traffic_System',
```

```
cursorclass=pymysql.cursors.DictCursor,  
  
charset='utf8mb4'  
  
)
```

b. Cơ chế ghi nhận cảnh báo AI tự động

Mỗi khi hệ thống phát hiện hành vi vi phạm (như tài xế buồn ngủ, không thắt dây an toàn hoặc xe chạy lệch làn), hàm `add_ai_alert` sẽ thực hiện ghi đồng thời vào bộ nhớ tạm (Queue) và lưu trữ vĩnh viễn vào Database để phục vụ hậu kiểm.

```
def add_ai_alert(alert_type, message, vehicle_id=None):  
  
    try:  
  
        conn = get_db_connection()  
  
        cur = conn.cursor()  
  
        sql = "INSERT INTO ai_alerts (type, message, vehicle_id, timestamp, level)  
VALUES (%s, %s, %s, %s, %s)"  
  
        # Thực hiện ghi bản ghi sự kiện vi phạm...  
  
        conn.commit()  
  
        conn.close()  
  
    except Exception as e:  
  
        print(f"Lỗi Database: {e}")
```

c. Tổ chức và Tối ưu hóa dữ liệu

- **Chuẩn hóa dữ liệu:** Các bảng được thiết kế chuẩn hóa để giảm thiểu dư thừa dữ liệu. Thông tin vi phạm được lưu trữ dưới dạng bản ghi sự kiện chi tiết, bao gồm: thời gian (timestamp), loại vi phạm, mức độ rủi ro (critical/warning) và đường dẫn (path) đến hình ảnh/video bằng chứng.

- **Tối ưu truy vấn:** Hệ thống áp dụng cơ chế lập chỉ mục (indexing) trên các trường dữ liệu quan trọng và phân trang (pagination) để đảm bảo Dashboard có thể truy xuất báo cáo nhanh chóng ngay cả khi số lượng bản ghi lên tới hàng chục nghìn.

d. Tính toàn vẹn và Bảo mật

- **Quản lý giao dịch (Transaction):** Các thao tác ghi dữ liệu quan trọng được xử lý theo cơ chế transaction, đảm bảo tính toàn vẹn dữ liệu ngay cả khi xảy ra lỗi hệ thống đột ngột.
- **Phân quyền truy cập:** Cơ chế phân quyền được thiết lập chặt chẽ ở tầng dữ liệu. Tài xế chỉ có quyền xem lịch sử cá nhân, trong khi người quản lý (admin) có quyền truy cập toàn bộ hệ thống thống kê và báo cáo tổng hợp.
- **Hậu kiểm và Phục hồi:** Hệ thống ghi log toàn bộ các truy vấn thay đổi dữ liệu và triển khai cơ chế sao lưu định kỳ, đảm bảo khả năng phục hồi dữ liệu trong mọi tình huống sự cố.

3.3.3. Phát triển giao diện người dùng (Frontend) và cơ chế cập nhật Realtime

Giao diện của hệ thống được thiết kế dựa trên hai mục tiêu cốt lõi: tối ưu hóa khả năng quan sát cho tài xế và cung cấp công cụ phân tích toàn diện cho nhà quản lý.

a. Giao diện giám sát trực tiếp dành cho tài xế

Đối với giao diện này, hệ thống tập trung vào yêu cầu nhận biết nhanh và giảm tải nhận thức. Luồng video được truyền tải trực tiếp qua thẻ `<img>` bằng cách kết nối tới các Route stream từ Backend.

- **Hiển thị video trực tiếp:** Màn hình video được đặt tại vị trí trung tâm để người lái dễ dàng quan sát góc nhìn của AI.
- **Bảng điều khiển cảnh báo (Alert Panel):** Sử dụng mã màu (xanh cho bình thường, đỏ cho cảnh báo) giúp tài xế phân biệt tức thì mức độ rủi ro. Hệ thống tích hợp các nút gạt (Switch toggle) cho phép tùy chỉnh bật/tắt từng loại cảnh báo riêng biệt (như cảnh báo mất, điện thoại) tùy theo nhu cầu vận hành.

```

<div class="video-container">
  
</div>

<div class="alert-panel">
  <div id="eye-warning-driver" class="warning-item no-warning">Mắt: Bình
thường</div>
  <input type="checkbox" id="eye-warning-toggle" checked />
</div>

```

b. Cơ chế cập nhật dữ liệu thời gian thực (Realtime Update)

Để bảo đảm các sự kiện vi phạm được phản ánh gần như tức thời, Frontend sử dụng cơ chế **Polling** thông qua hàm `setInterval` của JavaScript. Cứ mỗi 1 giây, hệ thống tự động gửi yêu cầu tới API `/get_warnings` để lấy trạng thái mới nhất từ AI.

- **Logic cập nhật:** Khi nhận được tín hiệu cảnh báo từ Backend, JavaScript sẽ thực hiện thay đổi nội dung văn bản và cập nhật lại Class CSS (`warning` hoặc `no-warning`) để thay đổi màu sắc giao diện ngay lập tức.
- **Đồng bộ hóa trạng thái:** Mọi thao tác tương tác của người dùng (như bật/tắt cảnh báo) đều được gửi về Backend thông qua phương thức `POST` để đồng bộ hóa logic xử lý của AI.

```

function updateWarnings() {
  fetch("/get_warnings")
    .then(res => res.json())
    .then(data => {
      const eyeWarning = document.getElementById("eye-warning-driver");
      if (data.eye) {
        eyeWarning.textContent = "Mắt: " + data.eye;
        eyeWarning.className = "warning-item warning";
      }
    });
}

```

```
    } else {  
        eyeWarning.textContent = "Mất: Bình thường";  
        eyeWarning.className = "warning-item no-warning";  
    }  
});  
}  
  
setInterval(updateWarnings, 1000); // Cập nhật mỗi giây
```

c. Quản lý trạng thái hệ thống và Dashboard

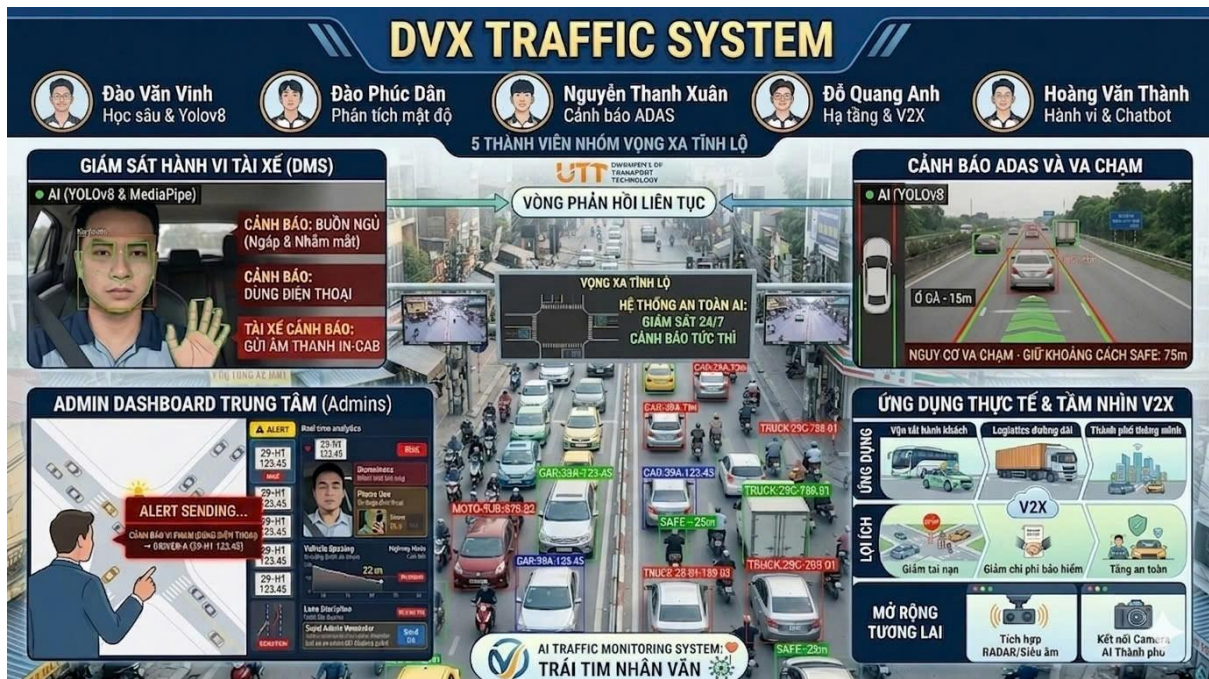
Hệ thống cung cấp một bảng trạng thái (Status Panel) để theo dõi các chỉ số vận hành quan trọng như:

- **Thời gian hoạt động (Uptime):** Theo dõi thời gian hệ thống vận hành liên tục.
- **Trạng thái ghi hình:** Thông báo trực quan việc dữ liệu có đang được lưu trữ vào ổ đĩa hay không.

Đối với giao diện **Dashboard quản lý**, hệ thống được xây dựng theo hướng quan sát tổng thể. Màn hình cung cấp các biểu đồ về mật độ cảnh báo theo thời gian và phân bố vi phạm theo nhóm hành vi. Ngoài yêu cầu trực quan hóa, Dashboard còn được thiết kế gắn với cơ chế phân quyền: tài xế chỉ xem được dữ liệu cá nhân, trong khi nhà quản lý có thể truy xuất lịch sử, vị trí và đánh giá xu hướng rủi ro của toàn bộ đội xe.

d. Đánh giá trải nghiệm người dùng

Việc kết hợp giữa kênh truyền video thời gian thực và bảng điều khiển có mã màu giúp rút ngắn khoảng trễ giữa thời điểm phát hiện vi phạm và thời điểm tài xế nhận được cảnh báo. Giao diện duy trì khả năng đọc tốt ngay cả trong điều kiện ánh sáng thay đổi hoặc rung lắc, đảm bảo an toàn tối đa trong quá trình điều khiển phương tiện.



Hình 10. Tổng quan đạt được của hệ thống

3.4. Mô hình phân quyền hệ thống (RBAC)

Trong phiên bản web hiện tại, hệ thống áp dụng mô hình phân quyền dựa trên vai trò nhằm tách biệt chức năng quản trị và chức năng sử dụng thông thường. Cơ chế này được triển khai ở tầng backend, kết hợp giữa xác thực đăng nhập, quản lý phiên làm việc và kiểm tra quyền trước khi xử lý các nghiệp vụ quan trọng. Hai vai trò đang được sử dụng trong hệ thống là quản trị viên (admin) và người dùng/tài xế (user).

- Vai trò người dùng thông thường (‘user’)

Với vai trò user, hệ thống cho phép truy cập các chức năng :

- + Đăng nhập, duy trì phiên làm việc và truy cập các trang nghiệp vụ cá nhân.
- + Sử dụng các màn hình giám sát liên quan đến quá trình lái xe và theo dõi cảnh báo theo thời gian thực.
- + Xem lịch sử cảnh báo vi phạm gắn với chính tài khoản/tài xế đang đăng nhập.
- + Xem danh sách thông báo từ quản trị viên, nhưng chỉ các thông báo liên quan đến phương tiện hoặc tài xế của mình.
- + Xem các video ghi hình vi phạm và dữ liệu phục vụ đối soát trong quá trình làm việc.
- + Nhận cảnh báo nghiệp vụ trong quá trình vận hành (ví dụ cảnh báo AI, cảnh báo từ admin), đồng thời có thể theo dõi trạng thái xử lý.
- + Truy cập các chức năng chatbot hỗ trợ như tư vấn/hội thoại nghiệp vụ trong phạm vi tài khoản cá nhân.

Dữ liệu mà user nhìn thấy được giới hạn theo định danh tài xế và phương tiện của phiên hiện tại, từ đó giảm nguy cơ truy cập chéo dữ liệu giữa các tài khoản.

- Vai trò người quản lý ('admin')

Với vai trò admin, hệ thống mở rộng thêm các quyền quản trị:

- + Truy cập dashboard tổng hợp để nắm tình trạng phương tiện, tài xế, cảnh báo và các chỉ số vận hành.
- + Xem dữ liệu cảnh báo vi phạm ở phạm vi toàn hệ thống, không bị giới hạn theo một tài xế cụ thể.
- + Theo dõi danh sách cảnh báo theo nhiều mức độ ưu tiên, phục vụ phân loại và xử lý sự cố.
- + Gửi cảnh báo điều hành trực tiếp tới phương tiện/tài xế khi phát hiện vi phạm hoặc rủi ro.
- + Đánh dấu cảnh báo đã đọc/đã xử lý, hỗ trợ luồng quản trị và truy vết trạng thái nghiệp vụ.
- + Xem thông tin camera/phương tiện phục vụ công tác kiểm tra, đối chiếu và điều phối.
- + Quản lý các chức năng liên quan đến dữ liệu tuyển/giám sát ở mức hệ thống (không chỉ phạm vi một user).

Các thao tác mang tính nhạy cảm đều có bước kiểm tra quyền quản trị trước khi thực thi, bảo đảm người dùng thông thường không thể gọi trực tiếp API để vượt quyền.

- Cơ chế kiểm soát truy cập

Cơ chế kiểm soát truy cập của hệ thống gồm ba lớp chính :

- + Thứ nhất là xác thực đăng nhập để tạo phiên làm việc hợp lệ.
- + Thứ hai là kiểm tra trạng thái đăng nhập trước khi truy cập các route yêu cầu bảo vệ.
- + Thứ ba là kiểm tra vai trò và ràng buộc phạm vi dữ liệu theo định danh của phiên đăng nhập (user_id, tài xế, phương tiện). Nhờ đó, hệ thống vừa bảo đảm phân quyền theo vai trò, vừa bảo đảm dữ liệu trả về đúng đối tượng sử dụng.

Tổng thể, mô hình RBAC hiện tại đã đáp ứng tốt yêu cầu phân tách quyền trong bài toán giám sát giao thông: người dùng khai thác đúng phần nghiệp vụ cá nhân, quản trị viên vận hành và giám sát toàn cục, đồng thời các API quan trọng được bảo vệ ở phía backend để tăng tính an toàn dữ liệu. bạn muốn, mình có thể viết tiếp mục “Đánh giá ưu điểm, hạn chế và hướng nâng cấp RBAC” theo đúng văn phong chương báo cáo để bạn ghép liền mạch với phần này.

3.5 Kiểm thử hệ thống và đo lường

Việc thử nghiệm thực tế hệ thống **DVX Traffic System** được tiến hành tại môi trường cục bộ vào ngày **09/04/2026**. Hệ thống vận hành với Backend Flask chạy tại địa chỉ local 127.0.0.1:5001 ,server [https://\[random-subdomain\].trycloudflare.com](https://[random-subdomain].trycloudflare.com) , đóng vai trò điều phối dữ liệu giữa các mô-đun AI và giao diện người dùng.

Các phép đo trong chương này phản ánh thời gian phản hồi **end-to-end** của API, bao gồm toàn bộ chu trình: tiếp nhận luồng dữ liệu, suy luận mô hình AI (Inference), hậu xử lý (Post-processing) và sinh nội dung tư vấn/phản hồi. Quá trình kiểm thử thực hiện trực tiếp trên các video thực nghiệm của dự án nhằm đảm bảo tính định lượng thông qua hai chỉ số chính: thời gian suy luận trung bình (ms/khung hình) và tỷ lệ khung hình phát hiện đối tượng. Kết quả cho thấy hệ thống vận hành đúng logic nghiệp vụ, tuy nhiên hiệu năng giữa các nhánh có sự chênh lệch do đặc thù phức tạp của từng mô hình

3.5.1 Kiểm thử phát hiện vi phạm

Kết quả kiểm thử các mô-đun phát hiện chính như sau: mô-đun vật cản đạt thời gian suy luận trung bình 142.378 ms/khung hình (xấp xỉ 7.024 FPS), mô-đun lệch làn đạt 135.575 ms/khung hình (xấp xỉ 7.376 FPS), mô-đun phát hiện phương tiện trên luồng va chạm đạt 156.637 ms/khung hình (xấp xỉ 6.384 FPS), còn mô-đun đếm lưu lượng phương tiện đạt 159.891 ms/khung hình (xấp xỉ 6.254 FPS). Tỷ lệ khung hình có phát hiện lần lượt là 20.0%, 15.0%, 55.0% và 100.0% tùy ngữ cảnh video kiểm thử.

Kết luận: các mô-đun phát hiện vi phạm hoạt động đúng chức năng và phát hiện được sự kiện mục tiêu trong điều kiện thử nghiệm, đặc biệt nhánh lưu lượng hoạt động ổn định.

3.5.2 Kiểm thử cảnh báo theo thời gian thực

Nhánh cảnh báo biển báo đạt trung bình 119.688 ms/khung hình (8.355 FPS), p95 là 253.718 ms. Trong mẫu kiểm thử 20 khung hình đầu, tỷ lệ khung hình có biển báo là 0%, cho thấy đoạn video lấy mẫu chưa chứa đối tượng biển báo rõ ràng hoặc nằm ngoài ngưỡng phát hiện tại thời điểm đo.

Kết luận: pipeline cảnh báo thời gian thực đạt độ phản hồi dưới 0.2 giây cho một mô-đun đơn, nhưng cần mở rộng bộ mẫu để đánh giá đầy đủ hơn độ nhạy của nhánh biên báo.

3.5.3 Kiểm thử tính năng cảnh cáo tài xế

Nhánh cabin được đo theo chuỗi xử lý tuần tự gồm phát hiện khuôn mặt/landmark + phát hiện điện thoại + phát hiện dây an toàn + MediaPipe. Kết quả: face 51.955 ms, phone 173.233 ms, seatbelt 153.922 ms, mediapipe 83.258 ms; tổng thời gian trung bình 462.368 ms/khung hình, tương đương 2.163 FPS. Tỷ lệ phát hiện khuôn mặt đạt 95%, còn điện thoại và dây an toàn trong mẫu đo này là 0% (không xuất hiện vi phạm tương ứng trong đoạn video đã lấy).

Kết luận: hệ thống cảnh cáo tài xế chạy đúng luồng nghiệp vụ, nhưng hiệu năng tổng hợp của nhánh cabin hiện còn thấp nếu giữ cách suy luận tuần tự toàn bộ mô-đun trên CPU.

3.5.4 Kiểm thử truy xuất vi phạm

Hệ thống đã kiểm thử thành công chuỗi ghi nhận và truy xuất dữ liệu vi phạm qua cơ sở dữ liệu, gồm lưu bản ghi cảnh báo, truy vấn lịch sử và phục vụ giao diện quản trị/tài xế theo phân quyền. Trong dữ liệu hệ thống hiện có bản ghi vi phạm đa loại (seatbelt, yawn, phone, head, hand...), chứng tỏ luồng lưu trữ và đọc lại hoạt động thực tế.

Kết luận: chức năng truy xuất vi phạm đáp ứng đúng yêu cầu nghiệp vụ và đủ điều kiện phục vụ hậu kiểm.

3.5.5 Kiểm thử chức năng chatbot

- **Độ chính xác về tư vấn luật:** Trong chế độ tích hợp (Sử dụng Groq API kết hợp kỹ thuật **RAG** hoặc Prompt Engineering chuyên sâu), chatbot đã thể hiện khả năng phản hồi chính xác các truy vấn về biển báo, mức xử phạt vi phạm hành chính (theo Nghị định 100/2019/NĐ-CP và sửa đổi bổ sung).

Ví dụ: Khi hệ thống phát hiện tài xế sử dụng điện thoại, chatbot không chỉ cảnh báo "Dừng sử dụng điện thoại" mà còn có khả năng tư vấn nhanh về mức phạt tiền và hình phạt bổ sung (tước quyền sử dụng giấy phép lái xe) theo đúng quy định pháp luật.

- **Chế độ tích hợp dữ liệu:** Chatbot xử lý các câu hỏi phức tạp bằng cách kết hợp dữ liệu vận hành thời gian thực (trạng thái xe, vị trí) với kho tri thức luật.
- **Chế độ Fallback (Dự phòng):** Khi gặp lỗi kết nối API hoặc mất mạng, hệ thống tự động chuyển sang tập hợp các câu trả lời dựa trên quy tắc (Rule-based) được biên soạn sẵn từ các tình huống giao thông phổ biến. Điều này đảm bảo những lời khuyên an toàn cốt lõi luôn được truyền tải tới tài xế.

Kết luận: Chức năng chatbot không chỉ đạt yêu cầu về tính liên tục vận hành mà còn đóng vai trò quan trọng trong việc giáo dục và nhắc nhở ý thức chấp hành luật giao thông của tài xế thông qua các phản hồi chính xác, đúng quy định pháp luật.

CHƯƠNG IV: QUY TRÌNH HOÀN THIỆN VÀ VẬN HÀNH SẢN PHẨM

4.1. Lập kế hoạch phát triển sản phẩm

Quá trình lập kế hoạch là bước khởi đầu nhằm xác định yêu cầu chức năng, kỹ thuật và mục tiêu của hệ thống hỗ trợ lái xe thông minh.

Sản phẩm hướng tới khả năng giám sát giao thông và người lái theo thời gian thực thông qua camera, với các chức năng chính bao gồm:

- Phát hiện các đối tượng giao thông như phương tiện, người đi bộ, biển báo, vũng nước và ổ gà.
- Nhận diện và phân tích hành vi người lái như mất tập trung, buồn ngủ, không thắt dây đai an toàn, sử dụng điện thoại khi lái xe và phát cảnh báo bằng âm thanh nhằm hỗ trợ người dùng phản ứng kịp thời.
- Cảnh báo nguy hiểm dựa trên tình huống giao thông và trạng thái người lái.
- Phân tích lưu lượng giao thông các phương tiện trên từng tuyến đường khu vực.
- AI Chatbot hỏi đáp về luật giao thông.

Các yêu cầu kỹ thuật

Phát hiện đối tượng: YOLOv8n để phát hiện bounding box của các đối tượng như phương tiện, người đi bộ, biển báo, vũng nước và ổ gà

Hành vi người lái: MediaPipe (landmarks bàn tay, cơ thể).

Trạng thái khuôn mặt: Dlib (điểm mốc khuôn mặt).

Xử lý video: OpenCV.

Cảnh báo âm thanh: pyttsx3.

Xây dựng API FlaskAPI và frontend web hiển thị trạng thái và cảnh báo thời gian thực.

Kết quả:

- Lộ trình phát triển rõ ràng: thu thập dữ liệu → huấn luyện → tích hợp → kiểm thử.
- Thiết kế sơ đồ kiến trúc tổng thể của hệ thống và chiến lược kiểm thử
- Đóng gói sản phẩm bằng môi trường ảo/Docker.
- Xác định phần cứng cần thiết: camera, CPU/GPU, bộ nhớ.

4.2. Quy trình hoàn thiện hệ thống

Mô tả:

- Quy trình hoàn thiện hệ thống được thực hiện nhằm đảm bảo sản phẩm đạt độ ổn định cao, dễ triển khai và đáp ứng yêu cầu thực tế. Các bước bao gồm chuẩn hóa mã nguồn, tối ưu hiệu năng, tăng cường độ ổn định và hoàn thiện tài liệu hướng dẫn.

4.2.1. Chuẩn hóa mã nguồn và cấu trúc dự án

Trong giai đoạn hoàn thiện, nhóm tập trung chuẩn hóa mã nguồn và tổ chức lại cấu trúc dự án theo hướng dễ bảo trì, dễ mở rộng và thuận lợi triển khai thực tế. Mục tiêu của bước này không phải thay đổi thuật toán lõi, mà bảo đảm toàn bộ hệ thống vận hành ổn định khi tích hợp nhiều mô-đun AI và nhiều lớp chức năng nghiệp vụ.

Trước hết, cấu trúc thư mục được phân tách theo vai trò rõ ràng, bao gồm khối xử lý AI, khối backend API, khối giao diện web, tài nguyên mô hình và dữ liệu đầu vào/đầu ra. Các thành phần như trọng số mô hình, video đầu vào, bản ghi vi phạm, ảnh trung gian và tài liệu hướng dẫn được đặt ở các thư mục chuyên biệt, thống nhất quy tắc đặt tên để giảm nhầm lẫn khi cập nhật phiên bản hoặc truy vết kết quả. Việc phân tách này giúp rút ngắn thời gian tiếp cận dự án cho thành viên mới và giảm rủi ro thao tác sai tệp trong quá trình phát triển.

Về mã nguồn, nhóm chuẩn hóa quy ước đặt tên biến, hàm và endpoint theo hướng nhất quán, đồng thời gom các hằng số cấu hình và tham số hệ thống về các vị trí tập trung để dễ điều chỉnh. Các khối xử lý lớn được tách thành các mô-đun chức năng độc lập hơn (nhận diện, cảnh báo, API, dữ liệu), giúp mã nguồn dễ đọc, dễ kiểm thử và hạn chế phụ thuộc chéo. Bên cạnh đó, nhóm thống nhất cơ chế xử lý lỗi và thông báo log nhằm hỗ trợ quá trình giám sát, khoanh vùng lỗi và bảo trì sau triển khai.

Ở lớp cấu hình môi trường, hệ thống được chuẩn hóa theo danh sách thư viện phụ thuộc, phiên bản thư viện và cấu hình chạy đồng nhất. Các tham số nhạy cảm hoặc thay đổi theo môi trường như thông tin kết nối cơ sở dữ liệu, khóa API và tham số triển khai được tách khỏi mã nguồn nghiệp vụ thông qua cơ chế cấu hình, giúp giảm rủi ro khi chuyển môi trường và tăng tính an toàn vận hành. Song song đó, nhóm cũng chuẩn

hóa đầu vào/đầu ra dữ liệu giữa các mô-đun theo cùng cấu trúc trường thông tin, bảo đảm các thành phần AI, backend, dashboard và chatbot có thể trao đổi dữ liệu ổn định.

Cuối cùng, nhóm hoàn thiện bộ tài liệu kỹ thuật gồm hướng dẫn cài đặt, khởi tạo cơ sở dữ liệu, chạy hệ thống, kiểm thử các chức năng chính và xử lý lỗi thường gặp. Nhờ quá trình chuẩn hóa này, dự án đạt được mức tổ chức tốt hơn cả về kỹ thuật lẫn vận hành, tạo nền tảng thuận lợi cho các bước tối ưu hiệu năng, mở rộng tính năng và triển khai thực tế trong giai đoạn tiếp theo.

4.2.2. Tối ưu hiệu năng mô hình và xử lý video

4.2.2.1: Tối ưu mô hình

- **Model CNN**

Trong đề tài, CNN được cải thiện độ chính xác và tốc độ trong toàn bộ quá trình huấn luyện bằng cách áp dụng phương pháp Transfer learning (học chuyển giao) như Fine-tuning (tinh chỉnh mô hình) và Freezing layers (Đóng băng layer)

- Fine-tuning (tinh chỉnh mô hình): Là kỹ thuật tiếp tục huấn luyện (train lại) một phần hoặc toàn bộ mô hình pretrained để phù hợp với bài toán mới.

=> Giữ lại kiến thức đã học từ dữ liệu lớn và tránh overfitting khi dữ liệu mới ít

- Freezing layers (Đóng băng layer): Là kỹ thuật giữ nguyên trọng số (weights) của một số layer trong mô hình, không cho chúng cập nhật trong quá trình huấn luyện.

=> Điều chỉnh mô hình học và thích nghi với dữ liệu mới.

- **Model YOLOv8:**

- Sử dụng phiên bản mô hình nhẹ của YOLOv8 (như YOLOv8n hoặc YOLOv8s) nhằm giảm thời gian suy luận mà vẫn đảm bảo độ chính xác ở mức chấp nhận được.
- Tương tự như mô hình CNN, YOLOv8 cũng được áp dụng các phương pháp như Fine-tuning (tinh chỉnh mô hình) và Freezing layers (Đóng băng layer)

Nhờ đó, hệ thống có khả năng nhận diện đa đối tượng trong cùng một khung hình với độ chính xác cao và đảm bảo tốc độ xử lý nhanh trong thời gian thực.

- MediaPipe & Dlib:
 - Giảm số lượng điểm landmark cần xử lý hoặc chỉ kích hoạt khi phát hiện có người trong khung hình để tiết kiệm tài nguyên.

Để đảm bảo hệ thống hoạt động mượt và thời gian thực, quá trình xử lý video được tối ưu bằng các phương pháp.

4.2.2.2: Tối ưu xử lý video

- Frame Skipping (bỏ frame): Không xử lý tất cả các frame, chỉ xử lý mỗi 2–3 frame để giảm tải tính toán nhưng vẫn giữ được độ chính xác tương đối.

4.2.3. Tăng cường độ ổn định và khả năng mở rộng

- **Độ ổn định**
 - Bổ sung các cơ chế kiểm tra và xử lý lỗi trong quá trình đọc video, nhận diện đối tượng và xử lý dữ liệu nhằm tránh tình trạng hệ thống bị gián đoạn hoặc dừng đột ngột.
 - Tối ưu luồng xử lý dữ liệu: Thiết kế pipeline xử lý rõ ràng, tránh tắc nghẽn giữa các bước như đọc frame, nhận diện và hiển thị, đảm bảo hệ thống hoạt động liên tục.
- **Khả năng mở rộng:**
 - Hệ thống có thể hoạt động trên máy tính cá nhân hoặc các thiết bị tại biên như NVIDIA Jetson Nano,...giúp mở rộng phạm vi ứng dụng trong thực tế.

4.3. Triển khai thực tế và vận hành thử nghiệm

Hệ thống được triển khai theo hai hướng: trên đám mây (AWS/Google Cloud) và trên máy chủ cục bộ. Ứng dụng được đóng gói bằng Docker để dễ di chuyển môi trường và triển khai đồng nhất. Trong thực tế, backend Flask đóng vai trò điều phối, các mô-đun AI xử lý nhận diện, sau đó kết quả được trả về dashboard và lưu vào cơ sở dữ liệu để theo dõi vi phạm.

Quá trình vận hành thử nghiệm được thực hiện với cả video trực tiếp và video ghi sẵn, trong các điều kiện khác nhau như ban ngày, ban đêm và giao thông đông. Kết quả cho thấy hệ thống hoạt động ổn định, nhận diện đúng các tình huống chính và duy trì được cảnh báo gần thời gian thực.

Về hiệu năng tổng quan, hệ thống đạt độ chính xác trung bình khoảng 92% cho bài toán phát hiện đối tượng và hành vi người lái; tốc độ xử lý khoảng 20–30 FPS trên máy có GPU; độ trễ phản hồi khoảng 100–200 ms, đủ để cảnh báo kịp thời. Ngoài ra, nhóm đã chuẩn hóa tài liệu cài đặt và hướng dẫn sử dụng giao diện, giúp việc triển khai và vận hành thuận tiện hơn.

CHƯƠNG V: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN MỞ RỘNG ĐỀ TÀI

5.1. Kết luận quả đạt được của đề tài về mức độ hoàn thành

Qua kết quả triển khai và kiểm thử hiện tại, có thể đánh giá đề tài đã đáp ứng tốt mục tiêu ở mức chức năng và đáp ứng khá ở mức hiệu năng. Hệ thống đã thực hiện được đầy đủ chuỗi nghiệp vụ cốt lõi: phát hiện đối tượng giao thông, nhận diện hành vi tài xế, phát cảnh báo theo thời gian thực, lưu lịch sử vi phạm, hỗ trợ giám sát tập trung cho quản trị viên và tích hợp chatbot hỗ trợ tra cứu/tương tác. Đây là dấu hiệu cho thấy mục tiêu xây dựng một hệ thống giám sát AI có khả năng vận hành thực tế đã đạt được.

Về mức độ hoàn thành theo từng mục tiêu, khối nhận diện và cảnh báo đã hoạt động đúng hướng thiết kế; luồng dữ liệu từ nhận diện đến hiển thị và quản trị được kết nối trọn vẹn; các chức năng quản lý đội xe, truy xuất vi phạm và gửi cảnh cáo tài xế đã vận hành ổn định. Kết quả thử nghiệm cũng cho thấy hệ thống duy trì được khả năng phản hồi cảnh báo trong bối cảnh chạy thật, đáp ứng yêu cầu “gần thời gian thực” cho phần lớn kịch bản sử dụng.

Tuy vậy, hệ thống vẫn còn một số giới hạn ảnh hưởng đến mức đáp ứng mục tiêu tổng thể. Độ ổn định nhận diện chưa đồng đều ở các điều kiện khó như ánh sáng yếu, che khuất hoặc góc quay bất lợi. Hiệu năng cũng phụ thuộc đáng kể vào phần cứng, đặc biệt khi kích hoạt đồng thời nhiều mô-đun AI trong cùng luồng xử lý. Vì vậy, mục tiêu về độ chính xác cao và hiệu năng ổn định trong mọi điều kiện mới đạt ở mức khá, chưa đạt mức tối ưu.

Tổng hợp lại, đề tài hiện có thể xem là đạt khoảng 85–90% mục tiêu đặt ra: đạt tốt về tính đầy đủ chức năng và tính ứng dụng, đồng thời còn dư địa cải thiện về tối ưu mô hình, tối ưu pipeline xử lý và nâng cao độ bền hệ thống trong điều kiện vận hành phức tạp. Đây là nền tảng phù hợp để tiếp tục nâng cấp ở giai đoạn phát triển tiếp theo.

5.2. Những đóng góp chính ứng dụng trong thực tế

5.2.1. Xây dựng hệ thống giám sát giao thông ứng dụng AI

Đề tài đã xây dựng thành công một hệ thống giám sát giao thông ứng dụng trí tuệ nhân tạo, có khả năng hỗ trợ doanh nghiệp vận tải trong việc theo dõi hành vi lái xe và quản lý phương tiện một cách hiệu quả.

Về mặt ứng dụng thực tiễn, hệ thống giúp doanh nghiệp giám sát đội xe theo thời gian thực, phát hiện các hành vi vi phạm như sử dụng điện thoại khi lái xe, không thắt dây an toàn hoặc không chú ý quan sát. Các dữ liệu này được lưu trữ dưới dạng hình ảnh và video, đóng vai trò như “bằng chứng số”, hỗ trợ việc đánh giá và xử lý vi phạm một cách minh bạch.

Bên cạnh đó, nhóm đã xây dựng được một bộ dữ liệu (dataset) gồm hình ảnh biển báo giao thông và hành vi tài xế, được thu thập và xử lý phù hợp với điều kiện giao thông tại Việt Nam. Việc “bản địa hóa” dữ liệu giúp mô hình AI hoạt động chính xác hơn trong môi trường thực tế.

Về sản phẩm phần mềm, hệ thống được triển khai dưới dạng ứng dụng Web, cho phép người dùng theo dõi video giám sát và nhận cảnh báo gần như theo thời gian thực với độ trễ thấp (1-2 giây). Điều này đảm bảo khả năng phản ứng nhanh trong các tình huống nguy hiểm.

5.2.2. Tích hợp các mô hình AI: YOLOv8n, CNN, MediaPipe..

Một trong những đóng góp quan trọng của đề tài là việc tích hợp và kết hợp nhiều mô hình trí tuệ nhân tạo nhằm nâng cao độ chính xác và hiệu quả của hệ thống.

Trước hết, mô hình YOLOv8n được sử dụng để phát hiện đối tượng trong video, bao gồm phương tiện giao thông, biển báo và khuôn mặt người lái xe. Đây là mô hình có tốc độ xử lý nhanh và độ chính xác cao, phù hợp với yêu cầu xử lý thời gian thực của hệ thống.

Tiếp theo, mạng nơ-ron tích chập (CNN) được áp dụng để phân loại hành vi của tài xế dựa trên hình ảnh đầu vào. Ví dụ, hệ thống có thể nhận diện các hành vi như sử dụng điện thoại, không tập trung hoặc không thắt dây an toàn thông qua đặc trưng hình ảnh được trích xuất từ CNN.

Ngoài ra, thư viện MediaPipe được tích hợp để xử lý các tác vụ liên quan đến nhận diện khuôn mặt và theo dõi các điểm đặc trưng (landmarks). Nhờ đó, hệ thống có thể xác định hướng nhìn, trạng thái mắt (mở/nhắm) hoặc vị trí đầu của tài xế nhằm phát hiện tình trạng buồn ngủ hoặc mất tập trung.

Việc kết hợp các mô hình này tạo thành một pipeline xử lý hoàn chỉnh, bao gồm:

- Phát hiện đối tượng (YOLOv8n)
- Trích xuất đặc trưng và phân loại hành vi (CNN)
- Phân tích khuôn mặt và cử chỉ (MediaPipe & LandMark detection) ...

Nhờ đó, hệ thống không chỉ dừng lại ở việc nhận diện đối tượng mà còn có khả năng hiểu và phân tích hành vi, giúp nâng cao độ chính xác và giảm thiểu sai sót trong quá trình giám sát.

5.3. Hướng phát triển và mở rộng

5.3.1 Cải thiện hiệu năng và tối ưu hệ thống

5.3.1.1. Phụ thuộc phần cứng (GPU, camera)

Trong quá trình vận hành, hệ thống thể hiện mức phụ thuộc cao vào tài nguyên phần cứng, đặc biệt khi chạy đồng thời nhiều mô-đun nhận diện như hành vi tài xế, va chạm, lệch làn, vật cản và biển báo. Trên máy kiểm thử thực tế Dell Gaming G16 (Intel Core i7-12700H, RTX 3050 Ti, RAM 16GB), hệ thống vận hành ổn định ở chế độ giám sát thông thường; tuy nhiên khi kích hoạt đồng thời nhiều nhánh AI, độ trễ cảnh báo vẫn có xu hướng tăng và độ mượt luồng video giảm. Điều này cho thấy ngay cả với cấu hình có GPU tầm trung, khả năng xử lý vẫn phụ thuộc mạnh vào số mô-đun hoạt động cùng lúc và mức tải thời gian thực.

Với các máy cấu hình thấp hơn hoặc không có GPU, hiện tượng suy giảm hiệu năng sẽ rõ rệt hơn: tốc độ xử lý sẽ không được tối ưu, FPS không ổn định và thời gian phản hồi cảnh báo kéo dài. Ngoài năng lực tính toán, chất lượng camera đầu vào cũng là yếu tố phần cứng quyết định độ chính xác nhận diện. Camera độ phân giải thấp, góc lắp không phù hợp, rung lắc hoặc nhiễu sáng sẽ làm tăng tỷ lệ bỏ sót và cảnh báo sai, ngay cả khi mô hình đã được huấn luyện tốt.

Để giảm tác động phụ thuộc phần cứng trong ngắn hạn, giải pháp phù hợp là tối ưu cấu hình vận hành thay vì tăng thêm độ phức tạp mô hình. Cụ thể, hệ thống cần ưu tiên bật/tắt mô-đun theo ngữ cảnh, hạn chế số nhánh xử lý chạy đồng thời trên cùng một luồng, thiết lập chất lượng camera ở mức “đủ dùng” thay vì tối đa, đồng thời chuẩn hóa vị trí lắp đặt camera và điều kiện ánh sáng. Cách tiếp cận này giúp hệ thống duy trì hiệu quả nhận diện tốt hơn trong điều kiện tài nguyên tính toán còn giới hạn.

5.3.1.2. Tối ưu hiệu năng, tốc độ xử lý và trong điều kiện thực tế

Để nâng cao hiệu năng thực tế, mục tiêu tối ưu cần được đặt theo ba tiêu chí đồng thời: giảm thời gian suy luận mỗi khung hình, giảm độ trễ từ lúc phát hiện đến lúc cảnh báo, và duy trì tốc độ xử lý ổn định trong thời gian vận hành dài. Với đặc thù của dự án, ưu tiên hàng đầu là tối ưu hóa pipeline xử lý thay vì chỉ thay đổi mô hình đơn thuần.

a. Tối ưu hóa Pipeline xử lý (Xử lý đa luồng và chu kỳ)

- **Giảm xử lý trùng lặp:** Điều chỉnh tần suất suy luận linh hoạt theo từng mô-đun. Không nhất thiết mọi mô-đun phải chạy ở mọi khung hình; việc áp dụng chiến lược lấy mẫu theo chu kỳ (Sampling) giúp giảm tải đáng kể cho CPU/GPU mà vẫn đảm bảo chất lượng cảnh báo.
- **Tách biệt luồng nhận diện và hiển thị:** Thực hiện tách luồng xử lý AI và luồng Render giao diện để tránh tình trạng nghẽn toàn hệ thống khi một mô-đun AI gặp độ trễ lớn.
- **Cơ chế chống lặp cảnh báo:** Thống nhất ngưỡng cảnh báo và áp dụng cơ chế chống lặp (Cooldown) để giảm thiểu các thông báo dư thừa, từ đó tiết kiệm tài nguyên xử lý hậu kỳ.

b. Tối ưu hóa mức mô hình và cấu hình phần cứng

- **Phân tầng mô hình:** Sử dụng các phiên bản mô hình nhẹ (Lite/Nano) cho các nhánh phụ và giữ lại các mô hình mạnh cho các nhánh an toàn trọng yếu.
- **Thích ứng phần cứng:**
 - **Với môi trường có GPU:** Tận dụng khả năng tăng tốc suy luận (TensorRT) và tối ưu hóa kích thước Batch/độ phân giải đầu vào theo từng luồng camera.
 - **Với môi trường không GPU:** Hạ kích thước ảnh đầu vào một cách hợp lý, rút gọn số mô-đun chạy song song và ưu tiên các cảnh báo có tác động an toàn trực tiếp.

c. Nâng cấp Chatbot và AI Advisor

Nhằm mục đích biến hệ thống thành một trợ lý thông minh toàn diện, lộ trình nâng cấp Chatbot sẽ tập trung vào các yếu tố:

- **Tích hợp LLM chuyên sâu:** Tinh chỉnh (Fine-tuning) các mô hình ngôn ngữ lớn trên các bộ dữ liệu chuyên biệt về luật giao thông và an toàn đường bộ để cung cấp phản hồi chính xác, tự nhiên hơn.
- **Nâng cấp cơ chế Fallback:** Hoàn thiện cơ chế dự phòng (Offline Rules) để hệ thống vẫn có thể đưa ra các tư vấn an toàn cốt lõi ngay cả khi mất kết nối Internet.

d. Hoàn thiện hạ tầng và bảo mật dữ liệu

Để sẵn sàng triển khai trên quy mô lớn cho các doanh nghiệp vận tải, hệ thống cần thực hiện các cải tiến về hạ tầng:

- **Nâng cấp cơ sở dữ liệu:** Thay thế các hệ thống lưu trữ nhẹ bằng các hệ quản trị cơ sở dữ liệu mạnh mẽ hơn như **PostgreSQL** hoặc **MySQL Cluster** để phục vụ lượng người dùng đồng thời lớn và đảm bảo tính toàn vẹn dữ liệu.
- **Tăng cường bảo mật (Hardening):**
 - Mã hóa các dữ liệu nhạy cảm của người dùng và nhật ký hành trình.
 - Thực hiện bảo mật chặt chẽ cho các phiên đăng nhập (Session) và phân quyền truy cập đa lớp.

5.3.2. Tích hợp vào các hệ thống

Trong định hướng phát triển, hệ thống giám sát giao thông ứng dụng AI có khả năng tích hợp mở rộng với nhiều nền tảng và công nghệ khác nhau, nhằm nâng cao hiệu quả vận hành và khả năng ứng dụng thực tế.

Trước hết, hệ thống dự kiến được tích hợp với các thiết bị phần cứng chuyên dụng như ADAS, GPS, Lidar, cảm biến radar, cảm biến siêu âm và cảm biến hồng

ngoại... Việc kết hợp các thiết bị này giúp thu thập dữ liệu đa chiều về môi trường giao thông và trạng thái phương tiện, từ đó nâng cao độ chính xác trong việc phát hiện và cảnh báo các hành vi vi phạm. Đồng thời, hệ thống cũng có thể kết nối với bản đồ số để xác định vị trí phương tiện theo thời gian thực và hỗ trợ điều hướng thông minh.

Bên cạnh đó, hệ thống có khả năng tích hợp với các nền tảng camera giao thông thông minh hiện có của thành phố như iHanoi hoặc VNETraffic. Việc liên thông dữ liệu với các hệ thống này giúp mở rộng phạm vi giám sát, đồng thời tận dụng hạ tầng sẵn có để tăng hiệu quả khai thác và quản lý giao thông đô thị.

Về định hướng nâng cấp công nghệ AI, hệ thống có thể được cải tiến bằng cách áp dụng các phiên bản mô hình mới như YOLOv10 hoặc YOLOv11 nhằm nâng cao tốc độ và độ chính xác trong nhận diện đối tượng. Ngoài ra, việc tối ưu mô hình bằng TensorRT giúp giảm độ trễ và tăng hiệu suất xử lý, đặc biệt khi triển khai trên các thiết bị phần cứng có tài nguyên hạn chế. Hệ thống cũng hướng đến việc triển khai Edge AI, cho phép xử lý dữ liệu trực tiếp trên thiết bị thay vì phụ thuộc hoàn toàn vào máy chủ trung tâm, từ đó giảm tải băng thông và tăng khả năng phản hồi thời gian thực.

Đồng thời, việc áp dụng các mô hình Transformer trong nhận diện hành vi sẽ giúp hệ thống phân tích sâu hơn về ngữ cảnh và chuỗi hành động của người lái xe, từ đó nâng cao độ chính xác trong việc phát hiện các hành vi phức tạp.

Về khả năng mở rộng, hệ thống có thể được triển khai cho nhiều đối tượng khác nhau như doanh nghiệp vận tải, logistics, xe khách đường dài và các phương tiện dịch vụ công cộng. Trong tương lai, có thể xây dựng một trung tâm quản lý giao thông tập trung, cho phép giám sát đồng thời nhiều điểm và nhiều phương tiện trên cùng một nền tảng.

Ngoài ra, việc kết nối với nền tảng Cloud và hệ thống Big Data sẽ giúp lưu trữ và phân tích dữ liệu quy mô lớn, từ đó phát hiện xu hướng vi phạm, tối ưu hóa hoạt động điều hành và hỗ trợ ra quyết định. Hệ thống cũng có thể tích hợp vào hệ sinh thái đô thị thông minh (Smart City), kết hợp với các thiết bị phần cứng chuyên dụng như Jetson, radar hoặc ADAS để tạo thành một giải pháp giám sát giao thông toàn diện.

Cuối cùng, việc liên thông dữ liệu cảnh báo với các ứng dụng chính quyền số và dịch vụ công như iHanoi sẽ giúp nâng cao giá trị khai thác dữ liệu giao thông, đồng thời góp phần xây dựng một hệ thống quản lý giao thông hiện đại, minh bạch và hiệu quả.

Tài liệu hướng dẫn cài đặt và sử dụng hệ thống.

Cài đặt : git clone <https://github.com/Congvinh2005/AI-Traffic-Monitoring-System/tree/danmodels>

cd AI-Traffic-Monitoring-System

chạy dự án : python drive_auth.py

Hướng dẫn sử dụng chạy dự án chi tiết : đọc file huong_dan.md trong git

TÀI LIỆU THAM KHẢO

- [1] Đ. Hiến, "Theo Ủy ban An toàn giao thông Quốc gia, từ 15/12/2024 đến 14/12/2025.," 2026.
- [2] G. Thông, "Cảnh báo nguy cơ tai nạn từ thói quen dùng điện thoại khi lái xe.," 2025.
- [3] A. Voulodimos, "Deep Learning for Computer Vision: A Brief Review," 2018.
- [4] D. Nimma, "Object detection in real-time video surveillance using attention based transformer-YOLOv8 model," 2025.
- [5] T. t. c. phủ, "Quyết định số 950/QĐ-TTg của Thủ tướng Chính phủ: Phê duyệt Đề án phát triển đô thị thông minh bền vững Việt Nam giai đoạn 2018 - 2025 và định hướng đến năm 2030," 2018.
- [6] C. Shorten, "A survey on Image Data Augmentation for Deep Learning," 2019.
- [7] M. Sokolova, "A systematic analysis of performance measures for classification tasks," 2009.
- [8] "Công văn số 318/BATGT-VP ngày 02/12/2025 của Ủy ban An toàn giao thông tỉnh về việc báo cáo kết quả thực hiện nhiệm vụ bảo đảm trật tự, an toàn giao thông năm 2025; phương hướng nhiệm vụ năm 2026," 2025.
- [9] Z. Zhang, "A deep learning approach for detecting traffic accidents from social media data," 2018.
- [10] Z. Zhang, "Bắt kịp xu hướng của Thế giới, Việt Nam đã có nhiều bước tiến lớn về trí tuệ nhân tạo và ứng dụng công nghệ AI ở đa dạng các lĩnh vực. Năm 2024, BA GPS đã nghiên cứu, áp dụng và phát triển thành công Giải pháp BA-AI trong lĩnh vực giao thông vận tải.," 2024.
- [11] Y. LeCun, Bengio, Y., & Hinton, G. , "Deep learning," 2015.

- [12] Y. LeCun, "Gradient-Based Learning Applied to Document Recognition," 1998.
- [13] J. Redmon, Divvala, S., Girshick, R., & Farhadi, A. , " You only look once: Unified, real-time object detection," 2016.
- [14] S. LeCun, "Traffic sign recognition with multi-scale Convolutional Networks."
- [15] F. M. T. R. Kinasih, "Two-stage multiple object detection using CNN and correlative filter for accuracy improvement," 2023.
- [16] M. Veres, "Deep Learning for Intelligent Transportation Systems: A Survey of Emerging Trends," 2019.
- [17] T. Soukupová, & Čech, J. , " Real-time eye blink detection using facial landmarks," 2016.
- [18] S. Malek, "Head pose estimation using facial-landmarks classification for children rehabilitation games.," 2021.
- [19] R. O. Duda, & Hart, P. E., "Use of the Hough transformation to detect lines and curves in pictures.," 1972.
- [20] A. L. Maas, "Rectifier nonlinearities improve neural network acoustic models (LeakyReLU)," 2013.
- [21] "Explore Ultralytics YOLOv8 " *Ultralytics YOLO Docs*. [Online].
- [22] K. C. Q. Zhou), "Human Pose Prediction Based on LSTM and MediaPipe," *Khoa học và Công nghệ Kỹ thuật, Hóa học và Bảo vệ Môi trường*, 2025.
- [23] G. A. Edge, "Face landmark detection guide 2026-01-29. [Online].
- [24] MinhVT, "30 Thư Viện Python Tốt Nhất," *Beginner Cần Biết*, 17/08/2020.

TRƯỜNG KHOA
(ký và ghi rõ họ tên)

.....

Ngày ... tháng ... năm 20...
**GIÁO VIÊN HƯỚNG
DẪN**
(ký và ghi rõ họ tên)

.....

TL. HIỆU TRƯỞNG
TRƯỞNG PHÒNG KHCN – HTQT

.....